TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM

KHOA TOÁN - TIN HỌC

BÁO CÁO THỰC HÀNH

Môn học: Xử lý dữ liệu lớn

Tổng hợp các bài thực hành L01–L06

Sinh viên thực hiện: Nguyễn Thị Huyền Trang
Mã số sinh viên: 23280092
Lớp: 23KDL

TP. Hồ Chí Minh, Tháng 10 năm 2026

Mục lục

1 HDFS: tổ chức block, nhân bản và khả năng chịu lỗi DataNode **12**

1.1 Mục tiêu và môi trường . . . 12

1.2 Thực hiện và kết quả . . . 13

1.2.1 Khởi động cluster . . . 13

1.2.2 Bộ lệnh `hdfs dfs` . . . 15

1.2.3 Block size mặc định và block 16 MB . . . 16

1.2.4 Thay đổi hệ số nhân bản . . . 18

1.2.5 Dừng một DataNode và tái nhân bản . . . 19

1.2.6 Safe mode và fsimage . . . 24

1.3 Trả lời câu hỏi checkpoint . . . 25

1.3.1 Tệp 300 MB với cấu hình mặc định chiếm bao nhiêu block, replica và dung lượng? . . . 25

1.3.2 Vì sao block cuối của tệp taxi nhỏ hơn 16 MB, và có tốn 16 MB đĩa không? . . . 25

1.3.3 Vị trí block được lưu ở đâu, và khi NameNode khởi động lại thì sao? . . . 25

1.3.4 Vì sao tệp hệ số 2 được phục hồi còn tệp hệ số 3 thì không? . . . 25

1.3.5 `fsck` báo gì khi có block under-replicated, và vì sao đó chưa phải lỗi? . . . 26

1.3.6 Nếu giữ `recheck-interval` mặc định 5 phút thì sao, và vì sao timeout dài lại tốt? . . . 26

1.3.7 Vì sao NameNode khởi động trong safe mode và phải chặn ghi? . . . 26

1.3.8 Vì sao 1 triệu tệp 1 KB là vấn đề, còn 1 tệp 1 KB thì không? . . . 26

1.4 Kết luận . . . 27

Tài liệu tham khảo . . . 27

Phụ lục: Tệp nộp kèm và trích đoạn lệnh . . . 27

2 MapReduce với Hadoop Streaming, map-side join và combiner **29**

2.1 Mục tiêu và môi trường . . . 29

2.2 Thực hiện và kết quả . . . 30

2.2.1 Đếm từ, kiểm thử cục bộ . . . 30

2.2.2 Đếm từ trên YARN . . . 31

2.2.3 Chuẩn bị dữ liệu taxi và map-side join . . . 32

2.2.4 Có và không có combiner . . . 33
2.2.5 Giao diện YARN, JobHistory và CLI . . . 35
2.2.6 Mở rộng: giá cước trung bình theo khu vực . . . 40
2.3 Trả lời câu hỏi checkpoint . . . 41
2.3.1 Trong đường ống cục bộ, lệnh nào làm việc của shuffle, và thiếu nó thì sao? . . . 41
2.3.2 Vì sao reducer Streaming phải tự giữ `current_key`? . . . 41
2.3.3 Job taxi có bao nhiêu map task, bao nhiêu reduce task, và vì sao? . . . 41
2.3.4 Combiner giảm `Reduce shuffle bytes` bao nhiêu lần, và vì sao đầu ra combiner xấp xỉ số khu vực nhân số map task? . . . 41
2.3.5 Vì sao combiner không làm job nhanh hơn ở đây, và khi nào nó có ích rõ? 42
2.3.6 Có dùng lại reducer "trung bình" làm combiner được không? . . . 42
2.3.7 Map-side join trong bài hoạt động thế nào, và khi nào nó không còn hợp? 42
2.3.8 Counter `BAD_RECORDS` hiện ở đâu, và vì sao nên đếm dòng hỏng thay vì cho job dừng? . . . 42
2.4 Kết luận . . . 43
Tài liệu tham khảo . . . 43
Phụ lục: Trích đoạn mã nguồn . . . 43

3 Hive: truy vấn SQL trên HDFS, phân vùng, định dạng lưu trữ cột và bucketing 45
3.1 Mục tiêu và môi trường . . . 45
3.2 Thực hiện và kết quả . . . 45
3.2.1 Khởi động cụm và đưa CSV lên HDFS . . . 45
3.2.2 Bảng ngoài và nội dung Metastore . . . 47
3.2.3 Bảng phân vùng ORC và Parquet . . . 48
3.2.4 So sánh thời gian truy vấn CSV, ORC và Parquet . . . 52
3.2.5 Cắt tỉa phân vùng . . . 54
3.2.6 Bucketing . . . 55
3.2.7 Phần mở rộng: thống kê và bảng ACID . . . 56
3.3 Trả lời câu hỏi checkpoint . . . 59
3.3.1 Metastore giữ gì, HDFS giữ gì, và `DROP TABLE` xóa những gì? . . . 59
3.3.2 Vì sao `zones` cần `OpenCSVSerde` còn `trips_csv` chỉ cần `ROW FORMAT DELIMITED`? . . . 59
3.3.3 Vì sao ORC và Parquet nhỏ hơn CSV 4–5 lần? . . . 59
3.3.4 Vì sao Q1 chạy nhanh hơn trên ORC và Parquet? . . . 59
3.3.5 Phân vùng tĩnh khác phân vùng động ra sao, và vì sao mặc định là `strict`? 60
3.3.6 Phân vùng theo `pulocationid` cộng ngày, hoặc theo `tpep_pickup_datetime`, thì sao? . . . 60
3.3.7 Khi nào dùng partitioning, khi nào dùng bucketing, và vì sao số tệp bucket cố định? . . . 60
3.3.8 Những dòng nào rơi vào phân vùng lạ, và pipeline thật nên xử lý ra sao? 60

3.4 Kết luận . . . 61
Tài liệu tham khảo . . . 61
Phụ lục: Trích đoạn mã nguồn . . . 62

4 Nạp dữ liệu từ cơ sở dữ liệu OLTP vào HDFS: Sqoop và Spark JDBC, nạp toàn phần và nạp gia tăng 64
4.1 Mục tiêu và môi trường . . . 64
4.2 Thực hiện và kết quả . . . 66
4.2.1 Sqoop nạp toàn phần bảng `orders` . . . 66
4.2.2 Sqoop với cột chia bị lệch . . . 68
4.2.3 Sqoop nạp gia tăng, saved job, merge và export . . . 69
4.2.4 Spark JDBC: đọc song song và ghi Parquet . . . 72
4.2.5 Spark nạp gia tăng bằng watermark . . . 75
4.2.6 So sánh Sqoop và Spark JDBC . . . 78
4.3 Trả lời câu hỏi checkpoint . . . 78
4.3.1 Sqoop chạy câu SQL nào trước khi import, và từ đó ra 4 split bằng cách nào? . . . 78
4.3.2 Vì sao các file `customers` không đều, và nếu dùng `--split-by customer_id` thì sao? . . . 79
4.3.3 Với `lowerBound=1, upperBound=5003, numPartitions=4`, Spark có đọc dòng `order_seq` = 9,999 không? . . . 79
4.3.4 Vì sao phải chốt cận trên trước khi đọc, và chỉ lưu watermark sau khi ghi xong? . . . 79
4.3.5 Chế độ `append` hay `lastmodified` bắt được UPDATE, và chế độ nào bắt được DELETE? . . . 80
4.3.6 Watermark trên `updated_at` không giải quyết được gì, và CDC khác ở đâu? . . . 80
4.3.7 Điều gì xảy ra với cơ sở dữ liệu nguồn khi đặt `-m 50` hoặc `numPartitions=50`? . . . 80
4.4 Kết luận . . . 80
Tài liệu tham khảo . . . 81
Phụ lục: Tệp nộp kèm . . . 81

5 PySpark cơ bản: DataFrame, Spark SQL, kế hoạch thực thi và Spark UI 82
5.1 Mục tiêu và môi trường . . . 82
5.2 Thực hiện và kết quả . . . 84
5.2.1 Đọc dữ liệu và lazy evaluation . . . 84
5.2.2 Làm sạch dữ liệu và chế độ ANSI . . . 84
5.2.3 Join với bảng khu vực và tổng hợp . . . 86
5.2.4 Hàm cửa sổ . . . 87
5.2.5 Spark SQL và kế hoạch vật lý . . . 89

5.2.6 Spark UI . . . 91
5.2.7 Ghi Parquet phân vùng và partition pruning . . . 99
5.3 Trả lời câu hỏi checkpoint . . . 101
5.3.1 Vì sao tệp 48 MB lại có 4 partition đầu vào? . . . 101
5.3.2 Dòng nào là transformation, dòng nào là action, và sao dựng plan nhanh vậy? . . . 102
5.3.3 `by_borough.show()` sinh bao nhiêu job, stage, và stage bị cắt ở đâu? . . . 102
5.3.4 Vì sao năm cột có cùng 140,162 NULL, và nên xử lý thế nào? . . . 102
5.3.5 `HashAggregate` partial trước `Exchange` làm gì, ứng với khái niệm nào ở L02? . . . 103
5.3.6 Vì sao bảng khu vực được broadcast, và khi nào Spark chuyển sang sort-merge join? . . . 103
5.3.7 `cache()` thay đổi gì trong plan và UI, và khi nào không nên cache? . . . 103
5.3.8 Vì sao cần `repartition("pickup_date")` trước `partitionBy("pickup_date")`? . . . 104
5.4 Kết luận . . . 104
Tài liệu tham khảo . . . 104
Phụ lục: Trích đoạn mã nguồn . . . 105

6 Tối ưu hiệu năng Spark: shuffle, chiến lược join, data skew, AQE, phân vùng và cache 106
6.1 Mục tiêu và môi trường . . . 106
6.2 Thực hiện và kết quả . . . 107
6.2.1 E1: số partition shuffle và AQE coalescing . . . 107
6.2.2 E2: broadcast hash join và sort-merge join . . . 109
6.2.3 E3: data skew, salting và AQE skew join . . . 112
6.2.4 E4: repartition và coalesce trước khi ghi . . . 117
6.2.5 E5: cache . . . 118
6.2.6 Bảng kết quả và độ ổn định . . . 118
6.3 Trả lời câu hỏi checkpoint . . . 123
6.3.1 Vì sao 200 partition chậm, và vì sao không nên đặt 1 partition dù nó chạy nhanh? . . . 123
6.3.2 `AQEShuffleRead coalesced` làm gì, và AQE biết gì mà planner tĩnh không biết? . . . 124
6.3.3 Con số nào cho thấy skew, vì sao salting không rút ngắn thời gian tổng trên 4 core, và khi nào salting thắng? . . . 124
6.3.4 AQE nhận ra partition lệch thế nào, xử lý ra sao, và kém salting ở đâu? 124
6.3.5 Khi nào không nên broadcast dù một phía "nhỏ"? . . . 125
6.3.6 `repartition(n)` hay `coalesce(n)`: cái nào shuffle, cái nào tạo file lệch, cái nào làm giảm song song stage trước? . . . 125

6.3.7 Vì sao cache có ích trong E5, còn cache một DataFrame rẻ thì thường không? . . . 125
6.3.8 Nên tinh chỉnh gì trước khi thêm phần cứng? . . . 125
6.4 Khuyến nghị cho job production trên cluster 20 node . . . 126
6.5 Kết luận . . . 126
Tài liệu tham khảo . . . 126
Phụ lục: Trích đoạn mã nguồn . . . 127

Danh mục hình

1.1 Trang Overview của NameNode khi cluster khỏe mạnh: 3 Live Nodes, 0 Dead Nodes, 0 block under-replicated. . . . 14
1.2 Trang Datanodes của NameNode khi cả ba DataNode đều ở trạng thái In service. 15
1.3 Đầu ra `dfsadmin -report` (3 DataNode sống, ánh xạ IP sang hostname), chuỗi lệnh `hdfs dfs` và kết quả `hdfs dfs -stat`. . . . 16
1.4 Kết quả `fsck -files -blocks -locations` của tệp Parquet với block mặc định (1 block) và block 16 MB (3 block). . . . 17
1.5 Trang Browse Directory hiển thị thư mục `/data/taxi` với hệ số nhân bản và block size của từng mục. . . . 17
1.6 Tệp block vật lý `blk_<id>` và `blk_<id>_<genstamp>.meta` trong thư mục dữ liệu của datanode1. . . . 18
1.7 Kết quả `setrep -w 2`, `du -h` sau khi giảm hệ số và tệp được ghi với hệ số nhân bản 1. . . . 19
1.8 `fsck` của tệp block 16 MB trước khi dừng datanode3: mỗi block có 2 replica trên datanode1 và datanode3. . . . 19
1.9 Dòng thời gian thăm dò `dfsadmin -report` sau khi dừng datanode3 và dòng log `removeDeadDatanode` của NameNode. . . . 20
1.10 Trang Overview của NameNode khi datanode3 đã bị tuyên bố chết. . . . 21
1.11 Trang Datanodes khi datanode3 chết: datanode1 và datanode2 In service, datanode3 ở trạng thái Down. . . . 22
1.12 `fsck` của tệp block 16 MB sau khi dừng datanode3: replica đã chuyển sang datanode2, tệp trở lại trạng thái OK. . . . 22
1.13 `fsck /data` khi datanode3 chết: hai tệp hệ số 3 under-replicated nhưng HEALTHY; md5 của tệp tải về trùng bản gốc. . . . 23
1.14 Trạng thái cluster sau khi datanode3 quay lại: không còn block under-replicated hay over-replicated. . . . 23
1.15 Safe mode thủ công (ghi bị từ chối, đọc vẫn hoạt động), safe mode tự động khi khởi động lại và nội dung fsimage. . . . 24

2.1 Kiểm thử cục bộ đường ống `cat | mapper | sort | reducer` trên bốn cuốn sách: top 5 từ và số dòng `the` khi bỏ `sort`. . . . 31

2.2 Counter cuối job `wordcount` trên YARN: 4 map task, `Reduce input groups=20967` và top 5 từ đọc từ HDFS. . . . 32
2.3 Chuyển Parquet sang CSV, 6 block trên HDFS, kiểm thử cục bộ map-side join trên 1,000 dòng và counter `BAD_RECORDS`. . . . 33
2.4 Trích các counter so sánh từ hai log và top 10 khu vực đón khách; `diff` giữa hai đầu ra cho kết quả IDENTICAL. . . . 35
2.5 YARN ResourceManager UI (cổng 8088): ba ứng dụng MapReduce đã hoàn thành và các chỉ số cluster. . . . 36
2.6 JobHistory UI (cổng 19888): danh sách các job đã hoàn thành. . . . 36
2.7 JobHistory UI: tổng quan job `zones_combiner` với 6 map task, 2 reduce task và thời gian từng pha. . . . 37
2.8 JobHistory UI: trang Counters của job không combiner. . . . 38
2.9 JobHistory UI: trang Counters của job có combiner. . . . 39
2.10 Kiểm tra các job đã hoàn thành bằng `yarn application -list` và `mapred job -list all`. . . . 40
2.11 Job tính giá cước trung bình theo khu vực với combiner `(sum, count)` và đối chiếu JFK với bản chạy không combiner. . . . 40

3.1 Khởi động cụm Hive (`docker compose ps`) và tải tệp CSV cùng bảng tra cứu zone lên HDFS. . . . 46
3.2 Giao diện web HiveServer2 (cổng 10002): danh sách truy vấn đã hoàn tất, tất cả chạy trên Tez. . . . 47
3.3 Kết quả truy vấn bảng ngoài và nội dung Metastore (PostgreSQL). . . . 48
3.4 `SHOW PARTITIONS` và số chuyến theo `pickup_month`. . . . 49
3.5 Thư mục warehouse `/user/hive/warehouse/taxi.db` trên giao diện NameNode. . . . 49
3.6 Năm thư mục phân vùng của bảng `trips_orc` trên giao diện NameNode. . . . 50
3.7 `DESCRIBE FORMATTED trips_orc` và 18 dòng nằm ngoài tháng 01/2024. . . . 51
3.8 Kích thước CSV, ORC và Parquet trên HDFS. . . . 52
3.9 Lần chạy thứ hai của `03_compare_formats.sql`. . . . 53
3.10 Kết quả `EXPLAIN DEPENDENCY` và `EXPLAIN EXTENDED`. . . . 55
3.11 Kết quả bucketing: mô tả bảng, lấy mẫu một bucket và vị trí các dòng JFK. . . . 56
3.12 Bốn tệp bucket của `trips_bucketed` trên giao diện NameNode. . . . 56
3.13 Phần mở rộng: thống kê cột và các thao tác ACID. . . . 58
3.14 Các thư mục `delta_*` và `delete_delta_*` của bảng `zone_notes` trên giao diện NameNode. . . . 58

4.1 Trạng thái container của stack L04 và 6 bảng nguồn trong PostgreSQL. . . . 65
4.2 Sqoop nạp toàn phần `orders`: bounding query, 4 split và số dòng mỗi file. . . . 67
4.3 Nhật ký PostgreSQL ghi lại bounding query và 4 câu SELECT theo khoảng `order_seq` do các map task của Sqoop gửi. . . . 67

4.4 JobHistory của job `job_1793356967101_0001` (`orders.jar`): 4 map task thành công, không có reduce task. . . . 68
4.5 Số dòng mỗi file khi nạp `customers` chia theo `customer_zip_code_prefix`, thể hiện dữ liệu bị lệch. . . . 69
4.6 Sqoop nạp `append` lần đầu, nạp `lastmodified` cơ sở và tạo saved job `orders_incr`. . . . 70
4.7 Sau `simulate_changes` lần 1: saved job, `lastmodified` kèm merge và export về PostgreSQL. . . . 71
4.8 YARN ResourceManager UI: các MapReduce job do Sqoop nộp, tất cả ở trạng thái FINISHED/SUCCEEDED. . . . 71
4.9 JobHistory Server liệt kê các job Sqoop đã hoàn tất. . . . 72
4.10 Nhật ký PostgreSQL ghi lại các câu `SELECT 1 FROM orders WHERE ...` theo khoảng do Spark sinh ra. . . . 73
4.11 Kết quả `spark-submit --step full`, gồm physical plan có `PartitionFilters` . . . 74
4.12 Spark UI, trang Jobs của ứng dụng `L04-jdbc-full`. . . . 74
4.13 Spark UI, trang Stages: các stage đọc JDBC song song và stage ghi Parquet đều có 4 task. . . . 75
4.14 Đầu ra Parquet phân vùng theo tháng trên HDFS và so sánh dung lượng với đầu ra văn bản của Sqoop. . . . 75
4.15 Ba lần chạy Spark `--step incremental`. . . . 77

5.1 Tab Environment của Spark UI: Java 17.0.20.1, Scala 2.13.17, `spark.driver.memory=2g`, event log được bật. . . . 83
5.2 Đọc dữ liệu: schema 19 cột, 2,964,624 dòng, 4 partition; dựng ba transformation mất 0.096 s, `count()` mất 0.28 s. . . . 84
5.3 Đếm NULL theo cột và số dòng vi phạm từng luật làm sạch; giữ lại 2,867,786 dòng (96.73 %). . . . 85
5.4 Chế độ ANSI: `try_divide` trả NULL, phép chia thường ném lỗi `[DIVIDE_BY_ZERO]`. . . . 86
5.5 Kết quả join hai lần với bảng khu vực, tổng hợp theo quận và theo giờ đón. . . . 87
5.6 Hàm cửa sổ: top-3 khu vực đón mỗi quận và chuỗi theo ngày với `lag`, tổng lũy kế. 89
5.7 Kết quả Spark SQL trùng khớp DataFrame và cây kế hoạch vật lý của `by_borough`. 90
5.8 Phần kế hoạch thể hiện `HashAggregate` partial, `Exchange hashpartitioning` và `HashAggregate` final. . . . 91
5.9 Tab Jobs: mỗi action sinh một hoặc nhiều job. . . . 92
5.10 Tab SQL/DataFrame: 21 truy vấn đã hoàn thành cùng thời lượng và danh sách job. . . . 93
5.11 Đồ thị kế hoạch truy vấn 10 trong tab SQL: 2,867,786 dòng vào, 24 dòng ra khỏi `HashAggregate` partial. . . . 95
5.12 Job 21: một stage, 4/4 task, Input 347.3 MiB từ cache, Shuffle Write 2.3 KiB. . . . 96

5.13 Job 22: stage 28 bị bỏ qua, stage 29 chạy 1 task với Shuffle Read 2.3 KiB. . . . 97
5.14 Tab Stages: 60 stage hoàn thành, 28 stage bị bỏ qua, kèm Input và Shuffle Read/Write. . . . 98
5.15 Tab Storage: `clean` được cache 4/4 partition, 100 %, 347.3 MiB trong bộ nhớ. . 99
5.16 Ghi Parquet phân vùng, đọc lại ngày 2024-01-15 (74,743 chuyến) và `PartitionFilters` trong kế hoạch. . . . 100
5.17 Đồ thị truy vấn 18 (ghi Parquet): `Exchange` do `repartition` shuffle toàn bộ 2,867,786 dòng. . . . 101

6.1 Kế hoạch E1 khi AQE tắt: `Exchange` với 200 partition shuffle. . . . 108
6.2 Kế hoạch E1 khi AQE bật: `Exchange` 200 partition theo sau là `AQEShuffleRead` ở chế độ coalesced; truy vấn đọc 2,964,624 dòng, 47.6 MiB. . . . 109
6.3 Kế hoạch E2 với broadcast: `BroadcastExchange` và `BroadcastHashJoin`, phía trips không có `Exchange` trước join. . . . 110
6.4 Kế hoạch E2 với ngưỡng -1: `Exchange` và `Sort` ở cả hai phía trước `SortMergeJoin`. . . . 111
6.5 Phân vị thời gian task và số bản ghi shuffle mỗi task của ba stage join E3, lấy từ Spark REST API. . . . 112
6.6 Kế hoạch E3 baseline (AQE tắt): sort-merge join trên dữ liệu lệch. . . . 113
6.7 Summary Metrics của stage 42 (baseline): max 61.9 MiB / 6,264,438 bản ghi so với median 3.4 MiB / 262,619 bản ghi; task index 13 mất 2 s. . . . 114
6.8 Summary Metrics của stage 46 (salting 64): phân bố bản ghi đều hơn, max 917,267 bản ghi, nhưng median thời gian task tăng lên 590 ms. . . . 115
6.9 Kế hoạch E3 với AQE skew join: `AQEShuffleRead` báo 1 partition lệch được tách thành 4 split (16 thành 19 partition), sau đó là `SortMergeJoin(skew=true)`. . . . 116
6.10 Summary Metrics của stage 52 (AQE skew join): max 682 ms và 1,576,365 bản ghi mỗi task. . . . 117
6.11 Tab Storage của Spark UI: DataFrame `cleaned` được cache 8/8 partition, 100 %, 59.4 MiB trong bộ nhớ. . . . 118
6.12 Đầu ra console của lần chạy thứ hai: thời gian, số task, median/max task và các nút kế hoạch của từng biến thể. . . . 119
6.13 Bảng kết quả Markdown do script in ra ở cuối lần chạy thứ hai. . . . 119
6.14 Tab SQL / DataFrame của Spark UI với 31 truy vấn đã hoàn tất trong phiên chạy. 122

Danh mục bảng

1.1 Phần cứng, phần mềm và cấu hình HDFS dùng trong thực nghiệm. 12
1.2 Ánh xạ địa chỉ IP sang hostname của các DataNode. 13
1.3 Số block của cùng một tệp khi dùng block 128 MB và 16 MB. 16
1.4 Các tệp block vật lý trên datanode1 và tệp checksum đi kèm. 18
1.5 Dung lượng logic và dung lượng thô theo `hdfs dfs -du -h` sau khi `setrep 2`. 18
1.6 Diễn biến sau khi dừng datanode3 theo `dfsadmin -report` và log NameNode. 20
1.7 Vị trí replica của tệp `small_blocks` (hệ số 2) trước và sau khi dừng datanode3. 22
1.8 Trạng thái các tệp trong `/data` khi chỉ còn 2 DataNode sống. 23
1.9 Danh mục tệp log nộp kèm báo cáo. 27

2.1 Môi trường thực nghiệm. 29
2.2 Dữ liệu sử dụng trong thực nghiệm. 30
2.3 Counter của job đếm từ trên YARN. 31
2.4 So sánh counter giữa hai lần chạy không và có combiner. 34
2.5 Counter bổ sung của hai lần chạy khu vực (trích từ `nocombiner.log` và `combiner.log`). 34
2.6 Giá cước trung bình (`fare_amount`) của 10 khu vực đón khách nhiều chuyến nhất. 40
2.7 Danh mục tệp nộp kèm trong `submission/L02/`. 43

3.1 Môi trường thực nghiệm. 45
3.2 Dữ liệu đầu vào. 46
3.3 Nội dung bảng `TBLS` và `SDS` trong Metastore sau bước tạo bảng ngoài. 48
3.4 Phân vùng của `trips_orc` sau khi nạp bằng phân vùng động. 48
3.5 Kích thước lưu trữ cùng dữ liệu tháng 01/2024 theo định dạng. 52
3.6 Thời gian truy vấn theo định dạng (số liệu dùng để so sánh là lần chạy thứ hai). 52
3.7 Kết quả Q1 (giống nhau trên CSV, ORC và Parquet). 54
3.8 Kết quả Q2: số chuyến và doanh thu theo borough. 54
3.9 Kết quả kiểm chứng cắt tỉa phân vùng. 54
3.10 Các tệp bucket của `trips_bucketed`. 55
3.11 Các giao dịch trên bảng ACID `zone_notes` và thư mục delta tương ứng. 57
3.12 Các dòng có thời gian đón khách nằm ngoài tháng 01/2024. 61

3.13 Danh mục tệp nộp kèm trong `submission/L03/`. 62

4.1 Môi trường thực nghiệm (stack `l04_ingestion`). 64
4.2 Phân bố trạng thái của 5,000 đơn hàng ban đầu. 65
4.3 Các split của Sqoop khi nạp `orders` với `--split-by order_seq -m 4`. . . . 66
4.4 Số dòng mỗi file khi chia theo cột phân bố đều và cột bị lệch. 68
4.5 Phân bố `customer_zip_code_prefix` theo `width_bucket(..., 1003, 99990, 4)`. 69
4.6 Kết quả các bước nạp gia tăng, merge và export của Sqoop. 69
4.7 So sánh các cách đọc bảng `orders` bằng Spark JDBC. 72
4.8 Số dòng các bảng phụ được Spark nạp sang Parquet. 73
4.9 Kết quả ba lần chạy Spark nạp gia tăng (thời điểm ngày 2026-10-30). 76
4.10 So sánh Sqoop và Spark JDBC khi nạp bảng `orders`. 78
4.11 Danh mục tệp nộp kèm báo cáo L04. 81

5.1 Môi trường thực nghiệm và cấu hình Spark. 82
5.2 Dữ liệu đầu vào. 83
5.3 Số dòng vi phạm từng luật làm sạch (một dòng có thể vi phạm nhiều luật). . . . 84
5.4 Tổng hợp theo quận đón sau làm sạch (`by_borough`). 86
5.5 Top-3 khu vực đón tại Manhattan và Queens. 88
5.6 Số chuyến và doanh thu theo ngày với `lag` và tổng lũy kế (10 ngày đầu). 88
5.7 Một số truy vấn trong tab SQL (log `ui_sql_queries.txt`). 93
5.8 Các job và stage của truy vấn 10 (`by_borough.show()`). 94
5.9 So sánh ghi Parquet phân vùng theo ngày có và không có `repartition`. 99

6.1 Môi trường thực nghiệm. 106
6.2 Dữ liệu thực nghiệm. 107
6.3 Phân vị min / 25 % / median / 75 % / max tại stage join của E3 (lần chạy thứ hai). 112
6.4 Kết quả đầy đủ của lần chạy thứ hai (17 biến thể). 119
6.5 So sánh một số chỉ số giữa hai lần chạy. 122
6.6 So sánh với bảng kết quả tham chiếu của bài lab. 123
6.7 Danh mục tệp nộp kèm. 127

Chương 1

HDFS: tổ chức block, nhân bản và khả năng chịu lỗi DataNode

1.1 Mục tiêu và môi trường

Bài lab dựng một cluster HDFS nhiều node bằng Docker Compose, dùng bộ lệnh `hdfs dfs`, dùng `hdfs fsck -files -blocks -locations` để xem tệp được chia thành block và replica, thay đổi block size và hệ số nhân bản, dừng một DataNode để quan sát under-replication và tái nhân bản, cuối cùng thử safe mode và đọc fsimage.

Cluster chạy trên một máy (Bảng 1.1). `recheck-interval` được rút xuống 30 s để thấy sự cố DataNode trong khoảng 90 s thay vì hơn 10 phút. Mọi lệnh chạy trong container `namenode` (đóng vai trò edge node), trừ `docker compose` chạy trên host.

Bảng 1.1. Phần cứng, phần mềm và cấu hình HDFS dùng trong thực nghiệm.

Thành phần	Giá trị
Máy chủ	WSL2, x86_64, 12 CPU, bộ nhớ Docker 31.9 GB
Phần mềm	Hadoop 3.4.3 (image `de-labs/hadoop:3.4.3`), Docker Compose
Dịch vụ	1 NameNode, 3 DataNode, ResourceManager, NodeManager, JobHistory
Topology	1 rack (`/default-rack`)
`dfs.replication`	3
`dfs.blocksize`	134,217,728 byte (128 MB, mặc định)
`dfs.heartbeat.interval`	3 s
`dfs.namenode.heartbeat.recheck-interval`	30,000 ms (mặc định 300,000 ms)

Thành phần	Giá trị
`dfs.namenode.stale.datanode.interval`	15,000 ms (mặc định 30,000 ms)
`dfs.namenode.redundancy.interval.seconds`	3 s
`dfs.namenode.safemode.extension`	5,000 ms (mặc định 30,000 ms)
`dfs.permissions.enabled`	false (chỉ dùng cho môi trường học tập)

Dữ liệu là hai tệp NYC Taxi đọc từ thư mục `/datasets` bind mount vào container: `taxi_zone_lookup.csv` (12,331 byte) và `yellow_tripdata_2024-01.parquet` (49,961,641 byte, `hdfs dfs -du -h` hiển thị 47.6 M).

1.2 Thực hiện và kết quả

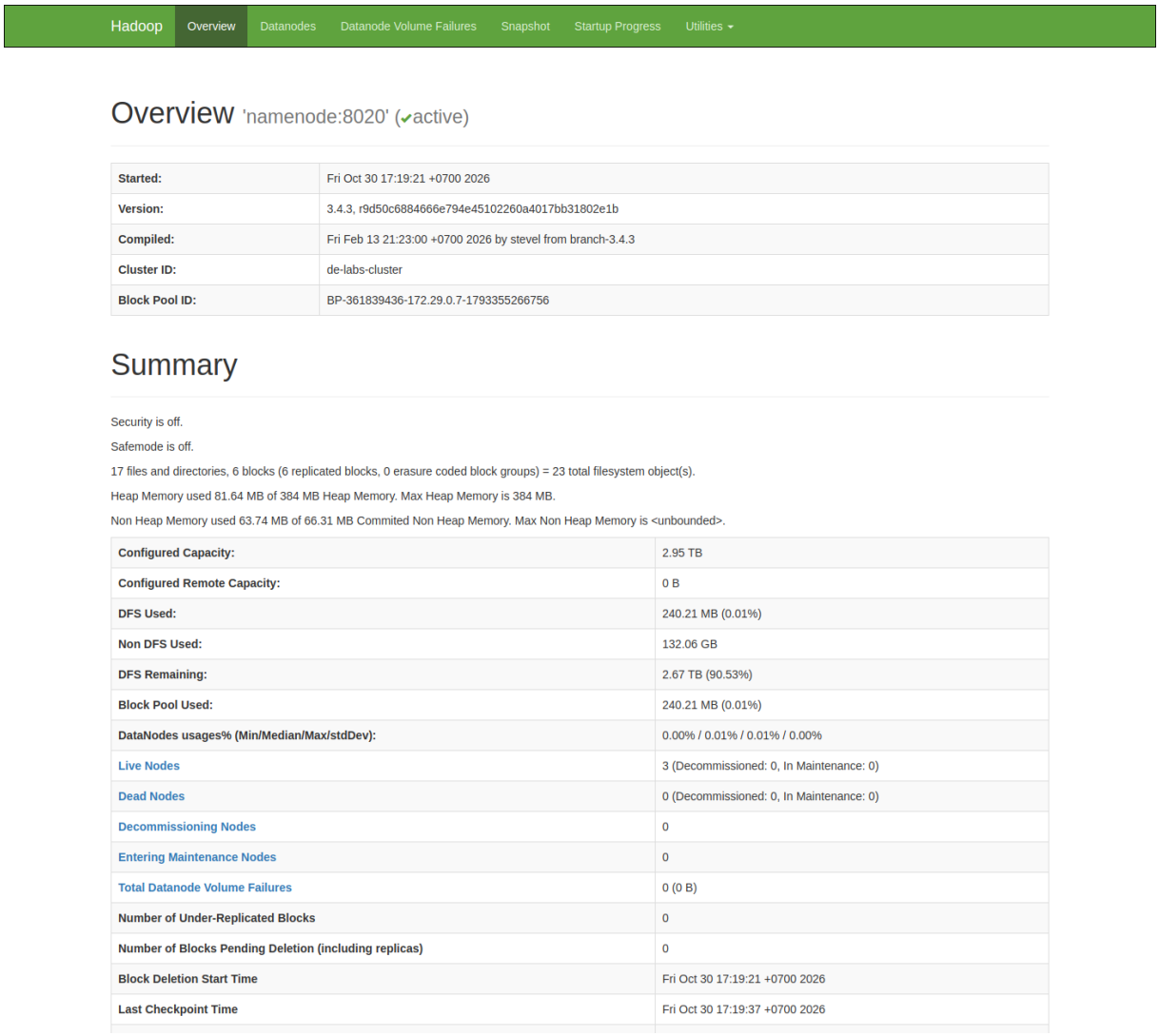
1.2.1 Khởi động cluster

Sau khi khởi động, `hdfs dfsadmin -report` ghi nhận `Live datanodes (3)`, safe mode OFF. Vì `fsck` in vị trí replica theo IP, chúng tôi ghi lại ánh xạ IP sang hostname ngay lúc này (Bảng 1.2). Block Pool ID `BP-361839436-172.29.0.7-1793355266756` cho biết NameNode ở 172.29.0.7.

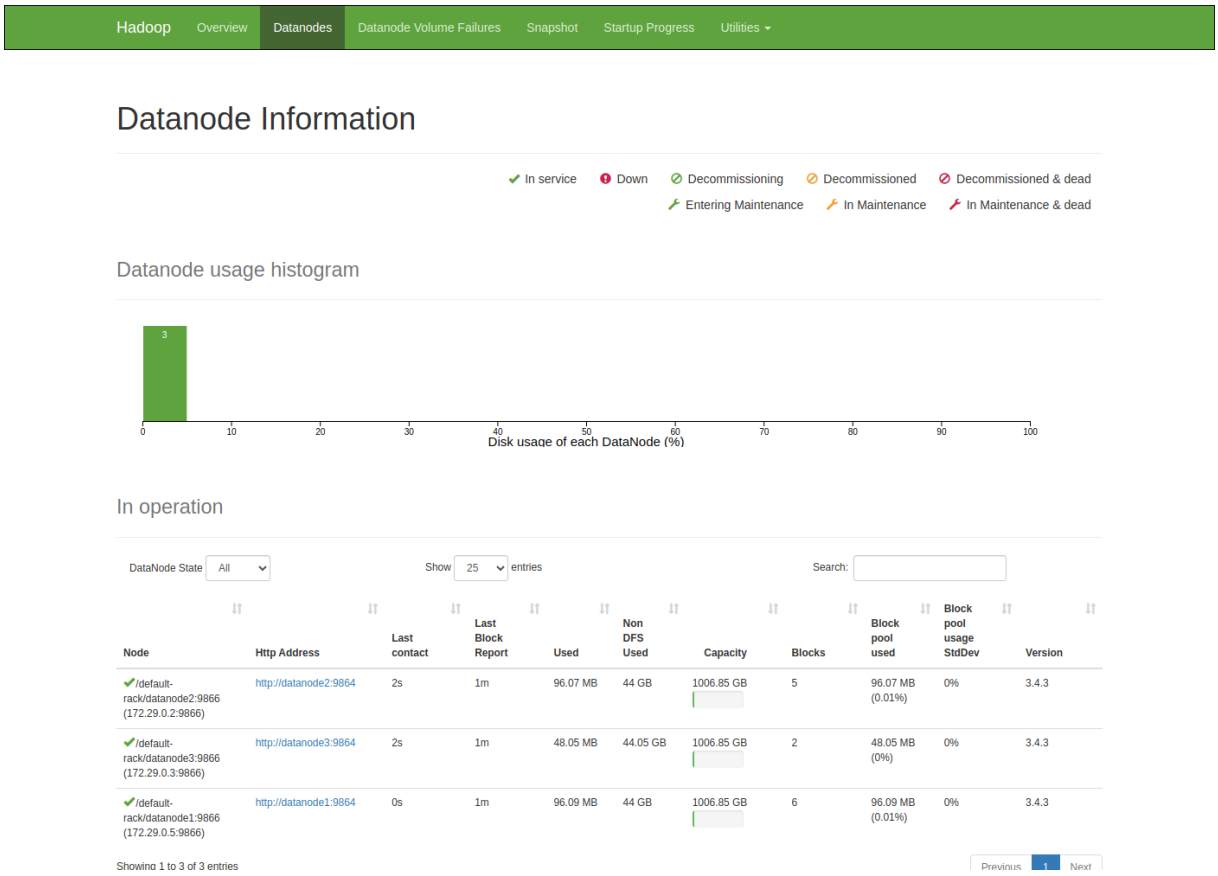
Bảng 1.2. Ánh xạ địa chỉ IP sang hostname của các DataNode.

Địa chỉ IP	Hostname
172.29.0.5	datanode1
172.29.0.2	datanode2
172.29.0.3	datanode3

Trên giao diện web cổng 9870, trang Overview báo NameNode `active`, phiên bản 3.4.3, safe mode tắt, 3 node sống, 0 node chết, 0 block under-replicated (Hình 1.1). Trang Datanodes liệt kê cả ba node ở trạng thái In service (Hình 1.2).



Hình 1.1. Trang Overview của NameNode khi cluster khỏe mạnh: 3 Live Nodes, 0 Dead Nodes, 0 block under-replicated.



Hình 1.2. Trang Datanodes của NameNode khi cả ba DataNode đều ở trạng thái In service.

1.2.2 Bộ lệnh `hdfs dfs`

Các lệnh `hdfs dfs` đều chạy được (Hình 1.3). `taxi_zone_lookup.csv` có hệ số 3 (cột thứ hai của `ls`), `cat` kết hợp `grep -c` đếm được 70 dòng Manhattan, bản tải về `/tmp/zones_copy.csv` đúng 12,331 byte. `du -h` in 12.0 K dung lượng logic và 36.1 K dung lượng thô, gấp ba do nhân bản. Lệnh `hdfs dfs -stat` (Phụ lục) trả về `blocksize=134217728 repl=3 size=12331`: tệp 12 KB vẫn được gán block size 128 MB.

```
root@namenode: hdfs dfsadmin -report / hdfs dfs ... / TODO 2.1
Safe mode is OFF
### 1. dfsadmin -report
Live datanodes (3):
Name: 172.29.0.2:9866 (l01_hdfs-datanode2-1.l01_hdfs_default)
Hostname: datanode2
Name: 172.29.0.3:9866 (l01_hdfs-datanode3-1.l01_hdfs_default)
Hostname: datanode3
Name: 172.29.0.5:9866 (l01_hdfs-datanode1-1.l01_hdfs_default)
Hostname: datanode1
### 2. dfs shell
Found 1 items
-rw-r--r--   3 root supergroup      12331 2026-10-30 10:14 /data/taxi/taxi_zone_lookup.csv
Manhattan rows: 70
-rw-r--r-- 1 root root 12331 Oct 30 10:15 /tmp/zones_copy.csv
Deleted /user/root/zones_copy.csv
12.0 K  36.1 K  /data/taxi/taxi_zone_lookup.csv
### TODO 2.1
name=taxi_zone_lookup.csv blocksize=134217728 repl=3 size=12331
```

Hình 1.3. Đầu ra `dfsadmin -report` (3 DataNode sống, ánh xạ IP sang hostname), chuỗi lệnh `hdfs dfs` và kết quả `hdfs dfs -stat`.

1.2.3 Block size mặc định và block 16 MB

Tệp Parquet được tải lên hai lần: block mặc định vào `/data/taxi/default_blocks` và `-D dfs.blocksize=16m` vào `/data/taxi/small_blocks`. `hdfs dfs -stat` cho bản thứ nhất `blocksize=134217728`, bản thứ hai `blocksize=16777216`, cả hai `repl=3`. Block size chỉ là giới hạn trên, block cuối chỉ lớn bằng phần dữ liệu còn lại [3]. Kết quả `fsck` ở Bảng 1.3 và Hình 1.4; trang Browse Directory cũng hiển thị `default_blocks`, `small_blocks` và `taxi_zone_lookup.csv` trong `/data/taxi` với Replication 3, Block Size 128 MB (Hình 1.5).

Bảng 1.3. Số block của cùng một tệp khi dùng block 128 MB và 16 MB.

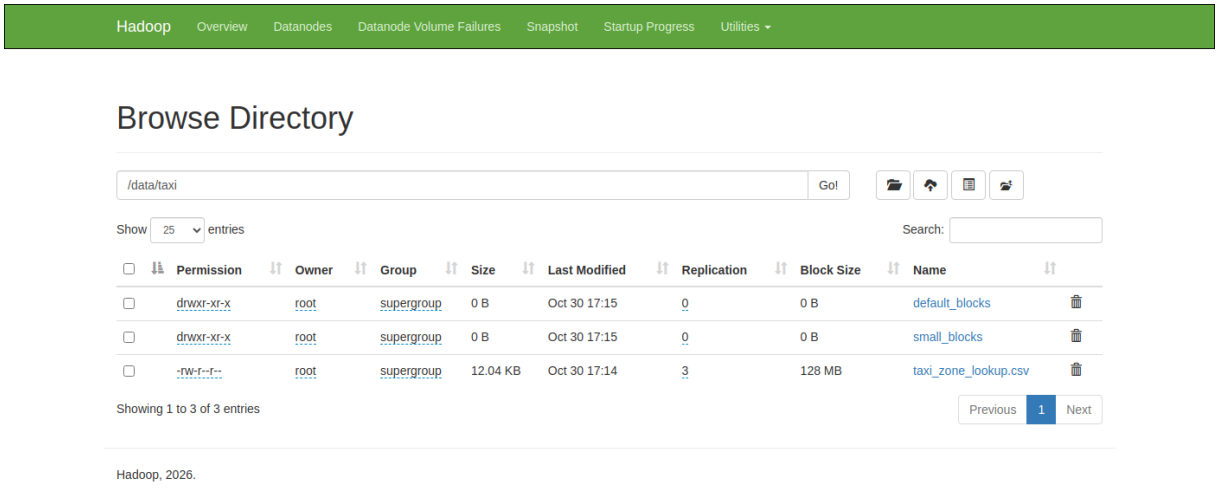
Cách tải lên	Block size (byte)	Số block	Kích thước từng block (byte)
Mặc định	134,217,728	1	49,961,641
`-D dfs.blocksize=16m`	16,777,216	3	16,777,216 + 16,777,216 + 16,407,209

```
hdfs fsck -files -blocks -locations  (128 MB vs 16 MB blocks)

### 3. blocks
yellow_tripdata_2024-01.parquet blocksize=134217728 repl=3 size=49961641
yellow_tripdata_2024-01.parquet blocksize=16777216 repl=3 size=49961641
--- fsck default_blocks
/data/taxi/default_blocks <dir>
/data/taxi/default_blocks/yellow_tripdata_2024-01.parquet 49961641 bytes, replicated: replication=3, 1 block(s):  OK
0. BP-361839436-172.29.0.7-1793355266756:blk_1073741827_1003 len=49961641 Live_repl=3
  [DatanodeInfoWithStorage[172.29.0.3:9866,DS-40f5e434-77f4-46f4-b595-2cea416937fe,DISK],
  DatanodeInfoWithStorage[172.29.0.5:9866,DS-a13511a4-4968-49b1-af9d-4b33712c6021,DISK], DatanodeInfoWithStorage[172.29.0.2:9866,DS-
  abdc23f4-4959-47fa-b34b-0829bd838ae0,DISK]]
Status: HEALTHY

### fsck small_blocks rep3 (settled)
/data/taxi/small_blocks <dir>
/data/taxi/small_blocks/yellow_tripdata_2024-01.parquet 49961641 bytes, replicated: replication=3, 3 block(s):  OK
0. BP-361839436-172.29.0.7-1793355266756:blk_1073741828_1004 len=16777216 Live_repl=3  [DatanodeInfoWithStorage[172.29.0.2:9866,DS-
  abdc23f4-4959-47fa-b34b-0829bd838ae0,DISK], DatanodeInfoWithStorage[172.29.0.3:9866,DS-40f5e434-77f4-46f4-b595-2cea416937fe,DISK],
  DatanodeInfoWithStorage[172.29.0.5:9866,DS-a13511a4-4968-49b1-af9d-4b33712c6021,DISK]]
1. BP-361839436-172.29.0.7-1793355266756:blk_1073741829_1005 len=16777216 Live_repl=3
  [DatanodeInfoWithStorage[172.29.0.3:9866,DS-40f5e434-77f4-46f4-b595-2cea416937fe,DISK],
  DatanodeInfoWithStorage[172.29.0.5:9866,DS-a13511a4-4968-49b1-af9d-4b33712c6021,DISK], DatanodeInfoWithStorage[172.29.0.2:9866,DS-
  abdc23f4-4959-47fa-b34b-0829bd838ae0,DISK]]
2. BP-361839436-172.29.0.7-1793355266756:blk_1073741830_1006 len=16407209 Live_repl=3
  [DatanodeInfoWithStorage[172.29.0.3:9866,DS-40f5e434-77f4-46f4-b595-2cea416937fe,DISK],
  DatanodeInfoWithStorage[172.29.0.5:9866,DS-a13511a4-4968-49b1-af9d-4b33712c6021,DISK], DatanodeInfoWithStorage[172.29.0.2:9866,DS-
  abdc23f4-4959-47fa-b34b-0829bd838ae0,DISK]]
Status: HEALTHY
 Under-replicated blocks:       0 (0.0 %)
```

Hình 1.4. Kết quả `fsck -files -blocks -locations` của tệp Parquet với block mặc định (1 block) và block 16 MB (3 block).



Hình 1.5. Trang Browse Directory hiển thị thư mục `/data/taxi` với hệ số nhân bản và block size của từng mục.

Ở lần `fsck` đầu tiên trên `small_blocks` (10:15:21, ngay sau khi ghi), hai block `blk_1073741829` và `blk_1073741830` chỉ có `Live_repl=2`, kèm `Under-replicated blocks: 2 (66.666664 %)` và `Missing replicas: 2 (22.222221 %)`, dù trạng thái vẫn `HEALTHY`. Lúc đầu chúng tôi nghĩ việc ghi bị lỗi, nhưng lần `fsck` tiếp theo (đầu tệp `02_setrep.txt`) đã cho cả ba block `Live_repl=3` và `Under-replicated blocks: 0`. NameNode chỉ cập nhật vị trí replica khi DataNode báo về, nên sau khi ghi có một khoảng trễ ngắn.

Trên datanode1, mỗi block là một tệp `blk_<id>` kèm tệp `blk_<id>_<genstamp>.meta` chứa checksum (Hình 1.6, Bảng 1.4). Block cuối chỉ có 16,407,209 byte, không bị đệm lên 16 MB. Tệp `.meta` 131,079 byte của block 16,777,216 byte khớp với 4 byte CRC cho mỗi 512 byte dữ

liệu ($16{,}777{,}216/512 \times 4 = 131{,}072$) cộng 7 byte header, tức chi phí checksum khoảng 0.78 %.

Bảng 1.4. Các tệp block vật lý trên datanode1 và tệp checksum đi kèm.

Tệp block	Dữ liệu (byte)	`.meta` (byte)	Thuộc tệp
`blk_1073741825`	12,331	107	`taxi_zone_lookup.csv`
`blk_1073741827`	49,961,641	390,335	`default_blocks/...parquet`
`blk_1073741828`	16,777,216	131,079	`small_blocks/...parquet`, block 0
`blk_1073741829`	16,777,216	131,079	`small_blocks/...parquet`, block 1
`blk_1073741830`	16,407,209	128,191	`small_blocks/...parquet`, block 2

```
datanode1: find /hadoop/dfs/data -name 'blk_*'

-rw-r--r-- 1 root root 128191 Oct 30 10:15 blk_1073741830_1006.meta
-rw-r--r-- 1 root root 16407209 Oct 30 10:15 blk_1073741830
-rw-r--r-- 1 root root 16777216 Oct 30 10:15 blk_1073741829
-rw-r--r-- 1 root root 131079 Oct 30 10:15 blk_1073741829_1005.meta
-rw-r--r-- 1 root root 107 Oct 30 10:14 blk_1073741825_1001.meta
-rw-r--r-- 1 root root 49961641 Oct 30 10:15 blk_1073741827
-rw-r--r-- 1 root root 12331 Oct 30 10:14 blk_1073741825
-rw-r--r-- 1 root root 16777216 Oct 30 10:15 blk_1073741828
-rw-r--r-- 1 root root 390335 Oct 30 10:15 blk_1073741827_1003.meta
-rw-r--r-- 1 root root 131079 Oct 30 10:15 blk_1073741828_1004.meta
```

Hình 1.6. Tệp block vật lý `blk_<id>` và `blk_<id>_<genstamp>.meta` trong thư mục dữ liệu của datanode1.

1.2.4 Thay đổi hệ số nhân bản

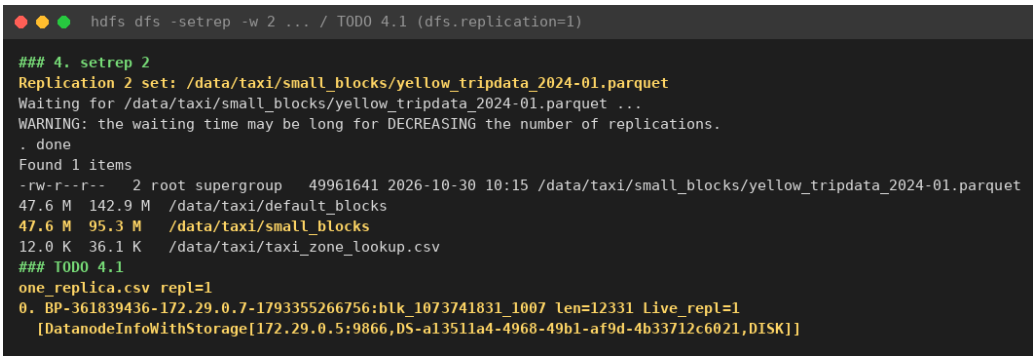
`hdfs dfs -setrep -w 2` trên tệp `small_blocks` in cảnh báo rằng việc giảm hệ số có thể phải chờ lâu, nhưng thực tế kết thúc gần như ngay (`. done`). Sau đó `ls` hiển thị hệ số 2 và `fsck` cho `Live_repl=2` ở cả ba block. Dung lượng thô của `small_blocks` giảm đúng một phần ba so với bản `default_blocks` còn hệ số 3 (Bảng 1.5).

Bảng 1.5. Dung lượng logic và dung lượng thô theo `hdfs dfs -du -h` sau khi `setrep 2`.

Đường dẫn	Hệ số	Dung lượng logic	Dung lượng thô
`/data/taxi/default_blocks`	3	47.6 M	142.9 M

Đường dẫn	Hệ số	Dung lượng logic	Dung lượng thô
`/data/taxi/small_blocks`	2	47.6 M	95.3 M
`/data/taxi/taxi_zone_lookup.csv`	3	12.0 K	36.1 K

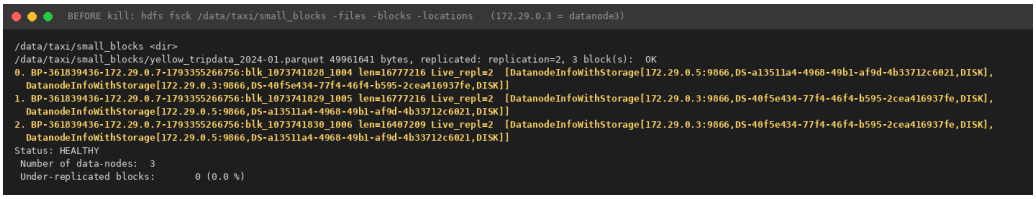
Hệ số nhân bản là tham số phía client, nên để ghi tệp với một replica mà không cần `setrep` chỉ cần ghi đè `-D dfs.replication=1` ngay trong lệnh `put` (Phụ lục). Kết quả là `one_replica.csv repl=1`, block duy nhất `blk_1073741831` (12,331 byte, `Live_repl=1`) nằm trên 172.29.0.5, tức datanode1 (Hình 1.7).



Hình 1.7. Kết quả `setrep -w 2`, `du -h` sau khi giảm hệ số và tệp được ghi với hệ số nhân bản 1.

1.2.5 Dừng một DataNode và tái nhân bản

Ngay trước khi dừng node (10:15:58), cả ba block của `small_blocks` đều có replica trên datanode1 và datanode3, không block nào nằm trên datanode2 (Hình 1.8, cột "Trước sự cố" của Bảng 1.7). Vì vậy chúng tôi chọn dừng datanode3: node này giữ replica của mọi block thuộc tệp hệ số 2.



Hình 1.8. `fsck` của tệp block 16 MB trước khi dừng datanode3: mỗi block có 2 replica trên datanode1 và datanode3.

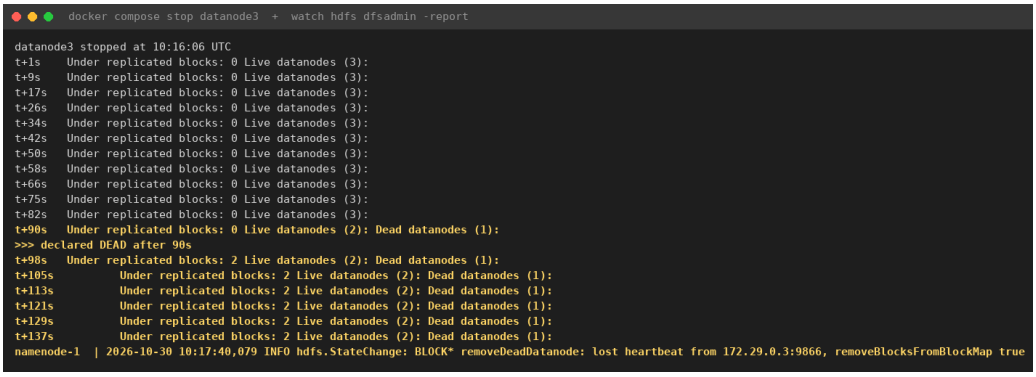
Datanode3 bị dừng bằng `docker compose stop` lúc 10:16:06 UTC, sau đó `hdfs dfsadmin -report` được thăm dò trong vòng lặp `sleep 5` (chu kỳ thực tế khoảng 8 s). DataNode gửi heartbeat mỗi 3 s; node im lặng bị coi là dead sau

$$T_{dead} = 2 \times \text{recheck-interval} + 10 \times \text{heartbeat-interval}$$

[5]. Với cấu hình lab, $T_{dead} = 2 \times 30 + 10 \times 3 = 90$ s. Diễn biến đo được ở Bảng 1.6 và Hình 1.9.

Bảng 1.6. Diễn biến sau khi dừng datanode3 theo `dfsadmin -report` và log NameNode.

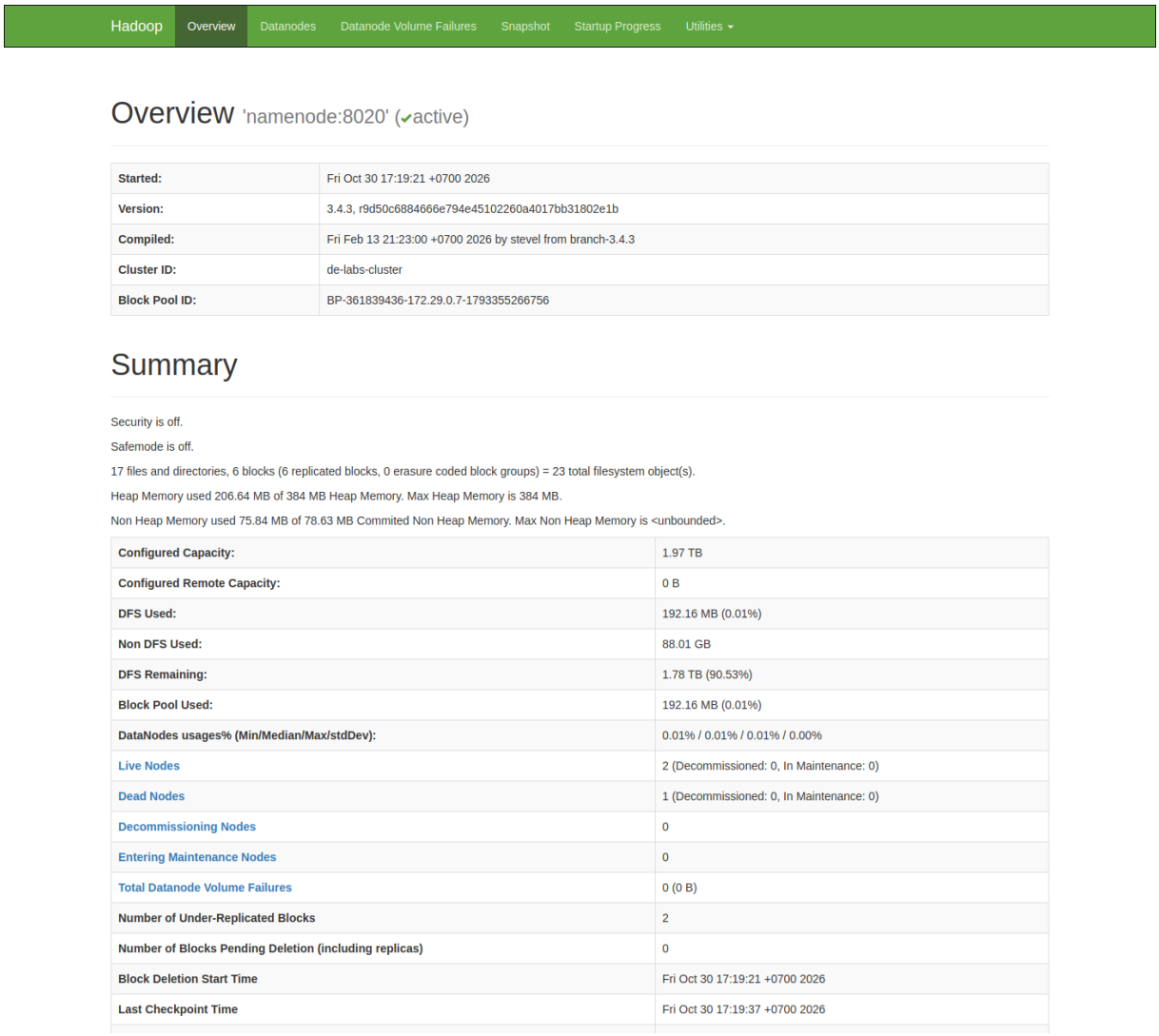
Thời điểm	Sự kiện
t+0 s (10:16:06)	`docker compose stop datanode3`
t+1 s đến t+82 s	`Live datanodes (3)`, `Under replicated blocks: 0` (11 lần thăm dò)
t+90 s	`Live datanodes (2)`, `Dead datanodes (1)`: node bị tuyên bố chết
t+94 s (10:17:40,079)	Log NameNode: `removeDeadDatanode: lost heartbeat from 172.29.0.3:9866`
t+98 s đến t+137 s	`Under replicated blocks: 2` ổn định qua 6 lần thăm dò



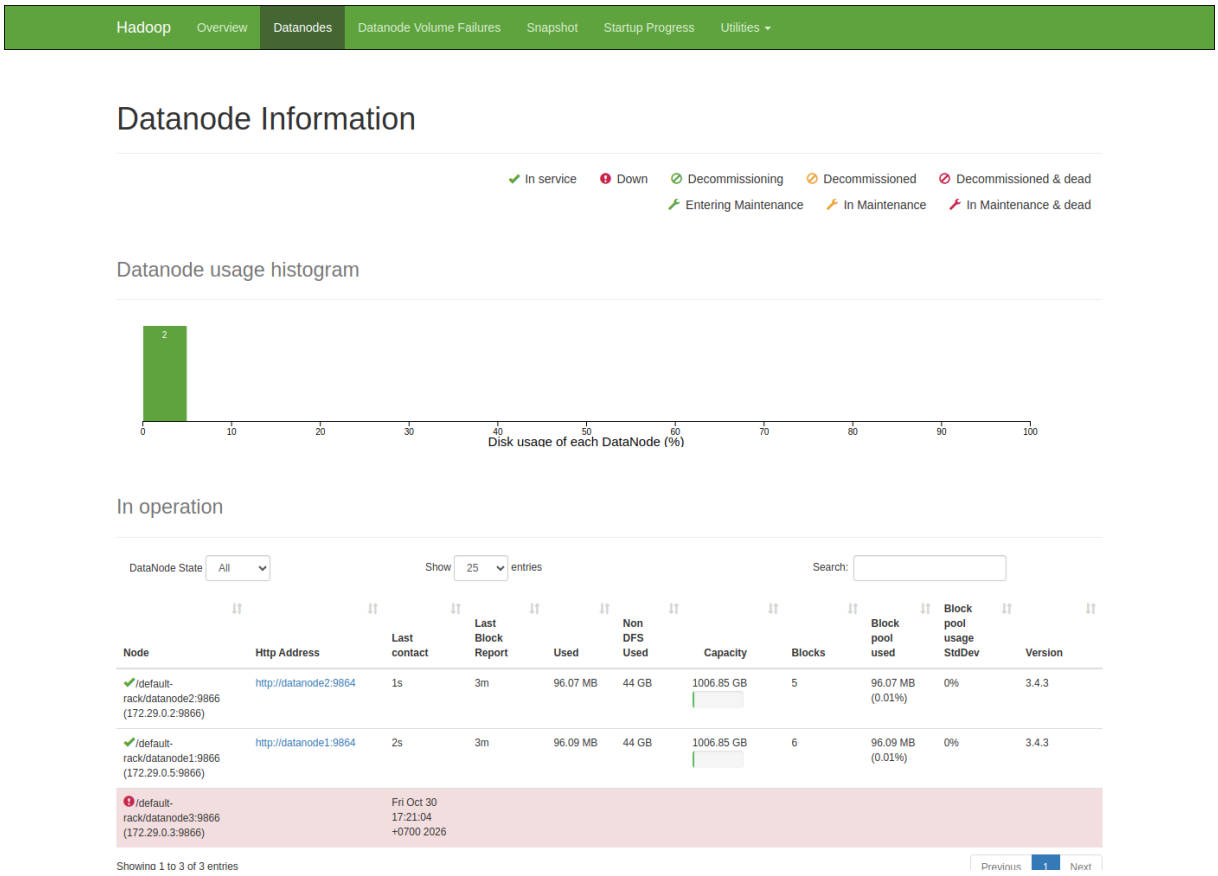
Hình 1.9. Dòng thời gian thăm dò `dfsadmin -report` sau khi dừng datanode3 và dòng log `removeDeadDatanode` của NameNode.

Node bị tuyên bố chết đúng sau **90 s**, khớp công thức. Replica bị gỡ khỏi block map muộn hơn 4 s vì bộ giám sát heartbeat chỉ chạy theo chu kỳ recheck-interval, và phải sau mốc này chỉ số under-replicated mới tăng. Có một điểm lúc đầu chúng tôi thấy lạ: ba block của tệp hệ số 2 cũng mất một replica, nhưng không lần thăm dò nào ghi nhận quá 2 block under-replicated. Cách giải thích hợp lý là ba block đó đã được tái nhân bản trong khoảng t+94 s đến t+98 s, nhanh hơn chu kỳ thăm dò. Giá trị 2 còn lại là hai block của các tệp hệ số 3 (Bảng 1.8).

Trên giao diện web, trang Overview hiện một Dead Node (Hình 1.10) và trang Datanodes đánh dấu datanode3 màu đỏ kèm thời điểm liên lạc cuối (Hình 1.11). Lần đo chính chúng tôi không kịp chụp giao diện, nên hai ảnh này được chụp ở một lần dừng datanode3 khác; mốc "Last contact" 17:21:04 +0700 (10:21:04 UTC) muộn hơn dòng thời gian trong log và không thuộc lần đo ở Bảng 1.6.



Hình 1.10. Trang Overview của NameNode khi datanode3 đã bị tuyên bố chết.

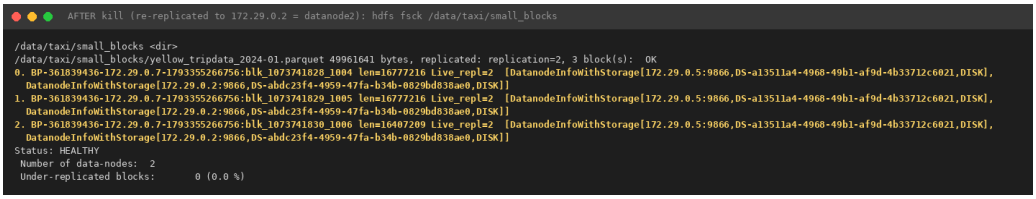


Hình 1.11. Trang Datanodes khi datanode3 chết: datanode1 và datanode2 In service, datanode3 ở trạng thái Down.

`fsck` lúc 10:18:38 cho thấy `small_blocks` đã trở lại `OK`: `Number of data-nodes: 2`, cả ba block `Live_repl=2`, `Under-replicated blocks: 0` (Hình 1.12). Replica trên datanode3 đã được chép sang datanode2, node còn sống duy nhất chưa giữ bản sao (Bảng 1.7).

Bảng 1.7. Vị trí replica của tệp `small_blocks` (hệ số 2) trước và sau khi dừng datanode3.

Block	Kích thước (byte)	Trước sự cố	Sau sự cố
`blk_1073741828`	16,777,216	datanode1, datanode3	datanode1, datanode2
`blk_1073741829`	16,777,216	datanode3, datanode1	datanode1, datanode2
`blk_1073741830`	16,407,209	datanode3, datanode1	datanode1, datanode2

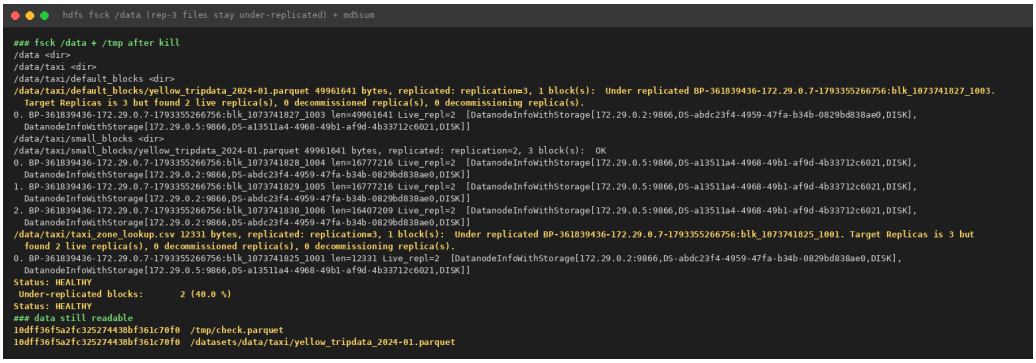


Hình 1.12. `fsck` của tệp block 16 MB sau khi dừng datanode3: replica đã chuyển sang datanode2, tệp trở lại trạng thái OK.

Với toàn bộ `/data`, `fsck` báo **`Under-replicated blocks: 2 (40.0 %)`** (2 trên 5 block) và `Status: HEALTHY` (Hình 1.13). Hai block này thuộc hai tệp hệ số 3, mỗi block còn 2 replica trên datanode2 và datanode1 (Bảng 1.8). Tệp Parquet hệ số 3 tải về có md5 `10dff36f5a2fc325274438bf361c70f0`, trùng với bản gốc. Tệp `/tmp/one_replica.csv` không bị MISSING vì replica duy nhất của nó nằm trên datanode1.

Bảng 1.8. Trạng thái các tệp trong `/data` khi chỉ còn 2 DataNode sống.

Tệp	Hệ số đích	Block	`Live_repl`	Trạng thái
`default_blocks/...parquet`	3	`blk_1073741827`	2	Under replicated
`small_blocks/...parquet`	2	3 block	2	OK
`taxi_zone_lookup.csv`	3	`blk_1073741825`	2	Under replicated



Hình 1.13. `fsck /data` khi datanode3 chết: hai tệp hệ số 3 under-replicated nhưng HEALTHY; md5 của tệp tải về trùng bản gốc.

Khởi động lại datanode3, `dfsadmin -report` trở về `Live datanodes (3)` và `fsck` báo `Under-replicated blocks: 0`, `Over-replicated blocks: 0`, `Status: HEALTHY` (Hình 1.14). Các block hệ số 3 đã có lại replica thứ ba; tệp hệ số 2 lúc này có replica trên cả ba node, và NameNode đã xóa bớt bản thừa.



Hình 1.14. Trạng thái cluster sau khi datanode3 quay lại: không còn block under-replicated hay over-replicated.

1.2.6 Safe mode và fsimage

Khi bật safe mode thủ công, lệnh ghi bị từ chối với thông báo `put: Cannot create file/tmp/x.csv._COPYING_. Name node is in safe mode.`, còn `hdfs dfs -ls /data/taxi` vẫn trả về 3 mục (Hình 1.15). Sau `docker compose restart namenode`, `-safemode get` báo `Safe mode is ON` trong vài giây đầu rồi chuyển sang OFF khi tỉ lệ block được báo cáo đạt ngưỡng 99.9 % (`threshold-pct`) cộng 5 s extension.

`saveNamespace` tạo `fsimage_0000000000000000059` dung lượng 1,574 byte (kèm `.md5` 62 byte, `seen_txid` 3 byte, `VERSION` 189 byte). Bản XML do `hdfs oiv` sinh ra có tên thư mục và tệp (`tmp`, `hadoop-yarn`, `user`, `root`, `data`, `taxi`, `taxi_zone_lookup.csv`, …) cùng block ID, nhưng tìm thông tin DataNode (cổng 9866) thì được 0 dòng: fsimage không chứa vị trí block.

```
safe mode (manual + at restart) / fsimage via hdfs oiv

### 6. safe mode manual
Safe mode is OFF
Safe mode is ON
put: Cannot create file/tmp/x.csv._COPYING_. Name node is in safe mode.
Found 3 items
drwxr-xr-x   - root supergroup          0 2026-10-30 10:15 /data/taxi/default_blocks
drwxr-xr-x   - root supergroup          0 2026-10-30 10:15 /data/taxi/small_blocks
Safe mode is OFF
### automatic safe mode at restart
Safe mode is ON
Safe mode is OFF
### 7. fsimage
Safe mode is ON
Save namespace successful
Safe mode is OFF
-rw-r--r-- 1 root root    1574 Oct 30 10:19 fsimage_0000000000000000059
-rw-r--r-- 1 root root      62 Oct 30 10:19 fsimage_0000000000000000059.md5
-rw-r--r-- 1 root root       3 Oct 30 10:19 seen_txid
-rw-r--r-- 1 root root     189 Oct 30 10:19 VERSION
<name></name>
<name>tmp</name>
<name>hadoop-yarn</name>
<name>staging</name>
<name>mr-history</name>
<name>user</name>
<name>root</name>
<name>done</name>
<name>tmp</name>
<name>data</name>
<name>taxi</name>
<name>taxi_zone_lookup.csv</name>
block location info in fsimage? 0
```

Hình 1.15. Safe mode thủ công (ghi bị từ chối, đọc vẫn hoạt động), safe mode tự động khi khởi động lại và nội dung fsimage.

1.3 Trả lời câu hỏi checkpoint

1.3.1 Tệp 300 MB với cấu hình mặc định chiếm bao nhiêu block, replica và dung lượng?

Số block là $\lceil 300/128 \rceil = 3$: hai block 128 MB và một block cuối 44 MB. Với hệ số 3 có $3 \times 3 = 9$ replica và khoảng $3 \times 300 = 900$ MB dung lượng thô, cộng thêm tệp `.meta` khoảng 0.78 % (Bảng 1.4). Block cuối không chiếm trọn 128 MB (Mục 1.3.2), và phép nhân ba dung lượng khớp với số đo ở Bảng 1.5.

1.3.2 Vì sao block cuối của tệp taxi nhỏ hơn 16 MB, và có tốn 16 MB đĩa không?

Vì tệp kết thúc ở đó: $49{,}961{,}641 - 2 \times 16{,}777{,}216 = 16{,}407{,}209$ byte. Tệp `blk_1073741830` trên datanode1 đúng bằng kích thước này (Bảng 1.4), nên không có phần đĩa nào bị bỏ phí, khác với block của hệ thống tệp cục bộ (ví dụ 4 KB) vốn được cấp phát trọn. Tệp 12,331 byte được gán block size 128 MB cũng chỉ chiếm 12,331 byte trên mỗi DataNode.

1.3.3 Vị trí block được lưu ở đâu, và khi NameNode khởi động lại thì sao?

Ánh xạ block sang DataNode chỉ nằm trong RAM của NameNode: fsimage có tên tệp và block ID nhưng không có thông tin DataNode nào (Mục 1.2.6, Hình 1.15). Khi khởi động lại, NameNode nạp namespace từ fsimage và edit log rồi dựng lại block map từ block report, nên phải ở safe mode cho tới khi nhận đủ báo cáo. Thiết kế này hợp lý vì chỉ DataNode mới biết chắc block nào đang nằm trên đĩa của nó; trong lúc NameNode tắt, đĩa có thể hỏng hoặc node bị thay, nên danh sách vị trí lưu sẵn sẽ nhanh chóng sai [1], [2]. Hiện tượng thiếu replica tạm thời ngay sau khi ghi (Mục 1.2.3) cũng xuất phát từ đây.

1.3.4 Vì sao tệp hệ số 2 được phục hồi còn tệp hệ số 3 thì không?

Muốn tái nhân bản phải có một DataNode còn sống chưa giữ replica của block, vì HDFS không đặt hai replica của cùng block trên một node [3]. Mỗi block của tệp hệ số 2 ban đầu nằm trên datanode1 và datanode3, nên datanode2 còn trống để nhận bản sao mới (Bảng 1.7, Hình 1.12). Block của hai tệp hệ số 3 thì đã có mặt trên cả hai node còn sống (Bảng 1.8), không còn chỗ cho bản thứ ba, nên phải chờ datanode3 quay lại (Hình 1.14). Số replica tối đa đạt được là $\min(\text{hệ số đích}, \text{số DataNode sống})$.

1.3.5 `fsck` báo gì khi có block under-replicated, và vì sao đó chưa phải lỗi?

`fsck /data` vẫn báo `Status: HEALTHY` khi có block under-replicated (Hình 1.13). HEALTHY nghĩa là mọi block còn ít nhất `dfs.namenode.replication.min` (= 1) replica sống, nên dữ liệu vẫn đọc được, và md5 trùng khớp ở Mục 1.2.5 xác nhận điều này. Under-replication chỉ làm giảm độ dư thừa: hỏng thêm một node thì các block này còn một bản. Lỗi thật là MISSING hoặc CORRUPT, khi block không còn replica hợp lệ nào; `/tmp/one_replica.csv` sẽ rơi vào trường hợp này nếu replica duy nhất của nó nằm trên datanode3.

1.3.6 Nếu giữ `recheck-interval` mặc định 5 phút thì sao, và vì sao timeout dài lại tốt?

Theo công thức ở Mục 1.2.5, node sẽ bị tuyên bố chết sau $2 \times 300 + 10 \times 3 = 630$ s, tức 10.5 phút, thay vì 90 s như Bảng 1.6. Trong môi trường thật, timeout dài tránh được *replication storm* (bão tái nhân bản): một node khởi động lại để vá lỗi hoặc mất mạng một phút cũng có thể kéo theo việc sao chép hàng terabyte block, và khi node quay lại thì các bản sao đó thành thừa, trong khi băng thông và I/O đĩa bị tiêu tốn đúng lúc cluster đang không ổn định. Hai ngưỡng phục vụ hai việc khác nhau: ngưỡng stale ngắn (mặc định 30 s, trong lab 15 s) giúp client tránh node bất thường gần như ngay, còn ngưỡng dead dài chỉ quyết định lúc bắt đầu tái nhân bản, việc tốn kém nhất.

1.3.7 Vì sao NameNode khởi động trong safe mode và phải chặn ghi?

Lúc khởi động, NameNode biết namespace nhưng chưa biết block nằm ở đâu (Mục 1.3.3), nên khi chưa đủ 99.9 % block được báo cáo, nó không phân biệt được block thật sự mất với block chỉ chưa kịp báo. Nếu cho ghi, xóa hay tái nhân bản lúc này, NameNode có thể sao chép hàng loạt block chưa được báo cáo hoặc xóa nhầm replica tưởng là thừa. Safe mode giữ hệ thống chỉ đọc cho tới khi block map đáng tin (Mục 1.2.6).

1.3.8 Vì sao 1 triệu tệp 1 KB là vấn đề, còn 1 tệp 1 KB thì không?

Mỗi tệp, thư mục và block là một đối tượng trong heap NameNode, khoảng 150 byte mỗi đối tượng [4]. Một triệu tệp nhỏ tạo ít nhất 2 triệu đối tượng (một inode và một block mỗi tệp), xấp xỉ 300 MB heap cho khoảng 1 GB dữ liệu. Khi ghi còn có 1 triệu lời gọi RPC tới NameNode, và khi xử lý bằng MapReduce thì có 1 triệu map task vì mỗi tệp tạo ít nhất một split (*small files problem*). Trong thực nghiệm, tệp 12,331 byte vẫn chiếm riêng một block ID và một replica trên mỗi DataNode (Bảng 1.4). Cách xử lý thường dùng là gộp tệp nhỏ thành tệp lớn hoặc dùng định dạng chứa như Parquet/ORC, SequenceFile, HAR.

1.4 Kết luận

Bài lab đã hoàn thành các bước 1 đến 7 của đề. Block size chỉ là giới hạn trên và dung lượng thô tỉ lệ với hệ số nhân bản. Khi dừng datanode3, node bị tuyên bố chết sau 90 s như công thức, tệp hệ số 2 tự tái nhân bản sang datanode2, còn hai tệp hệ số 3 under-replicated nhưng vẫn HEALTHY và đọc được. fsimage không chứa vị trí block, đó là lý do NameNode cần safe mode khi khởi động.

Tài liệu tham khảo

[1] S. Ghemawat, H. Gobioff, and S.-T. Leung, “The Google File System,” in *Proc. 19th ACM Symposium on Operating Systems Principles (SOSP)*, 2003, pp. 29-43.

[2] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, “The Hadoop Distributed File System,” in *Proc. IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, 2010, pp. 1-10.

[3] Apache Software Foundation, “HDFS Architecture,” Apache Hadoop 3.4 Documentation. https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html

[4] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA, USA: O’Reilly Media, 2015.

[5] Apache Software Foundation, “hdfs-default.xml,” Apache Hadoop 3.4 Documentation. https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/hdfs-default.xml

Phụ lục: Tệp nộp kèm và trích đoạn lệnh

Các log nộp kèm được liệt kê ở Bảng 1.9; ảnh chụp nằm trong `img/`.

Bảng 1.9. Danh mục tệp log nộp kèm báo cáo.

Tệp	Nội dung
`01_shell_blocks.txt`	`dfsadmin -report`, lệnh `hdfs dfs -stat`, `fsck` block 128 MB/16 MB, tệp block
`02_setrep.txt`	`fsck` tệp 16 MB khi đã ổn định, `setrep -w 2`, `du -h`, ghi tệp hệ số 1
`03_` đến `07_*.txt`	`fsck` trước/sau khi dừng datanode3, dòng thời gian, md5, datanode3 quay lại
`08_safemode_fsimage.txt`	Safe mode thủ công và tự động, `saveNamespace`, phân tích fsimage

In tên, block size, hệ số nhân bản và kích thước của `taxi_zone_lookup.csv`:

```
hdfs dfs -stat "name=%n blocksize=%o repl=%r size=%b"
 ↪  /data/taxi/taxi_zone_lookup.csv
# name=taxi_zone_lookup.csv blocksize=134217728 repl=3 size=12331
```

Ghi tệp với hệ số 1 trong một lệnh, không dùng `setrep`:

```
hdfs dfs -D dfs.replication=1 -put
 ↪  /datasets/data/taxi/taxi_zone_lookup.csv /tmp/one_replica.csv
# one_replica.csv repl=1   (replica duy nhất trên 172.29.0.5 = datanode1)
```

Các lệnh chính của thí nghiệm block và sự cố DataNode:

```
hdfs dfs -D dfs.blocksize=16m -put
 ↪  /datasets/data/taxi/yellow_tripdata_2024-01.parquet \
    /data/taxi/small_blocks/
hdfs fsck /data/taxi/small_blocks -files -blocks -locations
hdfs dfs -setrep -w 2
 ↪  /data/taxi/small_blocks/yellow_tripdata_2024-01.parquet
docker compose stop datanode3          # chạy trên máy host
while true; do hdfs dfsadmin -report | grep -E "Live|Dead|Under"; sleep 5;
 ↪  done
hdfs dfs -get /data/taxi/default_blocks/yellow_tripdata_2024-01.parquet
 ↪  /tmp/check.parquet
md5sum /tmp/check.parquet
 ↪  /datasets/data/taxi/yellow_tripdata_2024-01.parquet
```

Chương 2

MapReduce với Hadoop Streaming, map-side join và combiner

2.1 Mục tiêu và môi trường

Bài thực hành viết mapper và reducer bằng Python, thử bằng đường ống Unix `cat | mapper | sort | reducer`, rồi chạy trên YARN qua Hadoop Streaming [2]. Có hai bài toán: đếm từ trên bốn cuốn sách Project Gutenberg, và đếm số chuyến taxi theo khu vực đón khách, trong đó mã khu vực được đổi sang tên bằng map-side join qua distributed cache (`-files`). Thí nghiệm chính so sánh counter của cùng một job khi có và không có combiner. Sau đó các job được xem lại trên ResourceManager UI (cổng 8088), JobHistory UI (cổng 19888) và CLI. Phần mở rộng tính giá cước trung bình theo khu vực.

Bài dùng lại cluster Docker của L01 (Bảng 2.1). Thư mục bài được gắn vào `/lab/L02`, mọi lệnh chạy trong container `namenode` vì Streaming cần `python3` có mặt trên nút chạy task. Dữ liệu ở Bảng 2.2: bốn cuốn sách có mã 1342, 11, 84 và 2701, và 1,000,000 dòng đầu của tệp Parquet `yellow_tripdata_2024-01`, chuyển sang CSV không tiêu đề bằng `parquet_to_csv.py` vì Streaming chỉ đọc văn bản.

Bảng 2.1. Môi trường thực nghiệm.

Thành phần	Cấu hình
HDFS	1 NameNode, 3 DataNode, replication factor = 3
YARN	1 ResourceManager (Capacity Scheduler), 1 NodeManager (1 Active Node trên UI)
Tài nguyên YARN	NodeManager cấp 1.5 GB: AM 512 MB + 2 task 384 MB chạy đồng thời

Thành phần	Cấu hình
Phân bổ container	tối thiểu `<memory:128, vCores:1>`, tối đa `<memory:1536, vCores:4>`
Lịch sử job	JobHistory Server, cổng 19888; ResourceManager UI cổng 8088
Ngôn ngữ mapper/reducer	Python 3, chạy qua `hadoop-streaming-*.jar`

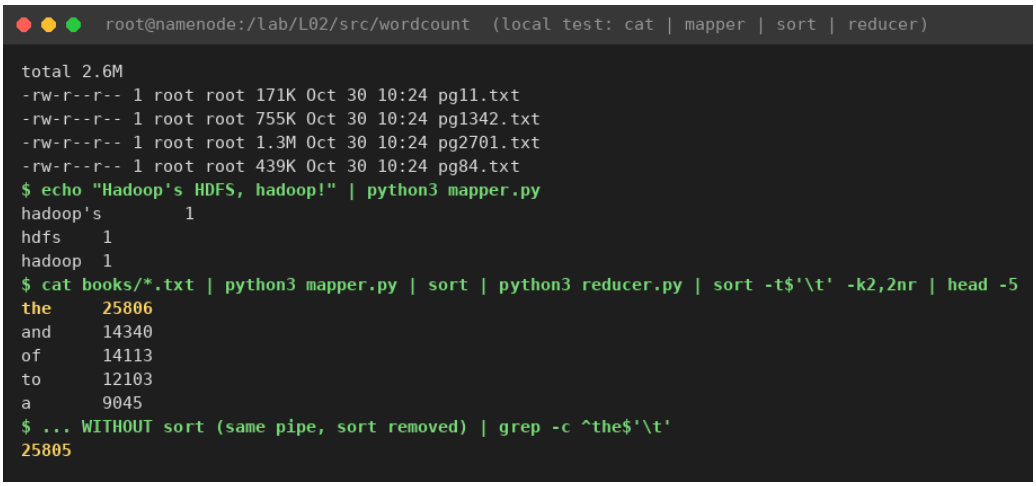
Bảng 2.2. Dữ liệu sử dụng trong thực nghiệm.

Bộ dữ liệu	Kích thước	Số bản ghi	Lưu trữ HDFS
Sách Gutenberg (4 tệp)	pg11 171K, pg1342 755K, pg2701 1.3M, pg84 439K (tổng 2.6M)	48,749 dòng	`/user/root/books`, 4 tệp
Taxi CSV 2024-01	92,847,108 byte (92.8 MB)	1,000,000 dòng	`/user/root/taxi_csv`, block 16 MB, 6 block
`taxi_zone_lookup.csv`	nhỏ	265 dòng	gửi qua `-files`

2.2 Thực hiện và kết quả

2.2.1 Đếm từ, kiểm thử cục bộ

Với Streaming, mapper và reducer đọc dòng văn bản từ stdin và in `key<TAB>value` ra stdout. Mapper đổi văn bản sang chữ thường rồi lấy từ theo biểu thức `[a-z][a-z']*`; với đầu vào `Hadoop's HDFS, hadoop!` nó in ra ba cặp `hadoop's 1`, `hdfs 1`, `hadoop 1`. Reducer tự nhận ra chỗ đổi khóa trên luồng đã sắp xếp và in thêm khóa cuối sau vòng lặp (xem Phụ lục). Chạy cả đường ống trên bốn cuốn sách, năm từ nhiều nhất là `the 25806`, `and 14340`, `of 14113`, `to 12103` và `a 9045`, đúng giá trị đề bài cho. Khi bỏ `sort`, reducer in ra 25,805 dòng bắt đầu bằng `the` thay vì một dòng tổng (Hình 2.1): gần như lần xuất hiện nào cũng thành một nhóm riêng, chỉ những chỗ `the` tình cờ đứng liền nhau mới được gộp.



Hình 2.1. Kiểm thử cục bộ đường ống `cat | mapper | sort | reducer` trên bốn cuốn sách: top 5 từ và số dòng `the` khi bỏ `sort`.

2.2.2 Đếm từ trên YARN

Sách được đưa lên `/user/root/books` và job chạy bằng `hadoop jar $STREAM` với `-files mapper.py,reducer.py` để gửi script tới thư mục làm việc của từng task. Job `wordcount` (`job_1793355278127_0001`) chạy thành công với `number of splits:4`: mỗi cuốn sách nhỏ hơn một block nên được một split, vì vậy có 4 map task và 1 reduce task theo mặc định (Bảng 2.3, Hình 2.2). Số khóa vào reducer bằng số dòng đầu ra, và top 5 đọc từ HDFS giống bản chạy cục bộ.

Bảng 2.3. Counter của job đếm từ trên YARN.

Counter	Giá trị
Launched map tasks / reduce tasks	4 / 1
Map input records	48,749
Map output records	462,618
Map output bytes	3,408,811
Map output materialized bytes = Reduce shuffle bytes	4,334,071
Reduce input groups = Reduce output records	20,967
HDFS bytes written (`part-00000`)	226,341
CPU time spent	7,010 ms
Thời gian lệnh (`real`)	23.572 s

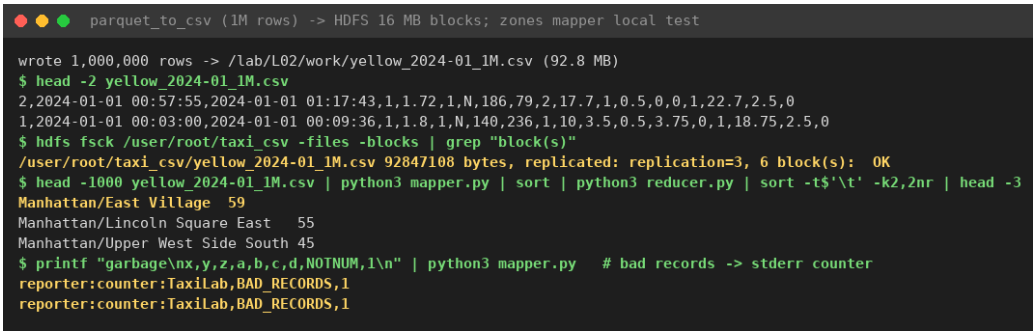


Hình 2.2. Counter cuối job `wordcount` trên YARN: 4 map task, `Reduce input groups=20967` và top 5 từ đọc từ HDFS.

2.2.3 Chuẩn bị dữ liệu taxi và map-side join

Tệp CSV được đưa lên bằng `hdfs dfs -D dfs.blocksize=16m -put` để job có nhiều split hơn. Số block dự kiến là $\lceil 92{,}847{,}108 / 16{,}777{,}216 \rceil = \lceil 5.53 \rceil = 6$, và `hdfs fsck` báo đúng `92847108 bytes, replicated: replication=3, 6 block(s): OK`.

Mapper khu vực nạp `taxi_zone_lookup.csv` vào một `dict` rồi đổi `PULocationID` thành `Borough/Zone` ngay khi đọc từng chuyến, nên bảng lớn không phải shuffle theo khóa nối. Dòng hỏng không làm mapper dừng mà được đếm bằng cách ghi `reporter:counter:TaxiLab,BAD_RECORDS,1` ra stderr. Lúc thử cục bộ, mapper chưa đọc được bảng tra cứu vì `-files` chỉ có tác dụng trên cluster, nên chúng tôi chép `taxi_zone_lookup.csv` vào thư mục hiện hành để giả lập distributed cache. Trên 1,000 dòng đầu, ba khu vực nhiều chuyến nhất là `Manhattan/East Village 59`, `Manhattan/Lincoln Square East 55` và `Manhattan/Upper West Side South 45`. Khi đưa vào hai dòng hỏng (một dòng `garbage`, một dòng có `PULocationID` không phải số), mapper ghi đúng hai dòng counter ra stderr (Hình 2.3). Dữ liệu thật không có dòng hỏng nào, nên nhóm `TaxiLab` không xuất hiện trong các job trên cluster và `Map input records` bằng `Map output records`.



Hình 2.3. Chuyển Parquet sang CSV, 6 block trên HDFS, kiểm thử cục bộ map-side join trên 1,000 dòng và counter `BAD_RECORDS`.

2.2.4 Có và không có combiner

Combiner là một reducer thu nhỏ chạy trên đầu ra từng map task trước shuffle. Hadoop có thể gọi nó 0, 1 hay nhiều lần, nên hàm gộp phải có tính kết hợp, giao hoán và giữ nguyên định dạng [1], [3]. Phép cộng thỏa điều kiện này nên reducer đếm được dùng lại làm combiner. Hai job chạy liền nhau, cùng đầu vào, cùng `-D mapreduce.job.reduces=2`, chỉ khác dòng `-combiner`:

```
hadoop jar $STREAM -D mapreduce.job.name=zones_combiner -D
 ↪  mapreduce.job.reduces=2 \
  -files mapper.py,reducer.py,/datasets/data/taxi/taxi_zone_lookup.csv \
  -mapper "python3 mapper.py" -combiner "python3 reducer.py" -reducer
   ↪  "python3 reducer.py" \
  -input /user/root/taxi_csv -output /user/root/zones_out_combiner
```

Bảng 2.4 gồm các counter đề bài yêu cầu cho `zones_nocombiner` (`job_1793355278127_0002`) và `zones_combiner` (`job_1793355278127_0003`); Bảng 2.5 gồm thêm vài counter lấy từ log đầy đủ. Wall-clock được tính bằng hiệu thời điểm giữa dòng log `Running job` và dòng `completed successfully`, không dùng lệnh `time`, nên không gồm phần khởi động client.

Bảng 2.4. So sánh counter giữa hai lần chạy không và có combiner.

Counter	Không combiner	Có combiner
Launched map tasks	6	6
Map output records	1,000,000	1,000,000
Combine input records	0	1,000,000
Combine output records	0	1,382
Map output materialized bytes	31,039,071	41,169
Reduce shuffle bytes	31,039,071	41,169
Reduce input records	1,000,000	1,382
Wall-clock time	30.3 s	33.3 s

Bảng 2.5. Counter bổ sung của hai lần chạy khu vực (trích từ `nocombiner.log` và `combiner.log`).

Counter	Không combiner	Có combiner
Map output bytes	29,038,999	29,038,999
Reduce input groups = Reduce output records	247	247
Spilled Records	2,000,000	2,764
FILE: Number of bytes written	64,607,796	2,614,896
FILE: Number of bytes read	31,039,011	41,109
Total time spent by all map tasks	17,287 ms	17,398 ms
Total time spent by all reduce tasks	13,013 ms	11,353 ms
CPU time spent	15,050 ms	14,020 ms
Shuffled Maps / Merged Map outputs	12 / 12	12 / 12
Killed reduce tasks	1	không xuất hiện

```
zones job: WITHOUT vs WITH combiner (-D mapreduce.job.reduces=2)

== /tmp/nocombiner.log
2026-10-30 10:25:37,187 INFO mapreduce.Job: Running job: job_1793355278127_0002
2026-10-30 10:26:07,488 INFO mapreduce.Job: Job job_1793355278127_0002 completed successfully
                Launched map tasks=6
                Launched reduce tasks=2
                Map output records=1000000
                Map output materialized bytes=31039071
                Combine input records=0
                Combine output records=0
                Reduce shuffle bytes=31039071
                Reduce input records=1000000
WALL nocombiner 30.3 s (10:25:37.187 Running job -> 10:26:07.488 completed)
== /tmp/combiner.log
2026-10-30 10:26:10,419 INFO mapreduce.Job: Running job: job_1793355278127_0003
2026-10-30 10:26:43,754 INFO mapreduce.Job: Job job_1793355278127_0003 completed successfully
                Launched map tasks=6
                Launched reduce tasks=2
                Map output records=1000000
                Map output materialized bytes=41169
                Combine input records=1000000
                Combine output records=1382
                Reduce shuffle bytes=41169
                Reduce input records=1382
WALL combiner 33.3 s (10:26:10.419 Running job -> 10:26:43.754 completed)
$ bash top_zones.sh /user/root/zones_out_combiner
Queens/JFK Airport      59777
Manhattan/Midtown Center        48750
Manhattan/Upper East Side South 47260
Manhattan/Upper East Side North 45867
Manhattan/Lincoln Square East   37424
Manhattan/Penn Station/Madison Sq West  37342
Manhattan/Midtown East  36421
Manhattan/Times Sq/Theatre District     36291
Queens/LaGuardia Airport        33240
Manhattan/Upper West Side South 30849
$ diff <(nocombiner output) <(combiner output)
IDENTICAL results
```

Hình 2.4. Trích các counter so sánh từ hai log và top 10 khu vực đón khách; `diff` giữa hai đầu ra cho kết quả IDENTICAL.

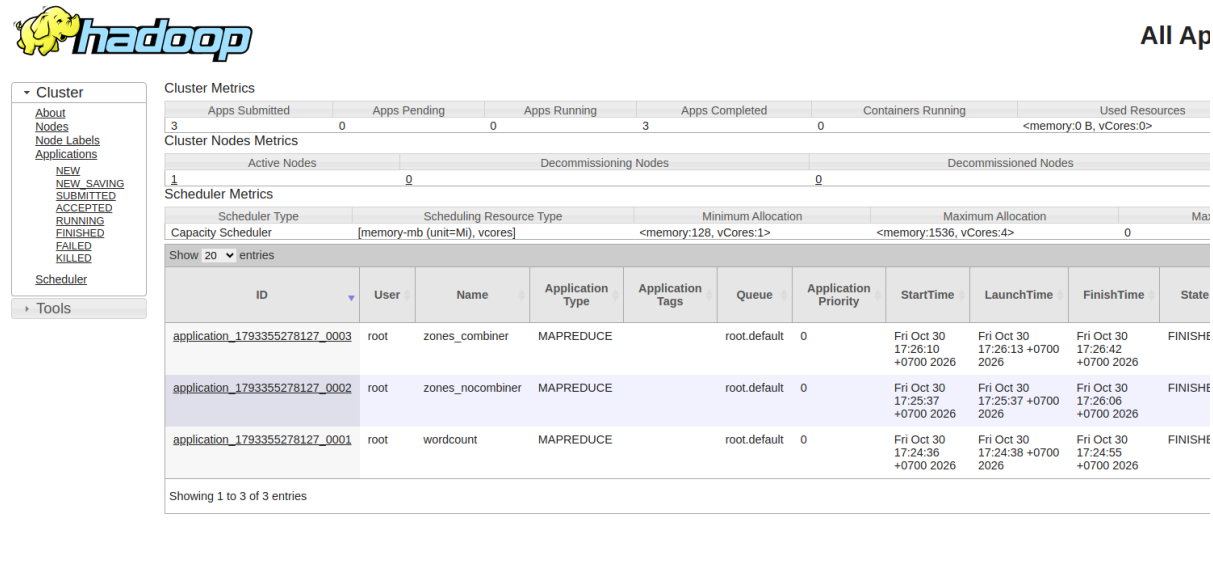
`Map output bytes` không đổi, tức mapper sinh cùng lượng dữ liệu ở cả hai lần; combiner thu gọn nó trước khi ghi đĩa và shuffle. Lượng shuffle giảm $31{,}039{,}071 / 41{,}169 \approx$ **754 lần** (khoảng 99.9 %), số bản ghi tràn đĩa và số byte ghi đĩa cục bộ giảm cùng cỡ. Cả hai lần đều có 247 khu vực vào reducer và `diff` hai đầu ra rỗng (Hình 2.4). Ba khu vực đứng đầu là `Queens/JFK Airport 59777`, `Manhattan/Midtown Center 48750` và `Manhattan/Upper East Side South 47260`.

Riêng thời gian thì ngược với dự đoán: bản có combiner chậm hơn 3.0 s. Tổng thời gian map gần như không đổi, chỉ thời gian reduce giảm. Lần chạy không combiner có `Killed reduce tasks=1`; đây là một attempt reduce bị hủy, cả hai reduce task vẫn chạy xong nên kết quả không bị ảnh hưởng.

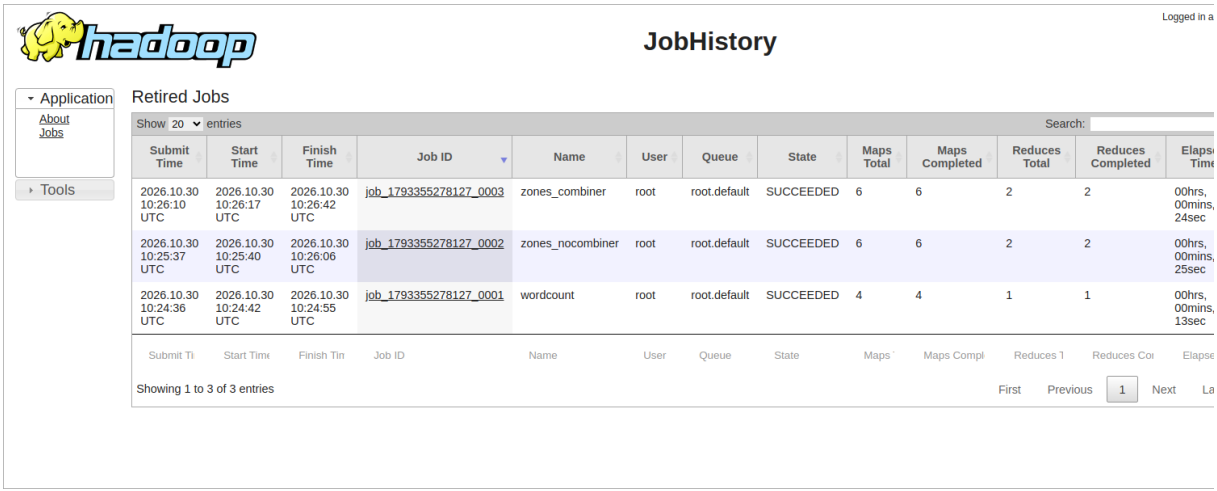
2.2.5 Giao diện YARN, JobHistory và CLI

ResourceManager UI liệt kê 3 ứng dụng đã xong (`wordcount`, `zones_nocombiner`, `zones_combiner`), 1 Active Node, Capacity Scheduler và giới hạn container như ở

Bảng 2.1 (Hình 2.5). Danh sách job trên JobHistory ở Hình 2.6.

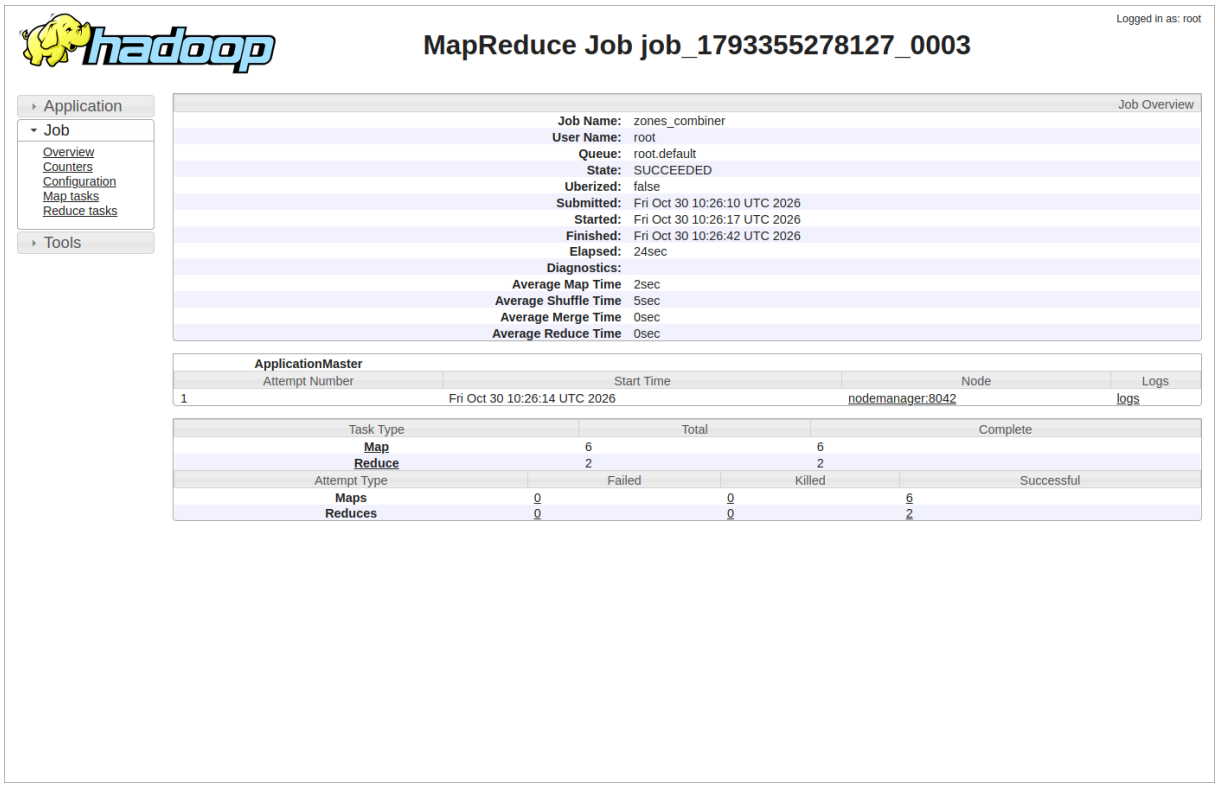


Hình 2.5. YARN ResourceManager UI (cổng 8088): ba ứng dụng MapReduce đã hoàn thành và các chỉ số cluster.



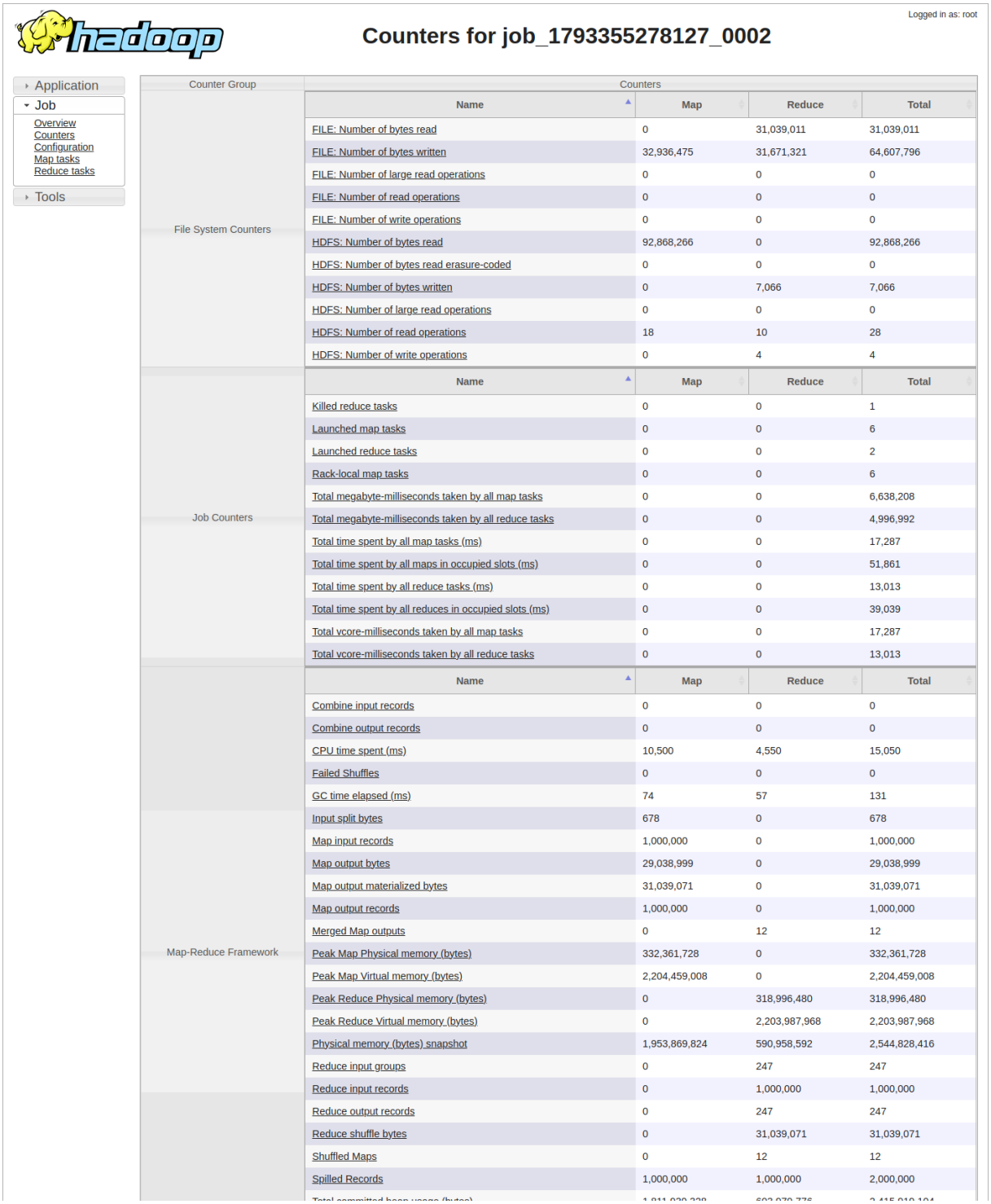
Hình 2.6. JobHistory UI (cổng 19888): danh sách các job đã hoàn thành.

Trang tổng quan của `job_1793355278127_0003` (Hình 2.7) ghi SUCCEEDED, không chạy chế độ uber, Elapsed 24 s, map trung bình 2 s, shuffle trung bình 5 s; cả 6 map và 2 reduce đều thành công, không attempt nào lỗi hay bị hủy. Elapsed ngắn hơn wall-clock ở Bảng 2.4 vì wall-clock còn gồm thời gian chờ ApplicationMaster khởi động.



Hình 2.7. JobHistory UI: tổng quan job `zones_combiner` với 6 map task, 2 reduce task và thời gian từng pha.

Trang Counters của hai job (Hình 2.8, Hình 2.9) cho đúng các giá trị in trên console.



Logged in as: root

Counters for job_1793355278127_0002

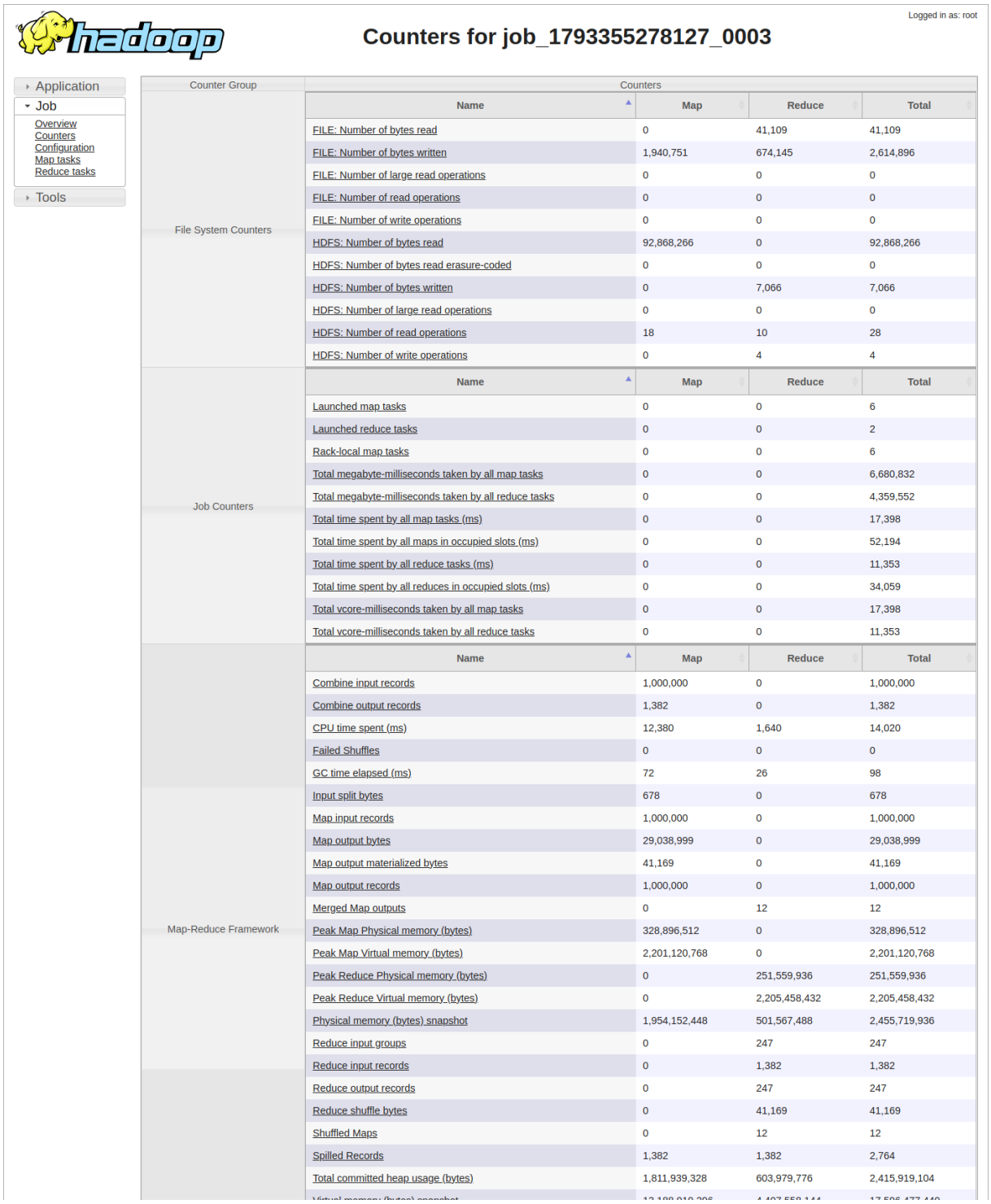
- Application
- Job
  - Overview
  - Counters
  - Configuration
  - Map tasks
  - Reduce tasks
- Tools

Counter Group	Name	Map	Reduce	Total
File System Counters	FILE: Number of bytes read	0	31,039,011	31,039,011
	FILE: Number of bytes written	32,936,475	31,671,321	64,607,796
	FILE: Number of large read operations	0	0	0
	FILE: Number of read operations	0	0	0
	FILE: Number of write operations	0	0	0
	HDFS: Number of bytes read	92,868,266	0	92,868,266
	HDFS: Number of bytes read erasure-coded	0	0	0
	HDFS: Number of bytes written	0	7,066	7,066
	HDFS: Number of large read operations	0	0	0
	HDFS: Number of read operations	18	10	28
	HDFS: Number of write operations	0	4	4

Counter Group	Name	Map	Reduce	Total
Job Counters	Killed reduce tasks	0	0	1
	Launched map tasks	0	0	6
	Launched reduce tasks	0	0	2
	Rack-local map tasks	0	0	6
	Total megabyte-milliseconds taken by all map tasks	0	0	6,638,208
	Total megabyte-milliseconds taken by all reduce tasks	0	0	4,996,992
	Total time spent by all map tasks (ms)	0	0	17,287
	Total time spent by all maps in occupied slots (ms)	0	0	51,861
	Total time spent by all reduce tasks (ms)	0	0	13,013
	Total time spent by all reduces in occupied slots (ms)	0	0	39,039
	Total vcore-milliseconds taken by all map tasks	0	0	17,287
	Total vcore-milliseconds taken by all reduce tasks	0	0	13,013

Counter Group	Name	Map	Reduce	Total
Map-Reduce Framework	Combine input records	0	0	0
	Combine output records	0	0	0
	CPU time spent (ms)	10,500	4,550	15,050
	Failed Shuffles	0	0	0
	GC time elapsed (ms)	74	57	131
	Input split bytes	678	0	678
	Map input records	1,000,000	0	1,000,000
	Map output bytes	29,038,999	0	29,038,999
	Map output materialized bytes	31,039,071	0	31,039,071
	Map output records	1,000,000	0	1,000,000
	Merged Map outputs	0	12	12
	Peak Map Physical memory (bytes)	332,361,728	0	332,361,728
	Peak Map Virtual memory (bytes)	2,204,459,008	0	2,204,459,008
	Peak Reduce Physical memory (bytes)	0	318,996,480	318,996,480
	Peak Reduce Virtual memory (bytes)	0	2,203,987,968	2,203,987,968
	Physical memory (bytes) snapshot	1,953,869,824	590,958,592	2,544,828,416
	Reduce input groups	0	247	247
	Reduce input records	0	1,000,000	1,000,000
	Reduce output records	0	247	247
	Reduce shuffle bytes	0	31,039,071	31,039,071
	Shuffled Maps	0	12	12
	Spilled Records	1,000,000	1,000,000	2,000,000

Hình 2.8. JobHistory UI: trang Counters của job không combiner.



Logged in as: root

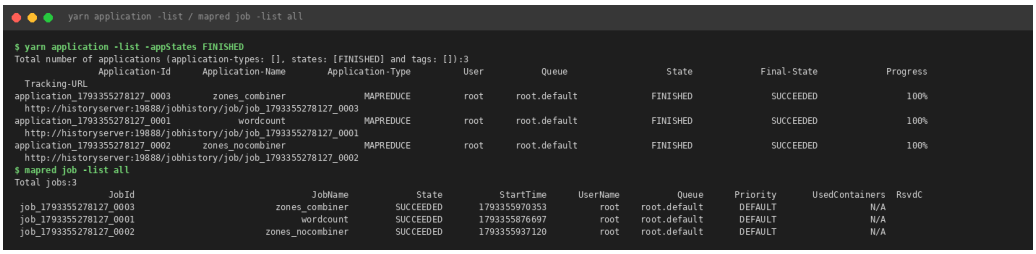
Counters for job_1793355278127_0003

- Application
- Job
  - Overview
  - Counters
  - Configuration
  - Map tasks
  - Reduce tasks
- Tools

Counter Group	Name	Map	Reduce	Total
File System Counters	FILE: Number of bytes read	0	41,109	41,109
	FILE: Number of bytes written	1,940,751	674,145	2,614,896
	FILE: Number of large read operations	0	0	0
	FILE: Number of read operations	0	0	0
	FILE: Number of write operations	0	0	0
	HDFS: Number of bytes read	92,868,266	0	92,868,266
	HDFS: Number of bytes read erasure-coded	0	0	0
	HDFS: Number of bytes written	0	7,066	7,066
	HDFS: Number of large read operations	0	0	0
	HDFS: Number of read operations	18	10	28
	HDFS: Number of write operations	0	4	4
Job Counters	Launched map tasks	0	0	6
	Launched reduce tasks	0	0	2
	Rack-local map tasks	0	0	6
	Total megabyte-milliseconds taken by all map tasks	0	0	6,680,832
	Total megabyte-milliseconds taken by all reduce tasks	0	0	4,359,552
	Total time spent by all map tasks (ms)	0	0	17,398
	Total time spent by all maps in occupied slots (ms)	0	0	52,194
	Total time spent by all reduce tasks (ms)	0	0	11,353
	Total time spent by all reduces in occupied slots (ms)	0	0	34,059
	Total vcore-milliseconds taken by all map tasks	0	0	17,398
	Total vcore-milliseconds taken by all reduce tasks	0	0	11,353
Map-Reduce Framework	Combine input records	1,000,000	0	1,000,000
	Combine output records	1,382	0	1,382
	CPU time spent (ms)	12,380	1,640	14,020
	Failed Shuffles	0	0	0
	GC time elapsed (ms)	72	26	98
	Input split bytes	678	0	678
	Map input records	1,000,000	0	1,000,000
	Map output bytes	29,038,999	0	29,038,999
	Map output materialized bytes	41,169	0	41,169
	Map output records	1,000,000	0	1,000,000
	Merged Map outputs	0	12	12
	Peak Map Physical memory (bytes)	328,896,512	0	328,896,512
	Peak Map Virtual memory (bytes)	2,201,120,768	0	2,201,120,768
	Peak Reduce Physical memory (bytes)	0	251,559,936	251,559,936
	Peak Reduce Virtual memory (bytes)	0	2,205,458,432	2,205,458,432
	Physical memory (bytes) snapshot	1,954,152,448	501,567,488	2,455,719,936
	Reduce input groups	0	247	247
	Reduce input records	0	1,382	1,382
	Reduce output records	0	247	247
	Reduce shuffle bytes	0	41,169	41,169
	Shuffled Maps	0	12	12
	Spilled Records	1,382	1,382	2,764
	Total committed heap usage (bytes)	1,811,939,328	603,979,776	2,415,919,104

Hình 2.9. JobHistory UI: trang Counters của job có combiner.

Trên dòng lệnh, `yarn application -list -appStates FINISHED` cho 3 ứng dụng có Final-State SUCCEEDED, với Tracking-URL trỏ tới `historyserver:19888`, và `mapred job -list all` báo `Total jobs:3` (Hình 2.10).



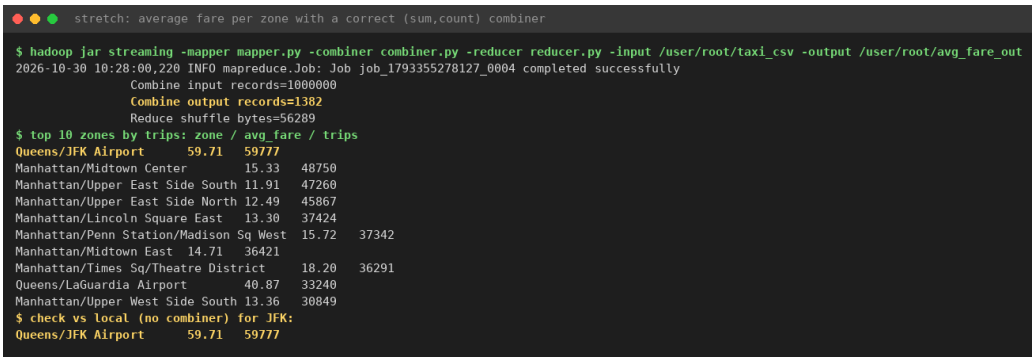
Hình 2.10. Kiểm tra các job đã hoàn thành bằng `yarn application -list` và `mapred job -list all`.

2.2.6 Mở rộng: giá cước trung bình theo khu vực

Mapper phát `zone\t<fare>,1`, trong đó `fare_amount` là cột thứ 10. Combiner cộng dồn cặp `(sum, count)` và xuất đúng định dạng nó nhận vào; reducer cộng tiếp và chỉ chia `sum/count` ở bước cuối. Job `job_1793355278127_0004` chạy xong với `Combine input records=1000000`, `Combine output records=1382` và `Reduce shuffle bytes=56289` (Bảng 2.6, Hình 2.11).

Bảng 2.6. Giá cước trung bình (`fare_amount`) của 10 khu vực đón khách nhiều chuyến nhất.

Khu vực	Giá cước trung bình	Số chuyến
Queens/JFK Airport	59.71	59,777
Manhattan/Midtown Center	15.33	48,750
Manhattan/Upper East Side South	11.91	47,260
Manhattan/Upper East Side North	12.49	45,867
Manhattan/Lincoln Square East	13.30	37,424
Manhattan/Penn Station/Madison Sq West	15.72	37,342
Manhattan/Midtown East	14.71	36,421
Manhattan/Times Sq/Theatre District	18.20	36,291
Queens/LaGuardia Airport	40.87	33,240
Manhattan/Upper West Side South	13.36	30,849



Hình 2.11. Job tính giá cước trung bình theo khu vực với combiner `(sum, count)` và đối chiếu JFK với bản chạy không combiner.

Tính lại JFK bằng bản cục bộ không combiner cũng được 59.71 trên 59,777 chuyến, và số chuyến từng khu vực khớp với job đếm ở Mục 2.2.4. Số bản ghi shuffle vẫn là 1,382 nhưng số byte nhiều hơn vì mỗi giá trị mang thêm một tổng số thực. Hai sân bay có giá trung bình cao hơn hẳn các khu vực Manhattan, do chuyến từ sân bay thường dài hơn.

2.3 Trả lời câu hỏi checkpoint

2.3.1 Trong đường ống cục bộ, lệnh nào làm việc của shuffle, và thiếu nó thì sao?

Lệnh `sort`. Trên cluster, shuffle phân vùng đầu ra map theo khóa rồi merge-sort ở reducer, nên các bản ghi cùng khóa luôn nằm liền nhau [3]; `sort` cho đúng bảo đảm đó. Không có nó, reducer gặp cùng một từ ở nhiều đoạn rời nhau và in ra nhiều lần với số đếm cục bộ, như các dòng `the` ở Hình 2.1.

2.3.2 Vì sao reducer Streaming phải tự giữ `current_key`?

Streaming chỉ đưa cho reducer các dòng văn bản đã sắp xếp; nhóm `(key, Iterable<values>)` chỉ có trong API Java [2]. Reducer Python phải so khóa với `current_key`, in tổng khi khóa đổi và nhớ in khóa cuối sau vòng lặp (Phụ lục). Quên bước cuối thì mất từ cuối cùng. Reducer của bài không mắc lỗi này: `Reduce output records` bằng `Reduce input groups` ở Bảng 2.3.

2.3.3 Job taxi có bao nhiêu map task, bao nhiêu reduce task, và vì sao?

Job taxi có 6 map task vì tệp CSV nằm trong 6 block 16 MB và mỗi block là một split (log ghi `number of splits:6`, Mục 2.2.3); job đếm từ chỉ có 4 vì mỗi cuốn sách nhỏ hơn một block. Số reduce task là 2 vì chúng tôi đặt `-D mapreduce.job.reduces=2`. Con số này do người dùng chọn chứ không suy ra từ dữ liệu, job đếm từ để mặc định là 1. Với 2 reducer, mỗi map task tạo hai phân vùng, nên `Shuffled Maps` bằng $6 \times 2 = 12$ (Bảng 2.5). NodeManager chỉ chạy hai task 384 MB cùng lúc nên 6 map chạy thành nhiều đợt; log thể hiện qua tiến độ map tăng dần 17 %, 33 %, 50 %, 67 %, 83 % rồi 100 %.

2.3.4 Combiner giảm `Reduce shuffle bytes` bao nhiêu lần, và vì sao đầu ra combiner xấp xỉ số khu vực nhân số map task?

Khoảng 754 lần (Mục 2.2.4). Combiner chạy riêng trên đầu ra từng map task, và mỗi task chỉ xuất một bản ghi cho mỗi khu vực nó gặp. Toàn bộ dữ liệu có 247 khu vực, còn mỗi task ra trung bình $1{,}382/6 \approx 230$ bản ghi, tức mỗi split gặp gần hết các khu vực. Chỉ các tổng cục bộ này đi qua shuffle, nên `Reduce input records` cũng chỉ còn bằng `Combine output records` (Bảng 2.4).

2.3.5 Vì sao combiner không làm job nhanh hơn ở đây, và khi nào nó có ích rõ?

Trên một máy, shuffle chỉ là chép dữ liệu qua đĩa cục bộ hoặc loopback, gần như không tốn gì, trong khi combiner thêm một tiến trình Python phải đọc, phân tích và sắp xếp lại toàn bộ bản ghi đầu ra map. Tổng thời gian map không giảm, còn phần tiết kiệm ở reduce, khoảng 1.7 s (Bảng 2.5), không bù được thời gian khởi động và dao động khi lập lịch container. Mỗi cấu hình mới chạy một lần nên chênh lệch 3.0 s chưa đủ để kết luận combiner làm chậm; bản tham chiếu của đề bài cũng ghi khoảng 28 s và 37 s. Trên cluster thật, shuffle đi qua mạng và mỗi reducer phải merge hàng GB từ hàng trăm mapper. Khi đó việc bỏ được gần hết lượng shuffle và số bản ghi tràn đĩa sẽ lớn hơn nhiều so với chi phí CPU của combiner.

2.3.6 Có dùng lại reducer "trung bình" làm combiner được không?

Không được. Trung bình của các trung bình sai khi các phân vùng có số bản ghi khác nhau: trung bình của $\{10\}$ và $\{20, 30\}$ là $(10 + 20 + 30)/3 = 20$, còn trung bình hai trung bình cục bộ là $(10 + 25)/2 = 17.5$. Reducer trung bình còn xuất ra một số thực, khác định dạng đầu vào, nên không gộp tiếp được khi Hadoop gọi combiner nhiều lần. Cách đúng là để combiner phát `(sum, count)` và chỉ reducer mới chia. Phần mở rộng làm theo cách này và kết quả JFK khớp với bản không combiner (Mục 2.2.6).

2.3.7 Map-side join trong bài hoạt động thế nào, và khi nào nó không còn hợp?

Bảng `taxi_zone_lookup.csv` được gửi tới mọi task qua `-files`, mỗi mapper nạp vào bộ nhớ và tra cứu ngay khi đọc chuyến (Mục 2.2.3), nên phép nối xong ở pha map. Cách này không còn hợp khi bảng "nhỏ" không vừa thoải mái trong bộ nhớ task (ở đây mỗi task chỉ có 384 MB), hoặc khi gửi bảng tới mọi task tốn hơn cả shuffle. Khi đó phải dùng reduce-side join, tức shuffle cả hai tập theo khóa nối [3]. Spark gọi ý tưởng thứ nhất là broadcast join, sẽ gặp lại ở L06.

2.3.8 Counter `BAD_RECORDS` hiện ở đâu, và vì sao nên đếm dòng hỏng thay vì cho job dừng?

Counter nằm trong job counters, nhóm `TaxiLab`: phần in ra console cuối job và mục Counters trên JobHistory UI. Nó chỉ hiện khi có ít nhất một dòng hỏng; dữ liệu trong bài không có dòng hỏng nên chỉ thấy được ở phép thử cục bộ (Hình 2.3). Một job xử lý hàng tỷ dòng không nên chết vì vài dòng lỗi và bỏ phí mọi công việc đã làm. Đếm dòng hỏng vẫn cho thấy vấn đề chất lượng dữ liệu, và có thể đặt ngưỡng để cảnh báo hoặc dừng job khi số dòng hỏng quá nhiều.

2.4 Kết luận

Bài đã hoàn thành các task của đề: mapper và reducer đếm từ cho cùng top 5 khi chạy cục bộ và trên YARN, mapper khu vực có map-side join và counter dòng hỏng, bảng counter so sánh có và không có combiner, kiểm tra job trên hai giao diện web và CLI, cùng phần mở rộng giá cước trung bình với combiner `(sum, count)`. Combiner giảm dữ liệu shuffle khoảng 754 lần mà không đổi kết quả, nhưng không làm job nhanh hơn; chúng tôi cho rằng lý do nằm ở môi trường một máy chứ không phải ở combiner. Hạn chế chính là mọi thứ chạy trên một NodeManager, shuffle không đi qua mạng thật, mỗi cấu hình chỉ chạy một lần, và `BAD_RECORDS` mới được kiểm chứng ở chế độ cục bộ.

Tài liệu tham khảo

[1] J. Dean and S. Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters,” in *Proc. 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2004.

[2] Apache Software Foundation, “Hadoop Streaming,” Apache Hadoop documentation. https://hadoop.apache.org/docs/stable/hadoop-streaming/HadoopStreaming.html

[3] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA, USA: O’Reilly Media, 2015.

Phụ lục: Trích đoạn mã nguồn

Mã đầy đủ và log của từng task nằm trong `submission/L02/` (Bảng 2.7).

Bảng 2.7. Danh mục tệp nộp kèm trong `submission/L02/`.

Tệp	Nội dung
`wc_mapper.py`, `wc_reducer.py`, `zones_mapper.py`	Mapper/reducer đếm từ, mapper khu vực
`stretch_avg_fare/`	Mapper, combiner, reducer giá cước trung bình
`01_local_pipe.txt` đến `06_stretch_avg_fare.txt`	Log từng task
`nocombiner.log`, `combiner.log`	Log đầy đủ hai job khu vực

Reducer đếm từ (`wc_reducer.py`) tự nhận ra chỗ đổi khóa và in thêm khóa cuối cùng sau vòng lặp.

```
current_word, current_count = None, 0
```

```
for line in sys.stdin:
    word, _, count = line.rstrip("\n").partition("\t")
    count = int(count)
    if word == current_word:
        current_count += count
    else:
        if current_word is not None:
            print(f"{current_word}\t{current_count}")
        current_word, current_count = word, count
if current_word is not None:  # the last word's total
    print(f"{current_word}\t{current_count}")
```

Mapper khu vực (`zones_mapper.py`) nạp bảng tra cứu, báo dòng hỏng qua stderr và phát `zone\t1`.

```
zones = {}
if os.path.exists("taxi_zone_lookup.csv"):
    with open("taxi_zone_lookup.csv", newline="") as f:
        for row in csv.DictReader(f):
            zones[row["LocationID"]] = f"{row['Borough']}/{row['Zone']}"
for line in sys.stdin:
    fields = line.rstrip("\n").split(",")
    if len(fields) <= PU_COL or not fields[PU_COL].strip().isdigit():
        sys.stderr.write("reporter:counter:TaxiLab,BAD_RECORDS,1\n")
        continue
    pu = fields[PU_COL].strip()
    print(f"{zones.get(pu, pu)}\t1")
```

Chương 3

Hive: truy vấn SQL trên HDFS, phân vùng, định dạng lưu trữ cột và bucketing

3.1 Mục tiêu và môi trường

Bài thực hành tạo bảng ngoài Hive trên CSV ở HDFS, chuyển sang bảng phân vùng ORC và Parquet, so sánh kích thước và thời gian truy vấn, kiểm chứng cắt tỉa phân vùng bằng `EXPLAIN`, tạo bảng bucketing, và làm thêm hai phần mở rộng (thống kê cột, bảng ACID). Cả cụm chạy bằng Docker Compose trên một máy (Bảng 3.1).

Bảng 3.1. Môi trường thực nghiệm.

Thành phần	Phiên bản / cấu hình
Hive	4.2.1 (`apache/hive:4.2.1`), HiveServer2 + Metastore độc lập
Bộ máy thực thi	Tez, chế độ cục bộ bên trong HiveServer2
Cơ sở dữ liệu Metastore	PostgreSQL 18 (database `metastore`)
HDFS	3.4.3, 1 NameNode + 1 DataNode (hệ số nhân bản 1)
Máy khách	Beeline qua `run_sql.sh` (in thời gian từng câu lệnh)

Đề bài gợi ý dùng mẫu 1 triệu dòng, nhưng chúng tôi dùng nguyên tháng 01/2024 của dữ liệu taxi vàng New York (Bảng 3.2), nên các con số trong bài lớn hơn số tham khảo của đề.

3.2 Thực hiện và kết quả

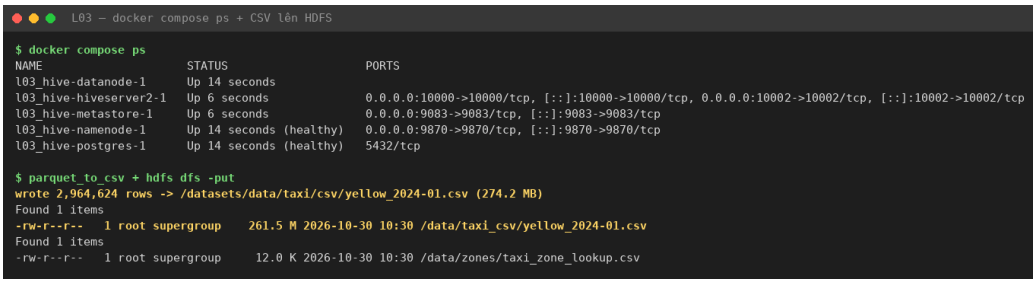
3.2.1 Khởi động cụm và đưa CSV lên HDFS

Năm container đều `Up`, NameNode và PostgreSQL ở trạng thái `healthy` (Hình 3.1). Tệp `yellow_tripdata_2024-01.parquet` được chuyển sang CSV không có dòng tiêu đề rồi tải

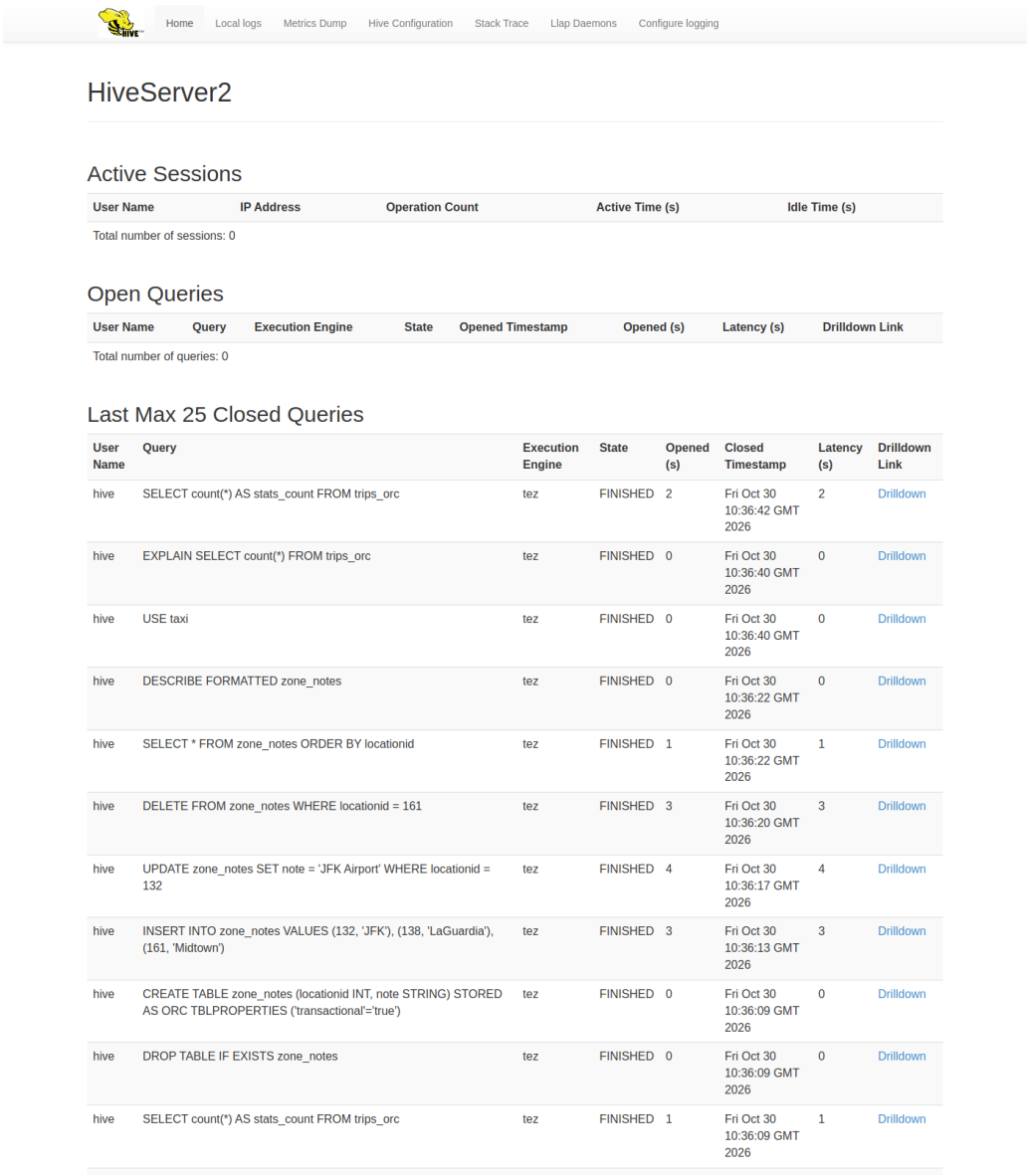
lên `/data/taxi_csv`, cùng bảng tra cứu zone lên `/data/zones` (Bảng 3.2). Giao diện web HiveServer2 ở cổng 10002 liệt kê các truy vấn đã chạy xong, tất cả trên `tez` (Hình 3.2).

Bảng 3.2. Dữ liệu đầu vào.

Tệp trên HDFS	Kích thước	Số dòng
`/data/taxi_csv/yellow_2024-01.csv`	274,217,811 B (274.2 MB, 261.5 MiB)	2,964,624
`/data/zones/taxi_zone_lookup.csv`	12.0 K	265 zone + 1 dòng tiêu đề



Hình 3.1. Khởi động cụm Hive (`docker compose ps`) và tải tệp CSV cùng bảng tra cứu zone lên HDFS.



Home | Local logs | Metrics Dump | Hive Configuration | Stack Trace | Llap Daemons | Configure logging

HiveServer2

Active Sessions

User Name	IP Address	Operation Count	Active Time (s)	Idle Time (s)

Total number of sessions: 0

Open Queries

User Name	Query	Execution Engine	State	Opened Timestamp	Opened (s)	Latency (s)	Drilldown Link

Total number of queries: 0

Last Max 25 Closed Queries

User Name	Query	Execution Engine	State	Opened (s)	Closed Timestamp	Latency (s)	Drilldown Link
hive	SELECT count(*) AS stats_count FROM trips_orc	tez	FINISHED	2	Fri Oct 30 10:36:42 GMT 2026	2	Drilldown
hive	EXPLAIN SELECT count(*) FROM trips_orc	tez	FINISHED	0	Fri Oct 30 10:36:40 GMT 2026	0	Drilldown
hive	USE taxi	tez	FINISHED	0	Fri Oct 30 10:36:40 GMT 2026	0	Drilldown
hive	DESCRIBE FORMATTED zone_notes	tez	FINISHED	0	Fri Oct 30 10:36:22 GMT 2026	0	Drilldown
hive	SELECT * FROM zone_notes ORDER BY locationid	tez	FINISHED	1	Fri Oct 30 10:36:22 GMT 2026	1	Drilldown
hive	DELETE FROM zone_notes WHERE locationid = 161	tez	FINISHED	3	Fri Oct 30 10:36:20 GMT 2026	3	Drilldown
hive	UPDATE zone_notes SET note = 'JFK Airport' WHERE locationid = 132	tez	FINISHED	4	Fri Oct 30 10:36:17 GMT 2026	4	Drilldown
hive	INSERT INTO zone_notes VALUES (132, 'JFK'), (138, 'LaGuardia'), (161, 'Midtown')	tez	FINISHED	3	Fri Oct 30 10:36:13 GMT 2026	3	Drilldown
hive	CREATE TABLE zone_notes (locationid INT, note STRING) STORED AS ORC TBLPROPERTIES ('transactional'='true')	tez	FINISHED	0	Fri Oct 30 10:36:09 GMT 2026	0	Drilldown
hive	DROP TABLE IF EXISTS zone_notes	tez	FINISHED	0	Fri Oct 30 10:36:09 GMT 2026	0	Drilldown
hive	SELECT count(*) AS stats_count FROM trips_orc	tez	FINISHED	1	Fri Oct 30 10:36:09 GMT 2026	1	Drilldown

Hình 3.2. Giao diện web HiveServer2 (cổng 10002): danh sách truy vấn đã hoàn tất, tất cả chạy trên Tez.

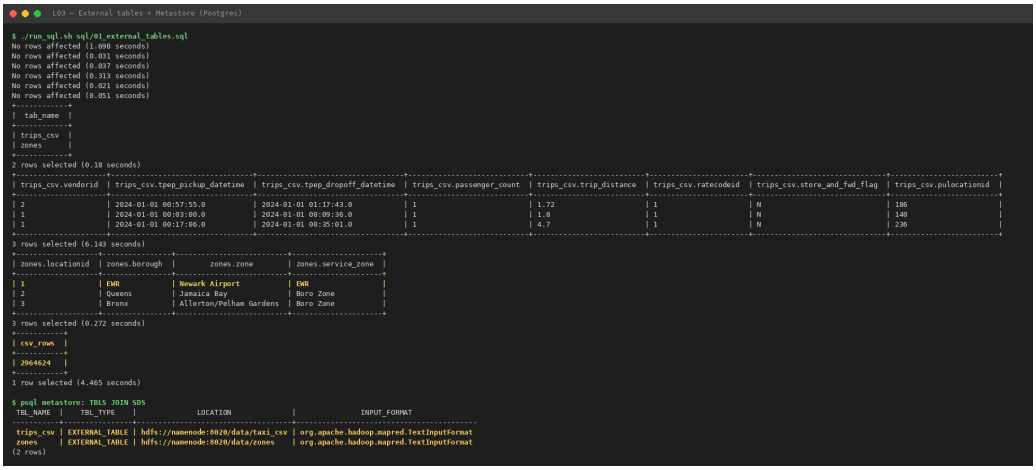
3.2.2 Bảng ngoài và nội dung Metastore

`01_external_tables.sql` tạo `trips_csv` với `ROW FORMAT DELIMITED` và `zones` với `OpenCSVSerde` cộng `skip.header.line.count=1` (mã ở Phụ lục). Cả hai đọc được ngay. `SELECT * FROM zones LIMIT 3` trả về `1 | EWR | Newark Airport | EWR`, `2 | Queens | Jamaica Bay | Boro Zone` và `3 | Bronx | Allerton/Pelham Gardens | Boro Zone`, tức dòng tiêu đề và dấu ngoặc kép đã được bỏ. `count(*)` trên `trips_csv` ra `csv_rows = 2,964,624` sau 4.465 s, khớp số dòng của tệp.

Truy vấn thẳng `TBLS JOIN SDS` trong PostgreSQL (Bảng 3.3, Hình 3.3) chỉ thấy tên bảng, loại bảng, vị trí và lớp định dạng đầu vào; không có dòng dữ liệu nào trong Metastore.

Bảng 3.3. Nội dung bảng `TBLS` và `SDS` trong Metastore sau bước tạo bảng ngoài.

TBL_NAME	TBL_TYPE	LOCATION	INPUT_FORMAT
trips_csv	EXTERNAL_TABLE	`hdfs://namenode:8020/data/taxi_csv`	`org.apache.hadoop.mapred.TextInputFormat`
zones	EXTERNAL_TABLE	`hdfs://namenode:8020/data/zones`	`org.apache.hadoop.mapred.TextInputFormat`



Hình 3.3. Kết quả truy vấn bảng ngoài và nội dung Metastore (PostgreSQL).

3.2.3 Bảng phân vùng ORC và Parquet

`02_partitioned_tables.sql` bật `hive.exec.dynamic.partition.mode=nonstrict`, tính `pickup_month` bằng `date_format(..., 'yyyy-MM')` ở cột cuối của `SELECT`, rồi nạp `trips_orc` (ZLIB) và `trips_parquet` (Snappy). `INSERT OVERWRITE` ghi 2,964,624 dòng vào `trips_orc` trong 42.761 s và vào `trips_parquet` trong 45.134 s. Chúng tôi chờ 4 phân vùng như kết quả mẫu của đề, nhưng `SHOW PARTITIONS` trả về 5 (Bảng 3.4, Hình 3.4): dữ liệu cả tháng có thêm phân vùng `2024-02`.

Bảng 3.4. Phân vùng của `trips_orc` sau khi nạp bằng phân vùng động.

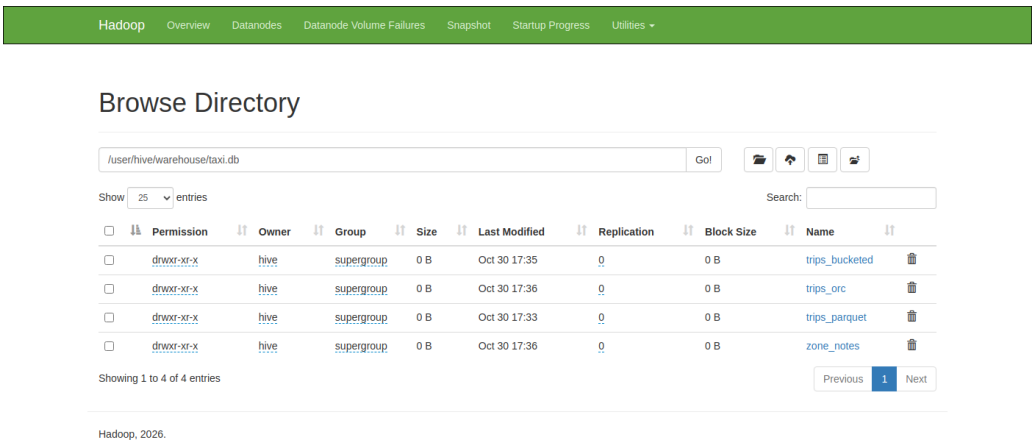
pickup_month	Số chuyến	Tệp ORC	Kích thước tệp
2002-12	2	`000007_0`	1,637 B
2009-01	3	`000004_0`	1,922 B
2023-12	10	`000001_0`	2,234 B
2024-01	2,964,606	`000005_0`	52,621,888 B
2024-02	3	`000005_0`	1,993 B

```
L03 — Dynamic partitioning ORC + Parquet

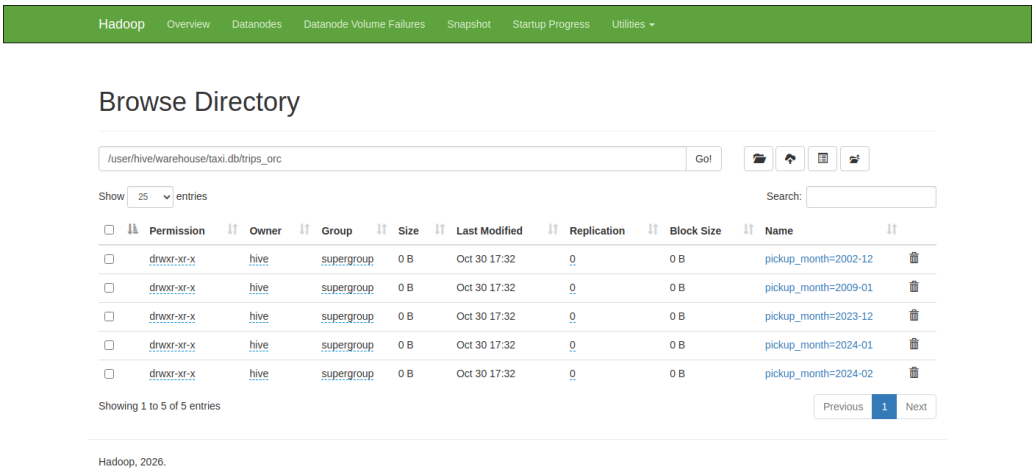
$ ./run_sql.sh sql/02_partitioned_tables.sql
No rows affected (0.06 seconds)
No rows affected (0.006 seconds)
No rows affected (0.004 seconds)
No rows affected (0.017 seconds)
No rows affected (0.07 seconds)
2,964,624 rows affected (42.761 seconds)
No rows affected (0.017 seconds)
No rows affected (0.052 seconds)
2,964,624 rows affected (45.134 seconds)
+----------------------+
|       partition      |
+----------------------+
| pickup_month=2002-12 |
| pickup_month=2009-01 |
| pickup_month=2023-12 |
| pickup_month=2024-01 |
| pickup_month=2024-02 |
+----------------------+
5 rows selected (0.077 seconds)
+---------------+----------+
| pickup_month  |  trips   |
+---------------+----------+
| 2002-12       | 2        |
| 2009-01       | 3        |
| 2023-12       | 10       |
| 2024-01       | 2964606  |
| 2024-02       | 3        |
+---------------+----------+
5 rows selected (2.24 seconds)
```

Hình 3.4. `SHOW PARTITIONS` và số chuyến theo `pickup_month`.

Trên giao diện NameNode, thư mục `taxi.db` có bốn bảng `trips_bucketed`, `trips_orc`, `trips_parquet`, `zone_notes` (Hình 3.5), và `trips_orc` có năm thư mục con `pickup_month=...` (Hình 3.6). Bốn phân vùng nhỏ chỉ có một tệp khoảng 2 KB.



Hình 3.5. Thư mục warehouse `/user/hive/warehouse/taxi.db` trên giao diện NameNode.



Hình 3.6. Năm thư mục phân vùng của bảng `trips_orc` trên giao diện NameNode.

`DESCRIBE FORMATTED trips_orc` (Hình 3.7) ghi `Table Type: EXTERNAL_TABLE` cùng `TRANSLATED_TO_EXTERNAL=TRUE` và `external.table.purge=TRUE`. Lý do là Hive 4 chỉ chấp nhận bảng quản lý dạng ACID, nên một câu `CREATE TABLE ... STORED AS ORC` thường bị Metastore chuyển thành bảng ngoài nằm trong warehouse [1]. Thống kê trong Metastore: `numFiles=5`, `numPartitions=5`, `numRows=2964624`, `rawDataSize=825609554`, `totalSize=52629674`. Hình này cũng liệt kê 18 dòng có `pickup_month` khác `2024-01`, phân tích ở Mục 3.3.8.

L03 — DESCRIBE FORMATTED trips_orc + các dòng sai tháng

```
$ ./run_sql.sh sql/06 (DESCRIBE FORMATTED trips_orc + rows outside 2024-01)
```

col_name	data_type	comment
vendorid	int	
tpep_pickup_datetime	timestamp	
tpep_dropoff_datetime	timestamp	
passenger_count	bigint	
trip_distance	double	
ratecodeid	bigint	
store_and_fwd_flag	string	
pulocationid	int	
dolocationid	int	
payment_type	bigint	
fare_amount	double	
extra	double	
mta_tax	double	
tip_amount	double	
tolls_amount	double	
improvement_surcharge	double	
total_amount	double	
congestion_surcharge	double	
airport_fee	double	
	NULL	NULL
# Partition Information	NULL	NULL
# col_name	data_type	comment
pickup_month	string	
	NULL	NULL
# Detailed Table Information	NULL	NULL
Database:	taxi	NULL
OwnerType:	USER	NULL
Owner:	hive	NULL
CreateTime:	Fri Oct 30 10:32:13 GMT 2026	NULL
LastAccessTime:	UNKNOWN	NULL
Retention:	0	NULL
Location:	hdfs://namenode:8020/user/hive/warehouse/taxi.db/trips_orc	NULL
Table Type:	**EXTERNAL_TABLE**	**NULL**
Table Parameters:	NULL	NULL
	COLUMN_STATS_ACCURATE	{\"BASIC_STATS\":\"true\"}
	EXTERNAL	TRUE
	TRANSLATED_TO_EXTERNAL	**TRUE**
	bucketing_version	2
	external.table.purge	**TRUE**
	numFiles	5
	numPartitions	5
	numRows	**2964624**
	orc.compress	ZLIB
	rawDataSize	825609554
	totalSize	52629674
	transient_lastDdlTime	1793356333
	NULL	NULL
# Storage Information	NULL	NULL
SerDe Library:	org.apache.hadoop.hive.ql.io.orc.OrcSerde	NULL
InputFormat:	org.apache.hadoop.hive.ql.io.orc.OrcInputFormat	NULL
OutputFormat:	org.apache.hadoop.hive.ql.io.orc.OrcOutputFormat	NULL
Compressed:	No	NULL
Num Buckets:	-1	NULL
Bucket Columns:	[]	NULL
Sort Columns:	[]	NULL
Storage Desc Params:	NULL	NULL
	serialization.format	1

57 rows selected (0.135 seconds)

pickup_month	tpep_pickup_datetime	tpep_dropoff_datetime	pulocationid	dolocationid	trip_distance	total_amount
2002-12	**2002-12-31 22:59:39.0**	**2002-12-31 23:05:41.0**	**170**	**170**	**0.63**	**-10.5**
2002-12	**2002-12-31 22:59:39.0**	**2002-12-31 23:05:41.0**	**170**	**170**	**0.63**	**10.5**
2009-01	**2009-01-01 00:24:09.0**	**2009-01-01 01:13:00.0**	**138**	**264**	**10.88**	**68.29**
2009-01	**2009-01-01 23:30:39.0**	**2009-01-02 00:01:39.0**	**237**	**264**	**10.99**	**50.0**
2009-01	**2009-01-01 23:58:40.0**	**2009-01-02 00:01:40.0**	**137**	**264**	**0.46**	**9.4**
2023-12	2023-12-31 23:39:17.0	2023-12-31 23:42:00.0	90	68	0.47	10.1
2023-12	2023-12-31 23:41:02.0	2023-12-31 23:48:03.0	246	246	0.4	12.2
2023-12	2023-12-31 23:47:28.0	2023-12-31 23:57:07.0	68	137	1.44	18.84
2023-12	2023-12-31 23:49:12.0	2024-01-01 00:04:32.0	234	237	3.14	28.6
2023-12	2023-12-31 23:54:27.0	2024-01-01 00:13:12.0	229	244	7.7	45.72
2023-12	2023-12-31 23:56:45.0	2024-01-01 00:00:28.0	163	237	0.97	13.5
2023-12	2023-12-31 23:56:46.0	2024-01-01 00:12:06.0	236	142	2.38	21.6
2023-12	2023-12-31 23:57:17.0	2024-01-01 00:01:50.0	144	211	0.53	12.96
2023-12	2023-12-31 23:58:35.0	2024-01-01 00:13:06.0	138	217	8.39	42.35
2023-12	2023-12-31 23:58:37.0	2024-01-01 00:08:37.0	161	170	0.59	18.75
2024-02	2024-02-01 00:00:17.0	2024-02-01 00:20:13.0	138	152	8.93	52.79
2024-02	2024-02-01 00:00:39.0	2024-02-01 00:12:08.0	186	79	2.22	22.2
2024-02	2024-02-01 00:01:15.0	2024-02-01 00:06:30.0	161	234	1.1	15.48

18 rows selected (2.868 seconds)

Hình 3.7. `DESCRIBE FORMATTED trips_orc` và 18 dòng nằm ngoài tháng 01/2024.

Kích thước đo bằng `hdfs dfs -du -s` nằm ở Bảng 3.5 và Hình 3.8. Tỉ lệ nén tính bằng $r = S_{\text{CSV}}/S_{\text{định dạng}}$, ví dụ $274{,}217{,}811/52{,}629{,}674 \approx 5.2$.

Bảng 3.5. Kích thước lưu trữ cùng dữ liệu tháng 01/2024 theo định dạng.

Định dạng	Kích thước (byte)	Kích thước (MiB)	Nhỏ hơn CSV
CSV (text)	274,217,811	261.5	1×
ORC + ZLIB	52,629,674	50.2	**5.2×**
Parquet + Snappy	65,305,645	62.3	4.2×
ORC bucketed (chỉ 5 cột)	25,271,072	24.1	10.9×

```
L03 — Kích thước CSV vs ORC vs Parquet

$ hdfs dfs -du -h /data/taxi_csv /user/hive/warehouse/taxi.db
261.5 M  261.5 M  /data/taxi_csv/yellow_2024-01.csv
50.2 M   50.2 M   /user/hive/warehouse/taxi.db/trips_orc
62.3 M   62.3 M   /user/hive/warehouse/taxi.db/trips_parquet

$ hdfs dfs -du -h -s (per table)
274217811  274217811  /data/taxi_csv
52629674   52629674   /user/hive/warehouse/taxi.db/trips_orc
65305645   65305645   /user/hive/warehouse/taxi.db/trips_parquet

$ hdfs dfs -ls -R /user/hive/warehouse/taxi.db/trips_orc
drwxr-xr-x   - hive supergroup          0 2026-10-30 10:32 /user/hive/warehouse/taxi.db/trips_orc/pickup_month=2002-12
-rw-r--r--   1 hive supergroup       1637 2026-10-30 10:32 /user/hive/warehouse/taxi.db/trips_orc/pickup_month=2002-12/000007_0
drwxr-xr-x   - hive supergroup          0 2026-10-30 10:32 /user/hive/warehouse/taxi.db/trips_orc/pickup_month=2009-01
-rw-r--r--   1 hive supergroup       1922 2026-10-30 10:32 /user/hive/warehouse/taxi.db/trips_orc/pickup_month=2009-01/000004_0
drwxr-xr-x   - hive supergroup          0 2026-10-30 10:32 /user/hive/warehouse/taxi.db/trips_orc/pickup_month=2023-12
-rw-r--r--   1 hive supergroup       2234 2026-10-30 10:32 /user/hive/warehouse/taxi.db/trips_orc/pickup_month=2023-12/000001_0
drwxr-xr-x   - hive supergroup          0 2026-10-30 10:32 /user/hive/warehouse/taxi.db/trips_orc/pickup_month=2024-01
-rw-r--r--   1 hive supergroup   52621888 2026-10-30 10:32 /user/hive/warehouse/taxi.db/trips_orc/pickup_month=2024-01/000005_0
drwxr-xr-x   - hive supergroup          0 2026-10-30 10:32 /user/hive/warehouse/taxi.db/trips_orc/pickup_month=2024-02
-rw-r--r--   1 hive supergroup       1993 2026-10-30 10:32 /user/hive/warehouse/taxi.db/trips_orc/pickup_month=2024-02/000005_0
```

Hình 3.8. Kích thước CSV, ORC và Parquet trên HDFS.

ORC + ZLIB nhỏ hơn Parquet + Snappy khoảng 19 %, hợp lý vì ZLIB thiên về tỉ lệ nén còn Snappy thiên về tốc độ. Bảng bucketed chỉ giữ 5 trên 19 cột nên con số 10.9× không đem so với ba dòng trên được.

3.2.4 So sánh thời gian truy vấn CSV, ORC và Parquet

Q1 tìm năm zone đón khách nhiều nhất trong tháng 01/2024, chỉ dùng `pulocationid`, `fare_amount` và điều kiện thời gian. Trên CSV, điều kiện là `tpep_pickup_datetime`; trên ORC và Parquet là `pickup_month = '2024-01'`. Q2 nối `trips_orc` với `zones`. `03_compare_formats.sql` được chạy hai lần với `hive.compute.query.using.stats=false`, `hive.query.results.cache.enabled=false` và `hive.fetch.task.conversion=none` để Hive phải quét thật; số dùng để so sánh là lần thứ hai, khi JVM và Tez đã khởi động xong (Bảng 3.6, Hình 3.9).

Bảng 3.6. Thời gian truy vấn theo định dạng (số liệu dùng để so sánh là lần chạy thứ hai).

Truy vấn	Lần chạy	CSV	ORC	Parquet
Q1 top 5 zone đón khách	1	9.222 s	1.987 s	1.767 s
Q1 top 5 zone đón khách	2	8.528 s	1.885 s	1.618 s
Q2 nối `zones` theo borough	1		2.939 s	

Truy vấn	Lần chạy	CSV	ORC	Parquet
Q2 nối zones theo borough	2		2.655 s	

L03 — Q1/Q2 trên CSV vs ORC vs Parquet (lần chạy 2)

```
$ ./run_sql.sh sql/03_compare_formats.sql   # RUN 2 (CSV, ORC, Parquet, join)
+---------------+---------+-----------+
| pulocationid  |  trips  | avg_fare  |
+---------------+---------+-----------+
| 132           | 145240  | 59.4      |
| 161           | 143469  | 15.21     |
| 237           | 142707  | 12.18     |
| 236           | 136464  | 12.71     |
| 162           | 106717  | 14.79     |
+---------------+---------+-----------+
5 rows selected (8.528 seconds)
+---------------+---------+-----------+
| pulocationid  |  trips  | avg_fare  |
+---------------+---------+-----------+
| 132           | 145240  | 59.4      |
| 161           | 143469  | 15.21     |
| 237           | 142707  | 12.18     |
| 236           | 136464  | 12.71     |
| 162           | 106717  | 14.79     |
+---------------+---------+-----------+
5 rows selected (1.885 seconds)
+---------------+---------+-----------+
| pulocationid  |  trips  | avg_fare  |
+---------------+---------+-----------+
| 132           | 145240  | 59.4      |
| 161           | 143469  | 15.21     |
| 237           | 142707  | 12.18     |
| 236           | 136464  | 12.71     |
| 162           | 106717  | 14.79     |
+---------------+---------+-----------+
5 rows selected (1.618 seconds)
+----------------+----------+-----------+
|   z.borough    |  trips   |  revenue  |
+----------------+----------+-----------+
| Manhattan      | 2646933  | 59247250  |
| Queens         | 273125   | 18639629  |
| Brooklyn       | 25258    | 828808    |
| Unknown        | 10360    | 304152    |
| Bronx          | 6905     | 246496    |
| N/A            | 1658     | 154901    |
| EWR            | 295      | 30738     |
| Staten Island  | 72       | 3966      |
+----------------+----------+-----------+
8 rows selected (2.655 seconds)
```

Hình 3.9. Lần chạy thứ hai của `03_compare_formats.sql`.

Ở lần thứ hai, ORC nhanh hơn CSV khoảng $8.528/1.885 \approx 4.5$ lần, Parquet khoảng $8.528/1.618 \approx 5.3$ lần. Mỗi truy vấn chỉ đo hai lần nên chưa nói được gì về phương sai, và với dữ liệu khoảng 274 MB thì chi phí khởi động Tez chiếm phần đáng kể trong các thời gian

1.6–2.7 s. Ba nguồn cho cùng kết quả Q1 (Bảng 3.7), nên đổi định dạng không làm sai dữ liệu. Zone `132` (sân bay JFK) đứng đầu, giá cước trung bình cao gấp mấy lần các zone nội đô.

Bảng 3.7. Kết quả Q1 (giống nhau trên CSV, ORC và Parquet).

pulocationid	Số chuyến	Giá cước trung bình
132	145,240	59.40
161	143,469	15.21
237	142,707	12.18
236	136,464	12.71
162	106,717	14.79

Theo Q2 (Bảng 3.8), Manhattan chiếm 89.3 % số chuyến. Cộng số chuyến các borough ra đúng 2,964,606, bằng số dòng của phân vùng `2024-01`.

Bảng 3.8. Kết quả Q2: số chuyến và doanh thu theo borough.

Borough	Số chuyến	Doanh thu (`total_amount`)
Manhattan	2,646,933	59,247,250
Queens	273,125	18,639,629
Brooklyn	25,258	828,808
Unknown	10,360	304,152
Bronx	6,905	246,496
N/A	1,658	154,901
EWR	295	30,738
Staten Island	72	3,966

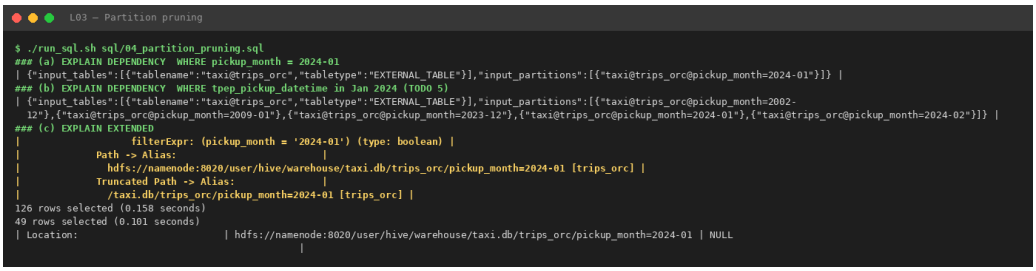
3.2.5 Cắt tỉa phân vùng

Ngoài hai câu `EXPLAIN` của đề, chúng tôi thêm một `EXPLAIN DEPENDENCY` lọc theo `tpep_pickup_datetime` để đối chiếu (Bảng 3.9, Hình 3.10).

Bảng 3.9. Kết quả kiểm chứng cắt tỉa phân vùng.

Thí nghiệm	Điều kiện lọc	Kết quả
(a) `EXPLAIN DEPENDENCY`	`pickup_month = '2024-01'`	`input_partitions` có **1** phân vùng (`pickup_month=2024-01`)
(b) `EXPLAIN DEPENDENCY`	`tpep_pickup_datetime` trong tháng 1	`input_partitions` có cả 5 phân vùng

Thí nghiệm	Điều kiện lọc	Kết quả
(c) `EXPLAIN EXTENDED`	`pickup_month = '2024-01'`	`filterExpr: (pickup_month = '2024-01')`; `Path -> Alias` chỉ có `.../trips_orc/pickup_month=2024-01`



Hình 3.10. Kết quả `EXPLAIN DEPENDENCY` và `EXPLAIN EXTENDED`.

Trong kế hoạch (c), `TableScan` ước lượng `Num rows: 2964606`, `Data size: 849321344`, `Basic stats: COMPLETE`, đúng bằng số dòng của riêng phân vùng `2024-01`. Cả `Map 1` và `Reducer 2` đều chạy `Execution mode: vectorized`. Ở (b), Hive không biết `tpep_pickup_datetime` và `pickup_month` có liên quan, vì trong lược đồ đó là hai cột độc lập, nên phải liệt kê mọi phân vùng rồi lọc từng dòng.

3.2.6 Bucketing

Bảng `trips_bucketed` (5 cột, `CLUSTERED BY (pulocationid) SORTED BY (pulocationid) INTO 4 BUCKETS`) nạp 2,964,606 dòng từ phân vùng `2024-01` trong 13.985 s. `DESCRIBE FORMATTED` ghi `Num Buckets: 4`, `Bucket Columns: [pulocationid]` và `Sort Columns: [Order(col:pulocationid, order:1)]`. Thư mục bảng có đúng bốn tệp (Bảng 3.10, Hình 3.12).

Bảng 3.10. Các tệp bucket của `trips_bucketed`.

Tệp	Kích thước	Ghi chú
`000000_0`	5,209,835 B	
`000001_0`	5,564,557 B	
`000002_0`	8,075,912 B	chứa toàn bộ 145,240 dòng JFK (zone 132)
`000003_0`	6,420,768 B	
Tổng	25,271,072 B	`numFiles=4`, `numRows=2964606`

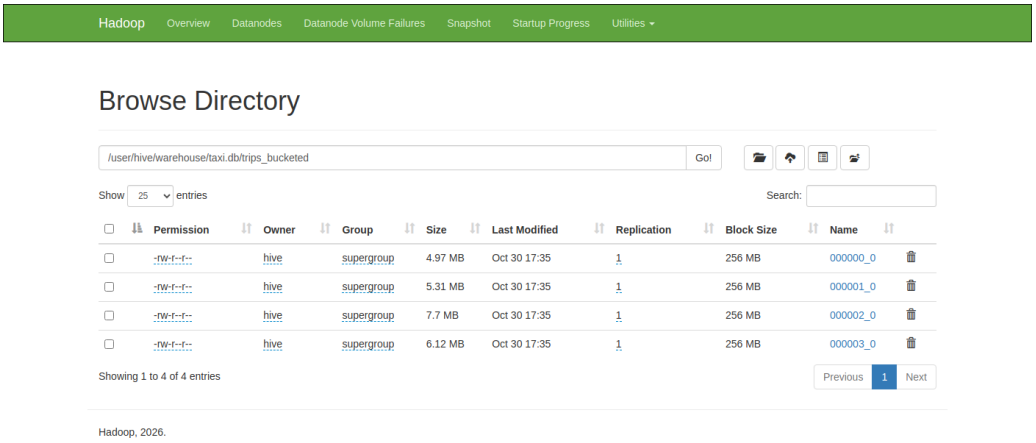
`TABLESAMPLE(BUCKET 1 OUT OF 4 ON pulocationid)` trả về 619,893 dòng sau 1.606 s, khoảng 20.9 % chứ không phải 25 %, vì hàm băm chia theo giá trị khóa chứ không theo số dòng;

bucket nào nhận các zone đông khách sẽ to hơn. Truy vấn `INPUT__FILE__NAME` cho zone 132 chỉ trả về tệp `000002_0` (Hình 3.11), cũng là tệp lớn nhất.

```
L03 — Bucketing 4 buckets + TABLESAMPLE + JFK

$ ./run_sql.sh sql/05_bucketing.sql
No rows affected (0.068 seconds)
No rows affected (0.017 seconds)
No rows affected (0.048 seconds)
2,964,606 rows affected (13.985 seconds)
+-------------------------------+--------------------------------------------------+--------------------------------------------------+
+-------------------------------+--------------------------------------------------+--------------------------------------------------+
| Num Buckets:                  | 4                                                | NULL                                             |
| Bucket Columns:               | [pulocationid]                                   | NULL                                             |
| Sort Columns:                 | [Order(col:pulocationid, order:1)]               | NULL                                             |
+-------------------------------+--------------------------------------------------+--------------------------------------------------+
38 rows selected (0.066 seconds)
+---------+
|   _c0   |
+---------+
| 619893  |
+---------+
+----------+
|   _c0    |
+----------+
| 2964606  |
+----------+
+-----------------------------------------------------+--------------+
|                        file                         | rows_for_jfk |
+-----------------------------------------------------+--------------+
| hdfs://namenode:8020/user/hive/warehouse/taxi.db/trips_bucketed/000002_0 | 145240       |
+-----------------------------------------------------+--------------+
$ hdfs dfs -ls /user/hive/warehouse/taxi.db/trips_bucketed
Found 4 items
-rw-r--r--   1 hive supergroup    5209835 2026-10-30 10:35 /user/hive/warehouse/taxi.db/trips_bucketed/000000_0
-rw-r--r--   1 hive supergroup    5564557 2026-10-30 10:35 /user/hive/warehouse/taxi.db/trips_bucketed/000001_0
-rw-r--r--   1 hive supergroup    8075912 2026-10-30 10:35 /user/hive/warehouse/taxi.db/trips_bucketed/000002_0
-rw-r--r--   1 hive supergroup    6420768 2026-10-30 10:35 /user/hive/warehouse/taxi.db/trips_bucketed/000003_0
```

Hình 3.11. Kết quả bucketing: mô tả bảng, lấy mẫu một bucket và vị trí các dòng JFK.



Hình 3.12. Bốn tệp bucket của `trips_bucketed` trên giao diện NameNode.

3.2.7 Phần mở rộng: thống kê và bảng ACID

Trước khi có thống kê, `SELECT count(*)` trên `trips_orc` mất 2.132 s; câu `ANALYZE TABLE trips_orc PARTITION (pickup_month) COMPUTE STATISTICS FOR COLUMNS` mất 18.354 s. Chúng tôi chờ `count(*)` sau đó được trả lời ngay từ Metastore khi bật `hive.compute.query.`

`using.stats=true`, nhưng không: kết quả vẫn là 2,964,624 qua DAG Tez, mất 1.569 s lần đầu và 2.592 s khi đo lại, dù `EXPLAIN` có `TableScan [TS_0] (rows=2964624 width=290)` với `Tbl:COMPLETE, Col:COMPLETE`. Giải thích hợp lý nhất là `trips_orc` thực chất là bảng EXTERNAL được chuyển đổi, và Hive không tin thống kê của bảng ngoài vì tệp có thể bị sửa từ bên ngoài. Chúng tôi chưa kiểm chứng trên bảng ACID, nên đây vẫn là giả thuyết.

Bảng `zone_notes (locationid INT, note STRING) STORED AS ORC TBLPROPERTIES ('transactional'='true')` mới là `MANAGED_TABLE` thật. Các giao dịch và thư mục sinh ra trên HDFS nằm ở Bảng 3.11, Hình 3.13 và Hình 3.14. Cuối cùng còn 2 dòng: `132 | JFK Airport` và `138 | LaGuardia`.

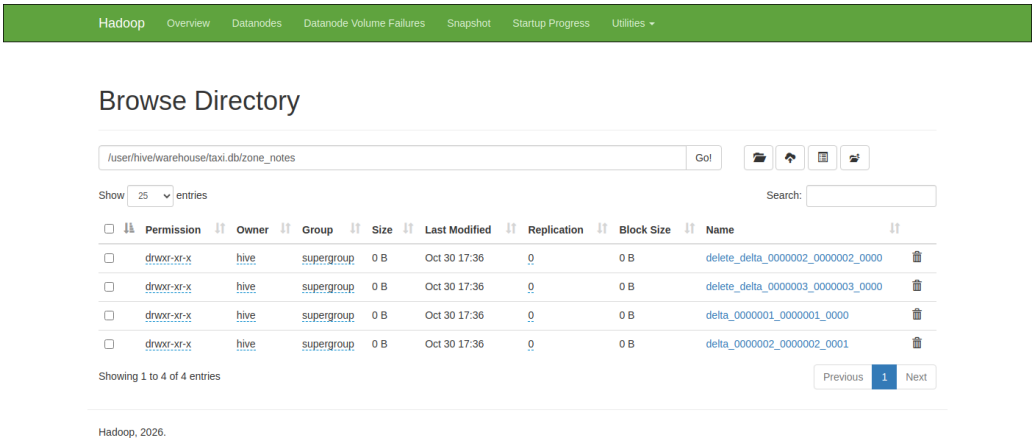
Bảng 3.11. Các giao dịch trên bảng ACID `zone_notes` và thư mục delta tương ứng.

Giao dịch	Câu lệnh	Thời gian	Thư mục sinh ra
1	`INSERT` (132 JFK), (138 LaGuardia), (161 Midtown)	3.342 s (3 dòng)	`delta_0000001_0000001_0000`
2	`UPDATE ... SET note='JFK Airport' WHERE locationid=132`	4.34 s (1 dòng)	`delete_delta_0000002_0000002_0000` + `delta_0000002_0000002_0001`
3	`DELETE ... WHERE locationid=161`	3.121 s (1 dòng)	`delete_delta_0000003_0000003_0000`

```
L03 — Stretch: statistics + ACID (delta / delete_delta)

$ stretch: statistics + ACID
+-------------+
| scan_count  |
+-------------+
| 2964624     |
+-------------+
1 row selected (2.132 seconds)
+--------------+
| stats_count  |
+--------------+
| 2964624      |
+--------------+
1 row selected (1.569 seconds)
3 rows affected (3.342 seconds)
1 row affected (4.34 seconds)
1 row affected (3.121 seconds)
+-----------------------+------------------+
| zone_notes.locationid | zone_notes.note  |
+-----------------------+------------------+
| 132                   | JFK Airport      |
| 138                   | LaGuardia        |
+-----------------------+------------------+
2 rows selected (1.578 seconds)
| Table Type:                 | MANAGED_TABLE                                  | NULL                 |
|                             | transactional                                  | true                 |
|                             | transactional_properties                       | default              |
$ hdfs dfs -ls -R .../zone_notes
  zone_notes/delete_delta_0000002_0000002_0000
  zone_notes/delete_delta_0000002_0000002_0000/bucket_00000_0
  zone_notes/delete_delta_0000003_0000003_0000
  zone_notes/delete_delta_0000003_0000003_0000/bucket_00000_0
  zone_notes/delta_0000001_0000001_0000
  zone_notes/delta_0000001_0000001_0000/bucket_00000_0
  zone_notes/delta_0000002_0000002_0001
  zone_notes/delta_0000002_0000002_0001/bucket_00000_0
```

Hình 3.13. Phần mở rộng: thống kê cột và các thao tác ACID.



Hình 3.14. Các thư mục `delta_*` và `delete_delta_*` của bảng `zone_notes` trên giao diện NameNode.

Không tệp nào bị sửa tại chỗ: `UPDATE` là một `delete_delta` cộng một `delta` mới, mỗi thư mục có một tệp `bucket_00000_0` khoảng 700–800 B và tệp `_orc_acid_version`. Bộ đọc hợp nhất các delta khi truy vấn, compaction sẽ gộp chúng thành `base_*` về sau [4].

3.3 Trả lời câu hỏi checkpoint

3.3.1 Metastore giữ gì, HDFS giữ gì, và `DROP TABLE` xóa những gì?

Metastore (`TBLS`, `SDS`, `PARTITIONS`… trong PostgreSQL) giữ siêu dữ liệu: tên và kiểu cột, SerDe, lớp định dạng, loại bảng, `LOCATION` của bảng và từng phân vùng, thống kê (`numRows`, `totalSize`…) và trạng thái giao dịch ACID. HDFS chỉ chứa tệp dữ liệu (Bảng 3.3). `DROP TABLE trips_csv` chỉ xóa siêu dữ liệu, tệp trong `/data/taxi_csv` vẫn còn. `DROP TABLE trips_orc` xóa cả siêu dữ liệu lẫn thư mục `/user/hive/warehouse/taxi.db/trips_orc`, vì bảng có `external.table.purge=TRUE` (Hình 3.7).

3.3.2 Vì sao `zones` cần `OpenCSVSerde` còn `trips_csv` chỉ cần `ROW FORMAT DELIMITED`?

`ROW FORMAT DELIMITED` (`LazySimpleSerDe`) chỉ cắt trường theo dấu phân cách. Nó không bỏ dấu ngoặc kép và không hiểu dấu phẩy nằm trong ngoặc kép. Tệp trips sinh ra từ Parquet, không tiêu đề, không ngoặc kép, nên vậy là đủ. Tệp zones có dòng tiêu đề và giá trị như `"Allerton/Pelham Gardens"`, nên cần `OpenCSVSerde` cùng `skip.header.line.count=1`. Đổi lại, `OpenCSVSerde` coi mọi cột là `STRING` (Q2 phải viết `CAST(z.locationid AS INT)`), phân tích chậm hơn và không xử lý được giá trị NULL đặc biệt.

3.3.3 Vì sao ORC và Parquet nhỏ hơn CSV 4–5 lần?

Thứ nhất là lưu theo cột [2], [3]: giá trị cùng cột, cùng kiểu nằm cạnh nhau trong từng stripe/row group nên dictionary, *run-length encoding*, delta và bit-packing nén rất tốt, nhất là với cột ít giá trị như `vendorid`, `payment_type`. Thứ hai là kiểu nhị phân cộng nén khối: một timestamp chiếm 8 byte thay vì chuỗi 19 ký tự, rồi cả khối được nén thêm bằng ZLIB hoặc Snappy (codec là lý do chính khiến ORC và Parquet chênh nhau ở Bảng 3.5). Tỉ lệ `rawDataSize`/`totalSize` của `trips_orc` (Mục 3.2.3) cho thấy dữ liệu giải nén lớn gấp khoảng 15.7 lần trên đĩa.

3.3.4 Vì sao Q1 chạy nhanh hơn trên ORC và Parquet?

Bốn cơ chế cùng góp phần vào mức giảm ở Bảng 3.6. Column pruning: Q1 chỉ cần 2–3 trong 19 cột, ORC và Parquet chỉ đọc đúng các column chunk đó, còn CSV phải phân tích cả 274 MB từng dòng. Partition pruning: điều kiện trên `pickup_month` khiến Hive chỉ mở thư mục `pickup_month=2024-01` (Bảng 3.9), còn bảng CSV không phân vùng nên phải quét hết. Predicate pushdown: min/max của từng stripe/row group cho phép bỏ qua khối không thỏa điều kiện. Cuối cùng là thực thi vectorized theo lô 1,024 dòng (Mục 3.2.5), không phải đổi văn bản thành số. Thí nghiệm không tách riêng được đóng góp của từng cơ chế.

3.3.5 Phân vùng tĩnh khác phân vùng động ra sao, và vì sao mặc định là `strict`?

Phân vùng tĩnh ghi giá trị ngay trong câu lệnh, ví dụ `PARTITION (pickup_month='2024-01')`, và mọi dòng đều vào phân vùng đó. Phân vùng động lấy giá trị từ cột cuối của `SELECT`. Chế độ `strict` bắt phải có ít nhất một giá trị tĩnh, để một câu truy vấn viết sai không lỡ tay tạo ra hàng nghìn phân vùng, hàng nghìn tệp và đối tượng trong Metastore. Bài này gặp đúng chuyện đó ở quy mô nhỏ: 18 dòng bất thường đã sinh ra bốn phân vùng lạ (Bảng 3.4).

3.3.6 Phân vùng theo `pulocationid` cộng ngày, hoặc theo `tpep_pickup_datetime`, thì sao?

Theo `pulocationid` cộng ngày sẽ có khoảng $265 \times 31 \approx 8{,}000$ phân vùng mỗi tháng, mỗi phân vùng một tệp từ vài KB đến vài trăm KB, tức vấn đề tệp nhỏ (*small files problem*) [5]. NameNode phải giữ hàng chục nghìn đối tượng trong heap, Metastore phải lưu hàng nghìn dòng `PARTITIONS`/`SDS` nên lập kế hoạch chậm đi, và mỗi tệp nhỏ thành một task riêng nên chi phí khởi tạo lấn át xử lý thật. Bốn phân vùng lẻ khoảng 2 KB của `trips_orc` (Bảng 3.4) là dấu hiệu sớm. Phân vùng theo timestamp gốc còn tệ hơn: gần như mỗi giây một phân vùng, hàng triệu thư mục chứa tệp vài trăm byte. Nên chọn cột ít giá trị mà truy vấn hay lọc, sao cho mỗi phân vùng cỡ vài trăm MB đến khoảng 1 GB.

3.3.7 Khi nào dùng partitioning, khi nào dùng bucketing, và vì sao số tệp bucket cố định?

Partitioning tạo thư mục theo giá trị cột, hợp với cột ít giá trị mà truy vấn hay lọc (ngày, tháng, quốc gia), để bỏ qua nguyên thư mục khi đọc. Bucketing chia theo $\text{hash}(c) \bmod N$ vào N tệp, hợp với cột nhiều giá trị như mã định danh hay `pulocationid`; nó cho phép lấy mẫu bằng `TABLESAMPLE`, và bucket map join hay sort-merge-bucket join không cần shuffle khi hai bảng cùng bucket trên khóa join. Số tệp cố định vì N đã chốt trong DDL (`INTO 4 BUCKETS`): mỗi dòng luôn rơi vào bucket hash mod 4 dù dữ liệu nhiều hay ít, đúng như Bảng 3.10. Hai kỹ thuật dùng chung được, ví dụ phân vùng theo tháng rồi chia bucket theo zone.

3.3.8 Những dòng nào rơi vào phân vùng lạ, và pipeline thật nên xử lý ra sao?

18 dòng ngoài tháng 01/2024 (Hình 3.7) được tóm lại ở Bảng 3.12.

Bảng 3.12. Các dòng có thời gian đón khách nằm ngoài tháng 01/2024.

Phân vùng	Số dòng	Mô tả	Đánh giá
2002-12	2	2002-12-31 22:59:39, zone 170, một cặp `total_amount` +10.5/−10.5	Lỗi đồng hồ; một chuyến bị hoàn tiền
2009-01	3	Ngày 2009-01-01, cả ba trả khách tại zone 264	Lỗi đồng hồ thiết bị
2023-12	10	2023-12-31 23:39–23:58	Chuyến thật bắt đầu trước giao thừa
2024-02	3	2024-02-01 00:00:17–00:01:15	Chuyến thật bắt đầu ngay sau nửa đêm

Dòng năm 2002 và 2009 là lỗi đồng hồ, còn dòng 2023-12 và 2024-02 là chuyến thật ở biên tháng. Trong pipeline thật, chúng tôi sẽ so thời gian sự kiện với tháng của tệp nguồn, đẩy dòng ngoài khoảng hợp lệ vào một bảng quarantine kèm lý do và cảnh báo khi số dòng này tăng; dòng biên tháng hợp lệ thì ghi vào đúng phân vùng của nó hoặc giữ theo tháng của tệp nguồn, tùy quy ước. Không nên để phân vùng động lặng lẽ tạo phân vùng rác.

3.4 Kết luận

Bài đã hoàn thành đủ các task của đề cùng hai phần mở rộng (thống kê, ACID). Định dạng cột giảm kích thước 4–5 lần và thời gian Q1 tương ứng so với CSV (Bảng 3.5, Bảng 3.6); Hive chỉ cắt tỉa phân vùng khi điều kiện lọc nằm đúng trên cột phân vùng (Bảng 3.9). Hai điểm lệch so với kỳ vọng là phân vùng thứ năm `2024-02` và việc `count(*)` không dùng thống kê. Mọi thứ chạy trên một máy với Tez cục bộ và một DataNode, nên các con số không phản ánh tính song song hay chi phí mạng của cụm thật.

Tài liệu tham khảo

[1] Apache Software Foundation, “Apache Hive Language Manual: DDL,” Apache Hive Wiki. https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL

[2] Apache Software Foundation, “ORC Specification v1,” Apache ORC. https://orc.apache.org/specificatio

[3] Apache Software Foundation, “Apache Parquet Documentation,” Apache Parquet. https://parquet.apache.org/docs/

[4] Y. Huai, A. Chauhan, A. Gates, G. Hagleitner, E. N. Hanson, O. O'Malley, J. Pandey, Y. Yuan, R. Lee, X. Zhang, "Major Technical Advancements in Apache Hive," in *Proc. ACM SIGMOD*, 2014, pp. 1235–1246.

[5] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA: O'Reilly Media, 2015.

Phụ lục: Trích đoạn mã nguồn

Các tệp nộp kèm trong `submission/L03/` được tóm ở Bảng 3.13.

Bảng 3.13. Danh mục tệp nộp kèm trong `submission/L03/`.

Tệp	Nội dung
`01_external_tables.sql` … `05_bucketing.sql`	Các tệp SQL đã hoàn thành
`01_stack_and_upload.txt` … `10_bucketing.txt`	Log từng bước, theo thứ tự thực hiện
`11_stretch_stats_acid.txt`, `12_stretch_stats_answer.txt`	Phần mở rộng thống kê và ACID
`img/`	Ảnh chụp terminal và giao diện web

Trong `01_external_tables.sql`, bảng `zones` dùng `OpenCSVSerde` và bỏ dòng tiêu đề:

```
CREATE EXTERNAL TABLE zones (
  locationid STRING, borough STRING, zone STRING, service_zone STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
STORED AS TEXTFILE
LOCATION '/data/zones'
TBLPROPERTIES ('skip.header.line.count' = '1');
```

Trong `02_partitioned_tables.sql`, bật phân vùng động, tính `pickup_month` ở cột cuối của `SELECT` và khai báo bảng Parquet nén Snappy (câu nạp `trips_parquet` giống hệt câu nạp `trips_orc`):

```
SET hive.exec.dynamic.partition.mode = nonstrict;

INSERT OVERWRITE TABLE trips_orc PARTITION (pickup_month)
SELECT t.*,
       date_format(t.tpep_pickup_datetime, 'yyyy-MM') AS pickup_month
FROM trips_csv t;
```

```
CREATE TABLE trips_parquet ( ... cùng 19 cột như trips_orc ... )
PARTITIONED BY (pickup_month STRING)
STORED AS PARQUET
TBLPROPERTIES ('parquet.compression' = 'SNAPPY');
```

Trong `05_bucketing.sql`, bảng bucketing chia theo `pulocationid` vào 4 bucket:

```
CREATE TABLE trips_bucketed (
  tpep_pickup_datetime TIMESTAMP, pulocationid INT, dolocationid INT,
  fare_amount DOUBLE, total_amount DOUBLE
)
CLUSTERED BY (pulocationid) SORTED BY (pulocationid) INTO 4 BUCKETS
STORED AS ORC;
```

Chương 4

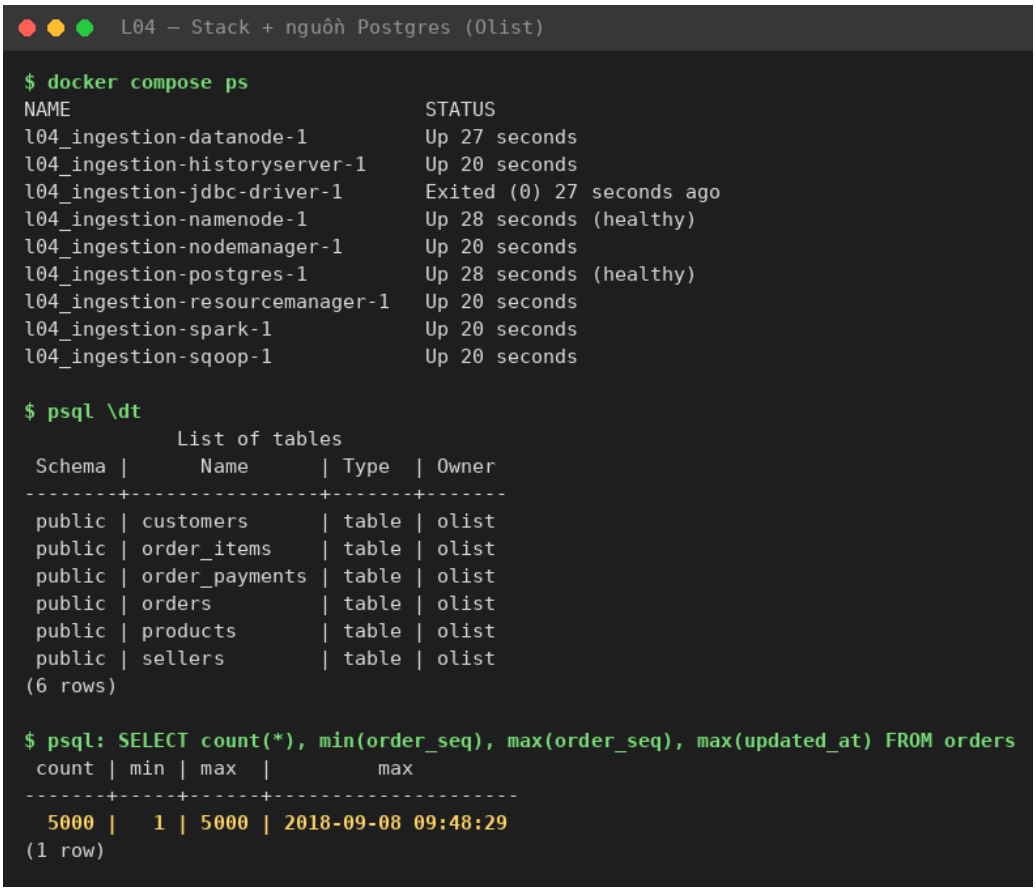
Nạp dữ liệu từ cơ sở dữ liệu OLTP vào HDFS: Sqoop và Spark JDBC, nạp toàn phần và nạp gia tăng

4.1 Mục tiêu và môi trường

Bài thực hành nạp dữ liệu từ PostgreSQL vào HDFS bằng Apache Sqoop 1.4.7 (đã vào Apache Attic từ tháng 6 năm 2021) [1] và Spark JDBC [2], theo hai cách nạp toàn phần và nạp gia tăng, rồi so sánh hai công cụ. PostgreSQL chạy với `log_statement=all` để đọc trực tiếp câu SQL mà mỗi công cụ gửi xuống. Hệ thống chạy bằng Docker Compose trên một máy (Bảng 4.1). Container `jdbc-driver` chỉ tải driver rồi thoát với mã 0; ban đầu chúng tôi tưởng nó lỗi, nhưng đó là hành vi đúng (Hình 4.1).

Bảng 4.1. Môi trường thực nghiệm (stack `l04_ingestion`).

Thành phần	Phiên bản / cấu hình
PostgreSQL	18.6, `log_statement=all`, cơ sở dữ liệu nguồn Olist
HDFS	3.4.3, 1 NameNode + 1 DataNode
YARN	ResourceManager, NodeManager, JobHistory Server (chạy job của Sqoop)
Sqoop	1.4.7 (image khóa học `de-labs/sqoop:1.4.7`)
Spark	4.1.3, local mode, Spark UI cổng 4040
JDBC driver	PostgreSQL JDBC (`/jars/postgresql.jar`), dùng chung cho Sqoop và Spark



Hình 4.1. Trạng thái container của stack L04 và 6 bảng nguồn trong PostgreSQL.

Nguồn là cơ sở dữ liệu Olist tổng hợp gồm 6 bảng (`customers`, `orders`, `order_items`, `order_payments`, `products`, `sellers`). Bảng `orders` có 5,000 dòng, `order_seq` chạy từ 1 đến 5000, `max(updated_at)` là `2018-09-08 09:48:29`. `order_seq` là khóa thay thế kiểu số, tăng theo thời gian mua, còn `order_id`, `customer_id` là chuỗi hash 32 ký tự. Mỗi bảng có cột `updated_at` do trigger tự cập nhật khi UPDATE. Phân bố trạng thái đơn hàng lấy bằng `sqoop eval` nằm ở Bảng 4.2.

Bảng 4.2. Phân bố trạng thái của 5,000 đơn hàng ban đầu.

`order_status`	Số đơn
delivered	4,793
shipped	102
unavailable	41
canceled	30
processing	19
invoiced	11
approved	3
created	1

Script `sql/simulate_changes.sql` chèn 3 khách hàng và 3 đơn mới, đổi 2 đơn `shipped` đầu tiên thành `delivered`. Chúng tôi chạy nó hai lần: trước phần Sqoop gia tăng và giữa hai lần chạy Spark gia tăng. Vì thế lúc chạy Spark, `orders` và `customers` đã có 5,003 dòng, khác giá trị tham chiếu trong README.

4.2 Thực hiện và kết quả

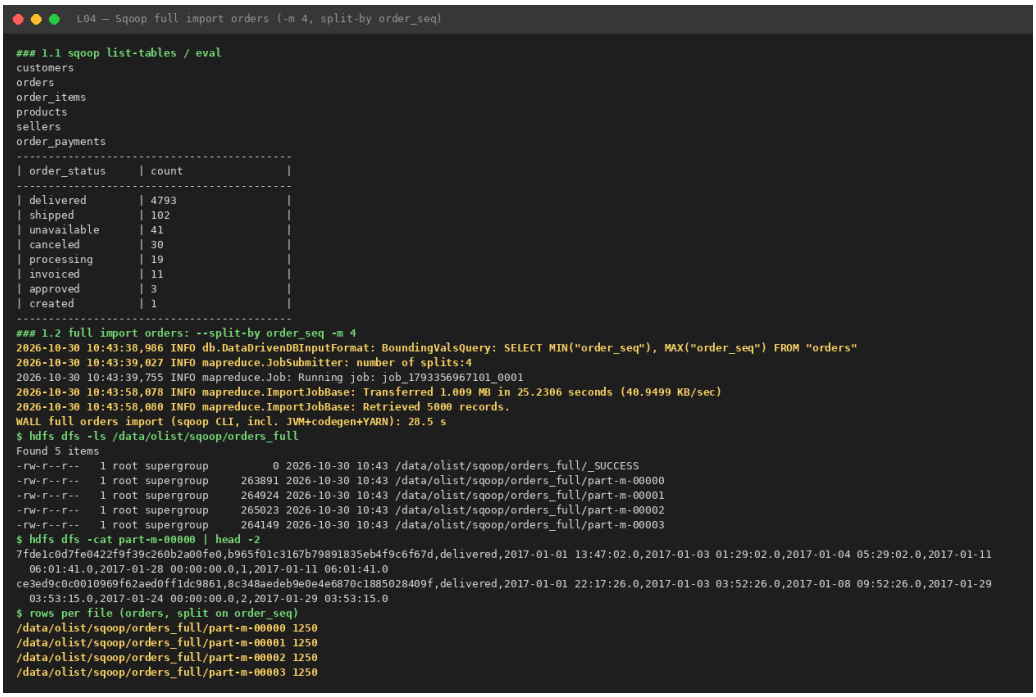
4.2.1 Sqoop nạp toàn phần bảng `orders`

Mỗi lệnh `import` được biên dịch thành một MapReduce job chỉ có pha map [1]. Với `--split-by order_seq -m 4`, Sqoop báo `number of splits:4` và nhật ký PostgreSQL có đúng 4 câu SELECT theo khoảng của 4 map (Bảng 4.3, Hình 4.3; cách chia giải thích ở Mục 4.3.1). Job `job_1793356967101_0001` báo `Transferred 1.009 MB in 25.2306 seconds (40.9499 KB/sec)` và `Retrieved 5000 records`; cả lệnh CLI mất 28.5 s vì còn khởi động JVM, sinh và biên dịch lớp record rồi nộp job lên YARN.

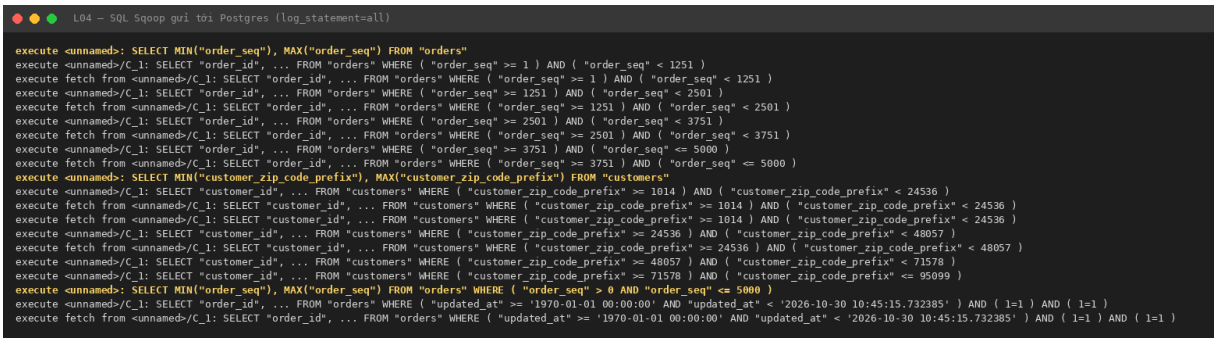
Bảng 4.3. Các split của Sqoop khi nạp `orders` với `--split-by order_seq -m 4`.

Map	Điều kiện WHERE	Số dòng	Kích thước file
0	`order_seq >= 1 AND order_seq < 1251`	1,250	263,891 B
1	`order_seq >= 1251 AND order_seq < 2501`	1,250	264,924 B
2	`order_seq >= 2501 AND order_seq < 3751`	1,250	265,023 B
3	`order_seq >= 3751 AND order_seq <= 5000`	1,250	264,149 B

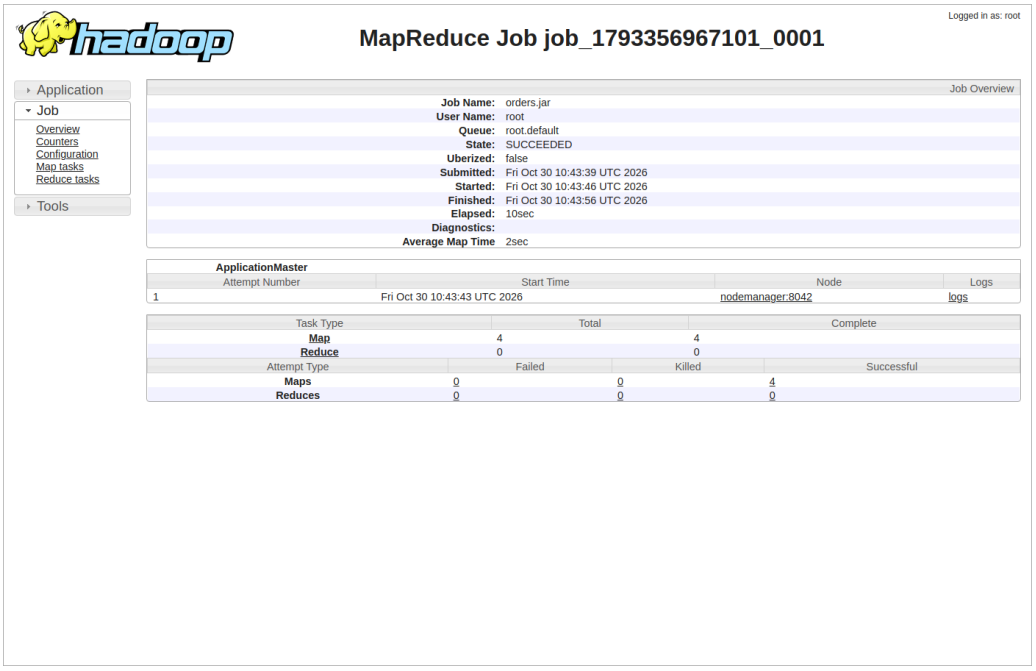
Bốn file `part-m-*` gần bằng nhau cả về số dòng lẫn kích thước (Hình 4.2), dạng văn bản CSV với timestamp `2017-01-03 01:29:02.0` và NULL ghi là `\N`. JobHistory xác nhận 4 map, 0 reduce (Hình 4.4).



Hình 4.2. Sqoop nạp toàn phần `orders`: bounding query, 4 split và số dòng mỗi file.



Hình 4.3. Nhật ký PostgreSQL ghi lại bounding query và 4 câu SELECT theo khoảng `order_seq` do các map task của Sqoop gửi.



Hình 4.4. JobHistory của job `job_1793356967101_0001` (`orders.jar`): 4 map task thành công, không có reduce task.

4.2.2 Sqoop với cột chia bị lệch

Bảng `customers` được nạp với `--split-by customer_zip_code_prefix -m 4` vào `/data/olist/sqoop/customers_full`. Bounding query trả về (1014, 95099), và nhật ký PostgreSQL cho thấy 4 khoảng rộng xấp xỉ 23,521 với các điểm cắt 24536, 48057 và 71578. Số dòng mỗi file đặt cạnh trường hợp `orders` ở Bảng 4.4.

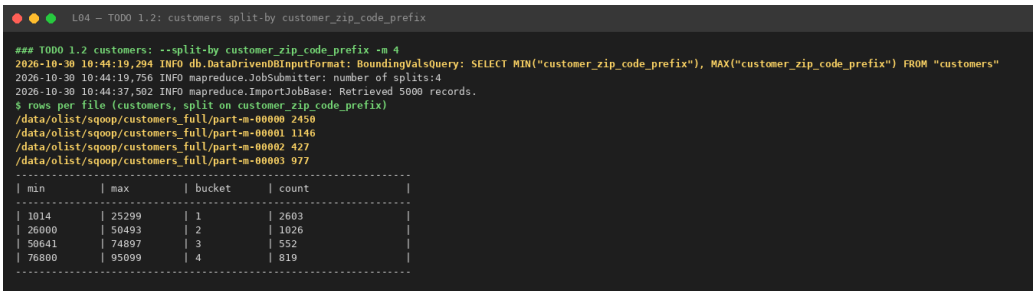
Bảng 4.4. Số dòng mỗi file khi chia theo cột phân bố đều và cột bị lệch.

Bảng / cột split	part-m-00000	part-m-00001	part-m-00002	part-m-00003
`orders` / `order_seq` (1–5000)	1,250	1,250	1,250	1,250
`customers` / `customer_zip_code_prefix` (1014–95099)	**2,450**	1,146	**427**	977

Split đầu nhận gần nửa bảng, gấp khoảng 5.7 lần split thứ ba (Hình 4.5). Đếm độc lập bằng `width_bucket` trên 4 thùng (Bảng 4.5) cho cùng xu hướng: thùng chứa vùng São Paulo (01xxx–19xxx) chiếm phần lớn khách hàng.

Bảng 4.5. Phân bố `customer_zip_code_prefix` theo `width_bucket(..., 1003, 99990, 4)`.

Thùng	min zip	max zip	Số khách hàng
1	1014	25299	2,603
2	26000	50493	1,026
3	50641	74897	552
4	76800	95099	819



Hình 4.5. Số dòng mỗi file khi nạp `customers` chia theo `customer_zip_code_prefix`, thể hiện dữ liệu bị lệch.

4.2.3 Sqoop nạp gia tăng, saved job, merge và export

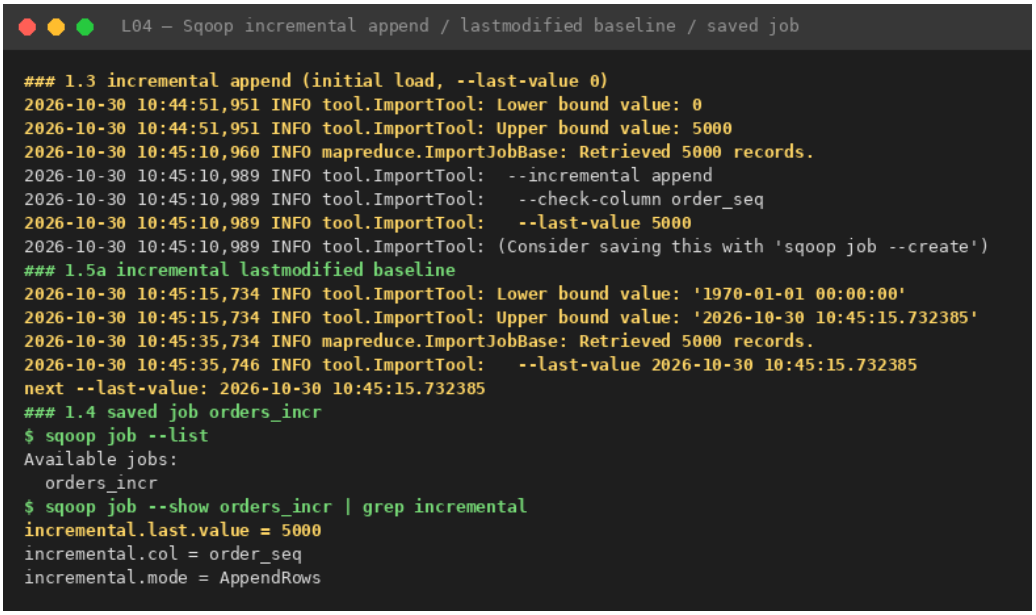
Chuỗi thao tác được tóm ở Bảng 4.6. Lần `append` đầu (`--last-value 0`) có điều kiện `order_seq > 0 AND order_seq <= 5000`, hai split `[1, 2501)` và `[2501, 5000]`. Saved job `orders_incr` lưu `incremental.col = order_seq` và `incremental.mode = AppendRows` trong metastore của Sqoop.

Bảng 4.6. Kết quả các bước nạp gia tăng, merge và export của Sqoop.

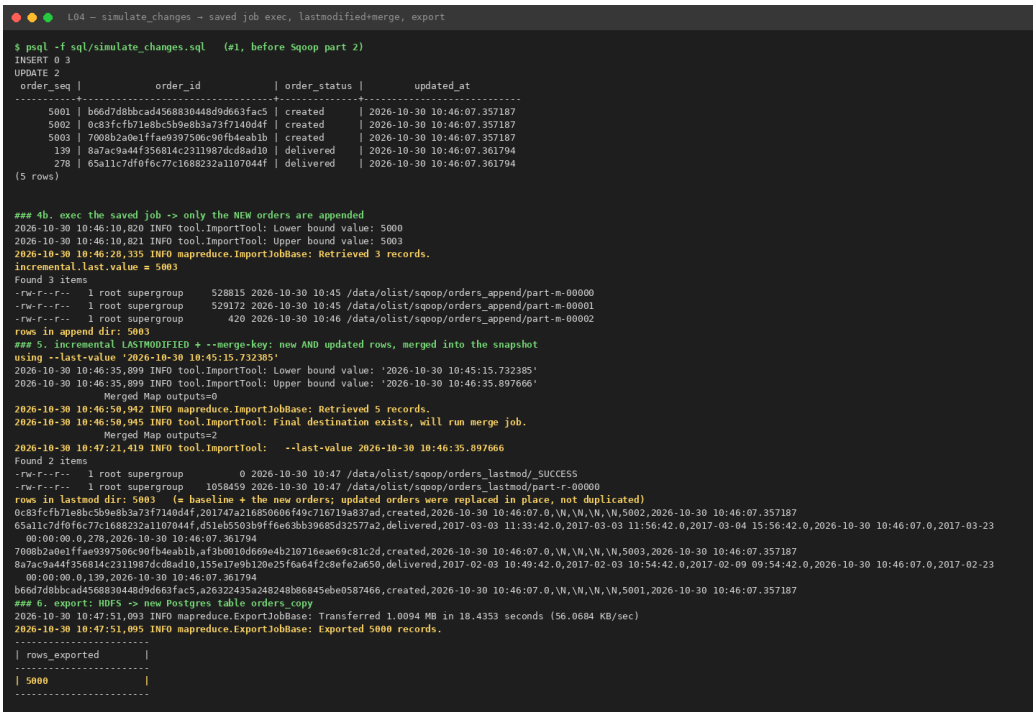
Bước	Cửa sổ	Kết quả
`append`, lần đầu (`-m 2`)	lower 0, upper 5000	5,000 dòng, gợi ý `--last-value 5000`
`lastmodified`, cơ sở (`-m 1`)	`'1970-01-01 00:00:00'` đến `'2026-10-30 10:45:15.732385'`	5,000 dòng
Saved job `orders_incr`		`incremental.last.value = 5000`
`simulate_changes` lần 1		tạo `order_seq` 5001–5003; đơn 139 và 278 chuyển `delivered`
`sqoop job --exec orders_incr`	lower 5000, upper 5003	**3** dòng; `last.value` thành 5003; thư mục có 5,003 dòng

Bước	Cửa sổ	Kết quả
`lastmodified` + `--merge-key order_id` `sqoop export` sang `orders_copy` (`-m 2`)	`'...10:45:15.732385'` đến `'...10:46:35.897666'`	**5** dòng; merge job tạo `part-r-00000` 5,003 dòng 1.0094 MB trong 18.4353 s, 5,000 record

Saved job chạy `... WHERE ( "order_seq" > 5000 AND "order_seq" <= 5003 )` và chỉ thêm một file `part-m-00002` (420 B) cạnh hai file 528,815 B và 529,172 B của lần đầu; đơn 139 và 278 vừa cập nhật không được nạp lại. Với `lastmodified`, do thư mục đích đã tồn tại, Sqoop chạy thêm một merge job (`Merged Map outputs=2`) thay bản cũ của cùng `order_id` bằng bản mới. Kết quả là một file `part-r-00000` (1,058,459 B) không trùng lặp, trong đó đơn 139 và 278 mang trạng thái `delivered` và `updated_at` mới. Một chi tiết chúng tôi chỉ thấy khi đọc log: `lastmodified` dùng cửa sổ nửa mở `updated_at >= lower AND updated_at < upper`, ngược với Spark ở phần sau (Hình 4.6, Hình 4.7).

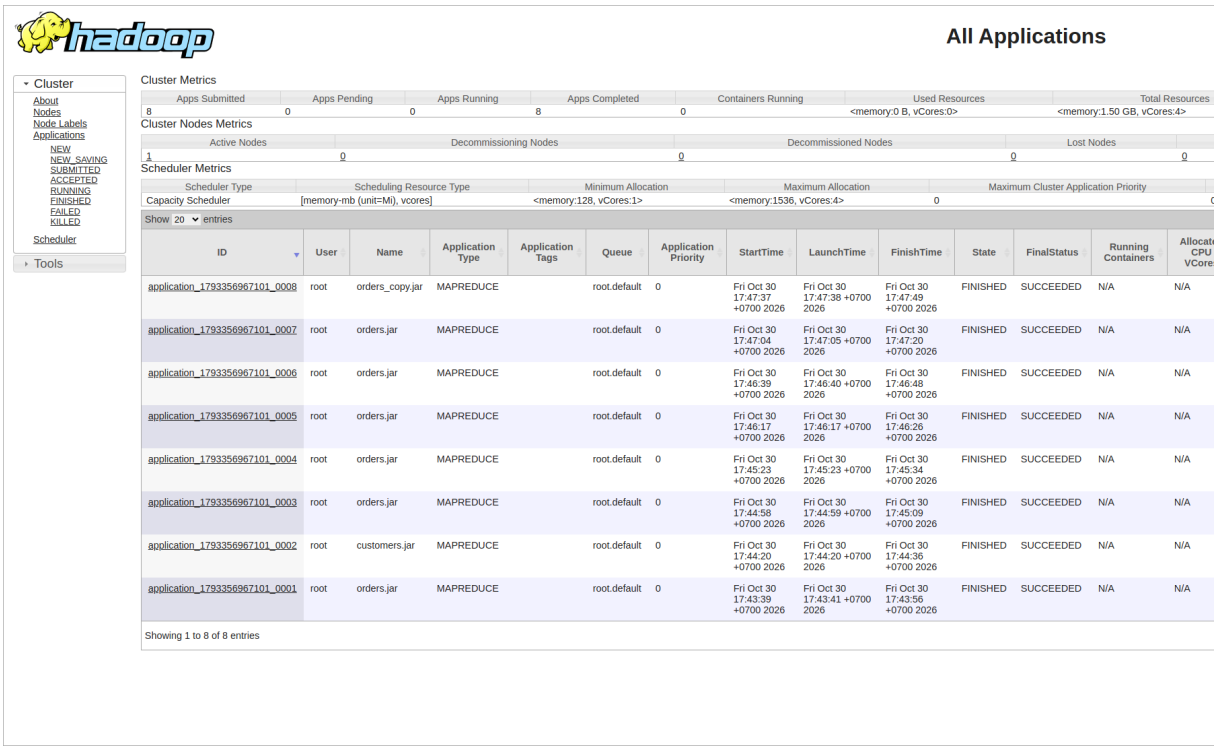


Hình 4.6. Sqoop nạp append lần đầu, nạp `lastmodified` cơ sở và tạo saved job `orders_incr`.

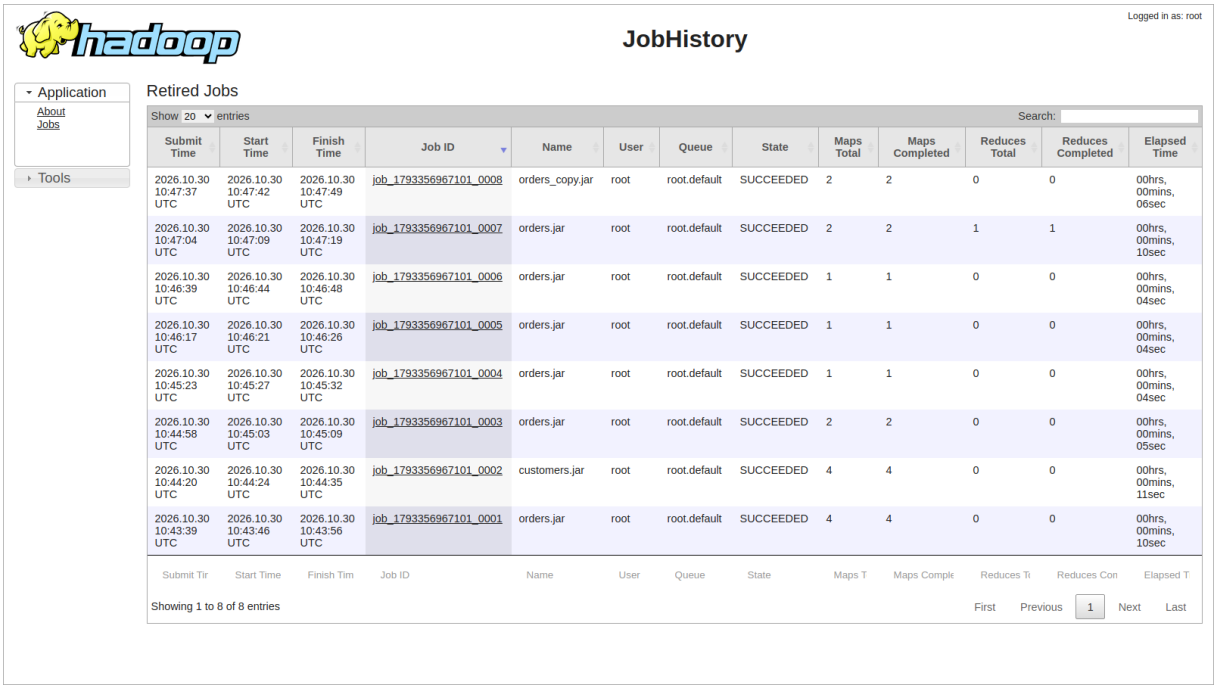


Hình 4.7. Sau `simulate_changes` lần 1: saved job, `lastmodified` kèm merge và export về PostgreSQL.

ResourceManager liệt kê 8 ứng dụng MapReduce do Sqoop nộp (`orders.jar`, `customers.jar`, `orders_copy.jar`), đều thành công (Hình 4.8, Hình 4.9).



Hình 4.8. YARN ResourceManager UI: các MapReduce job do Sqoop nộp, tất cả ở trạng thái FINISHED/SUCCEEDED.



Hình 4.9. JobHistory Server liệt kê các job Sqoop đã hoàn tất.

4.2.4 Spark JDBC: đọc song song và ghi Parquet

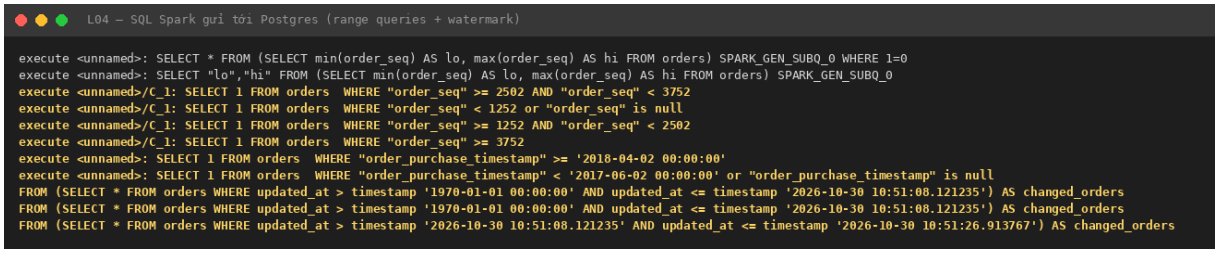
Nguồn JDBC của Spark nhận `partitionColumn`, `lowerBound`, `upperBound` và `numPartitions`, tính bước $s = (\text{upper} - \text{lower})/n$ rồi sinh n mệnh đề WHERE; partition đầu và cuối không có cận ngoài [2]. Script lấy cận thật bằng `SELECT min(order_seq) AS lo, max(order_seq) AS hi FROM orders`, nhận `1 .. 5003`, rồi đọc theo ba cách (Bảng 4.7).

Bảng 4.7. So sánh các cách đọc bảng `orders` bằng Spark JDBC.

Cách đọc	Số partition	Thời gian `count()`	Số dòng mỗi partition
Naive (không có `partitionColumn`)	1	1.17 s	5,003 dòng trong 1 task
`partitionColumn=order_seq`, bounds 1..5003	4	0.22 s	1,251 / 1,250 / 1,250 / 1,252
`partitionColumn=order_purchase_timestamp`, 2017-01-01 đến 2018-09-01	4	không đo	782 / 1,116 / 1,419 / 1,686

Nhật ký PostgreSQL (Hình 4.10) có 4 câu do Spark sinh: `WHERE "order_seq" < 1252 or "order_seq" is null`, `>= 1252 AND < 2502`, `>= 2502 AND < 3752` và `WHERE "order_seq"`

`>= 3752`. Bước chia là $(5003-1)/4 \approx 1250$, nên partition đầu có 1,251 dòng và partition cuối 1,252 dòng. Vì lệnh là `count()`, Spark chỉ gửi `SELECT 1` (column pruning đẩy xuống SQL). Với cột timestamp, partition đầu là `< '2017-06-02 00:00:00' or ... is null`, partition cuối là `>= '2018-04-02 00:00:00'`, và số dòng tăng dần vì lượng đơn Olist tăng theo thời gian. Thời gian `count()` chỉ đo một lần, lần song song chạy sau lần naive nên có thể được lợi từ JVM đã nóng; con số 0.22 s chưa đủ để kết luận về hiệu năng.

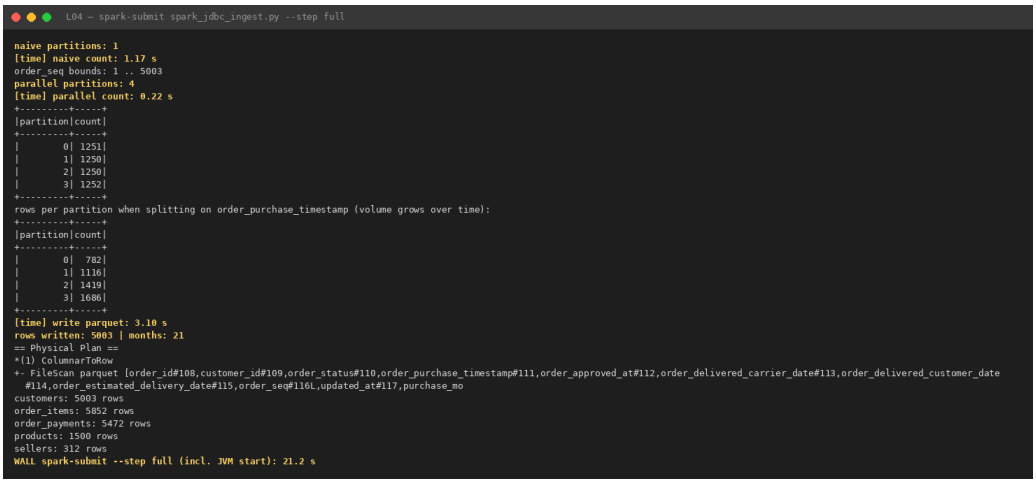


Hình 4.10. Nhật ký PostgreSQL ghi lại các câu `SELECT 1 FROM orders WHERE ...` theo khoảng do Spark sinh ra.

Ghi Parquet với `partitionBy("purchase_month")` mất 3.10 s, tạo 21 thư mục tháng (20 tháng lịch sử và tháng của các đơn mới) chứa đủ 5,003 dòng. Truy vấn lọc `purchase_month = '2018-08'` có physical plan chứa `PartitionFilters: [isnotnull(purchase_month#118), (purchase_month#118 = 2018-08)]`, tức partition pruning có tác dụng. Năm bảng còn lại được nạp mỗi bảng một partition (Bảng 4.8). Cả `spark-submit --step full` mất 21.2 s (Hình 4.11).

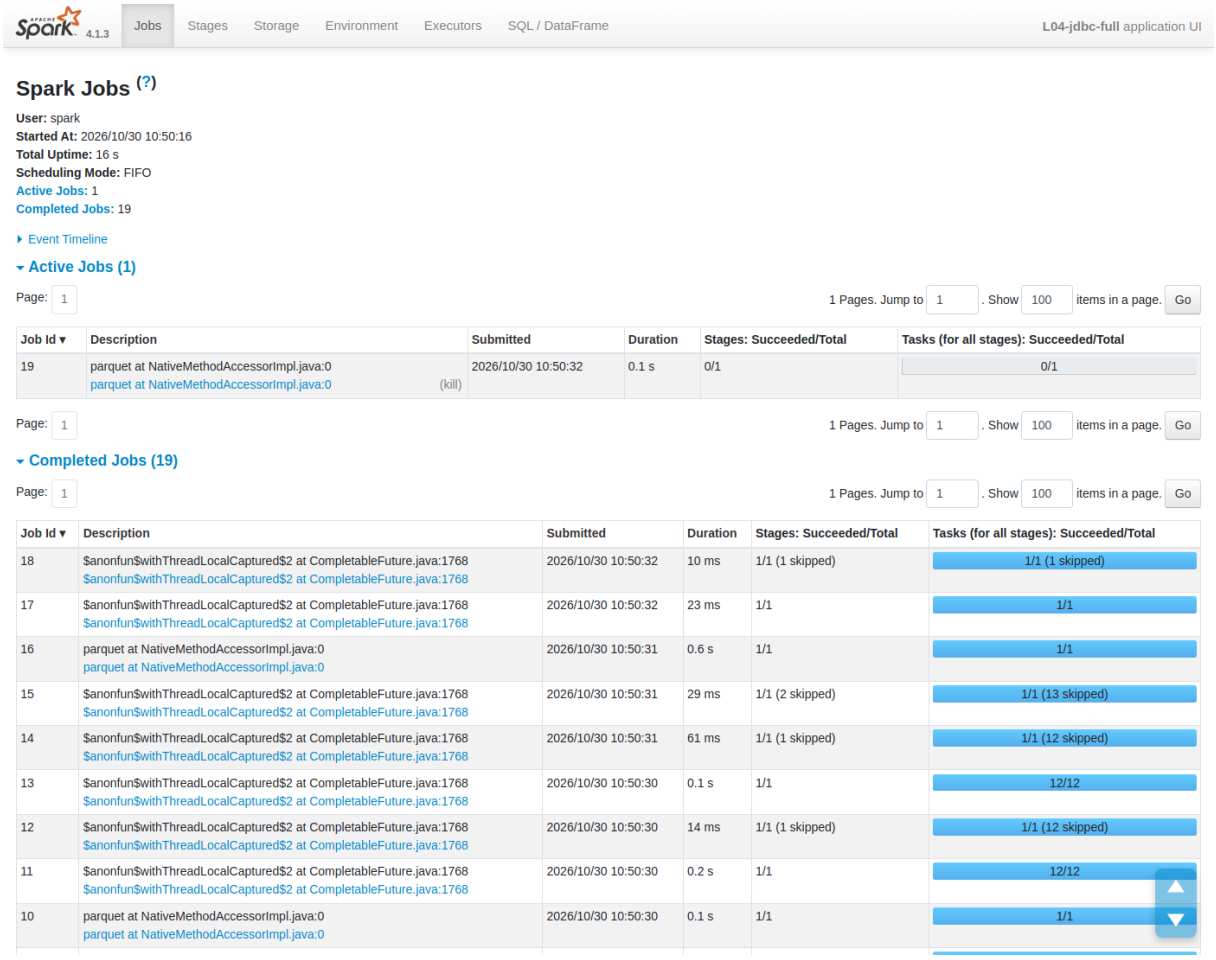
Bảng 4.8. Số dòng các bảng phụ được Spark nạp sang Parquet.

Bảng	Số dòng
customers	5,003
order_items	5,852
order_payments	5,472
products	1,500
sellers	312

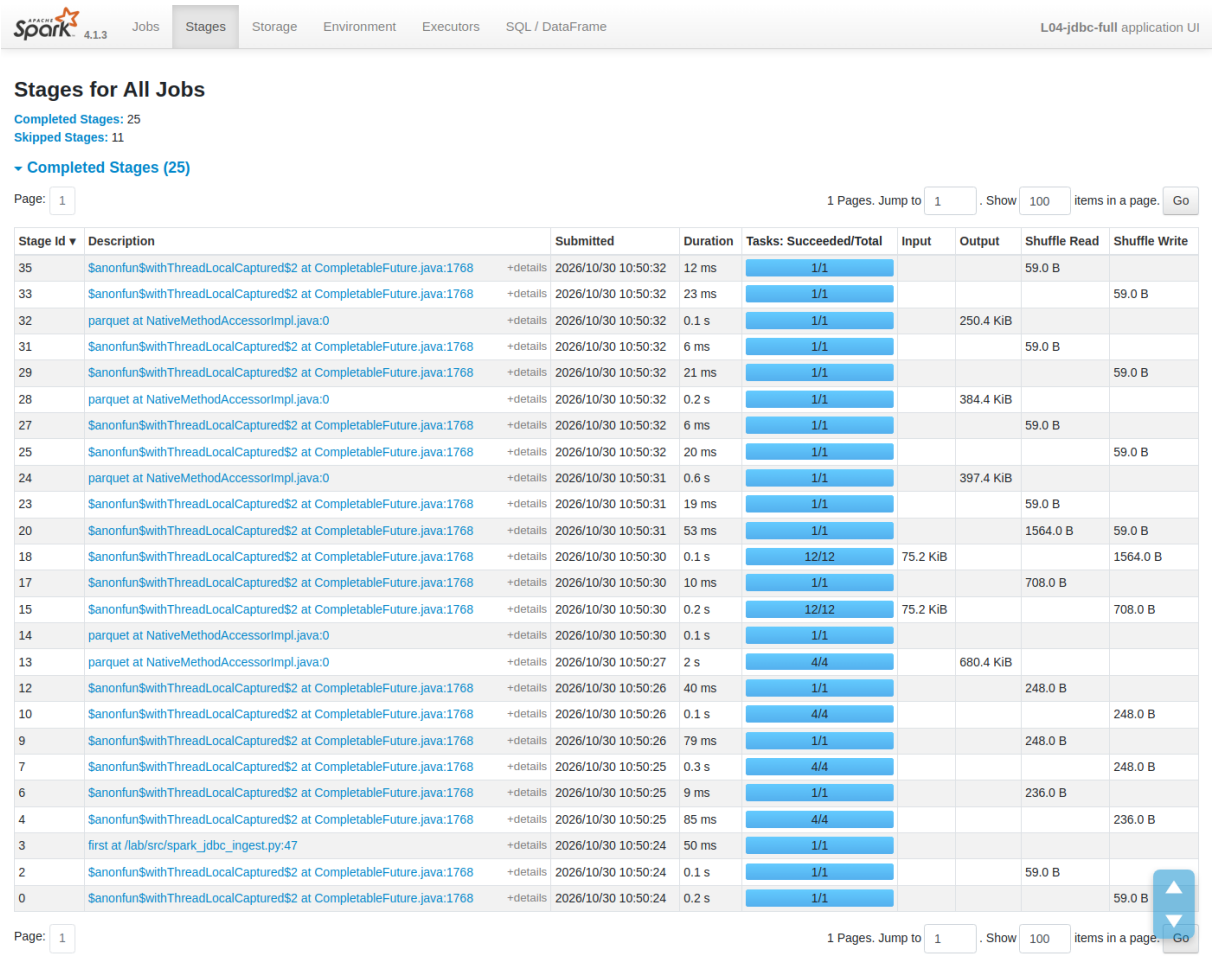


Hình 4.11. Kết quả `spark-submit --step full`, gồm physical plan có `PartitionFilters`.

Trên Spark UI, các stage đọc song song và stage ghi Parquet đều có 4/4 task, đầu ra 680.4 KiB (Hình 4.12, Hình 4.13). Trên HDFS, `/data/olist/spark/orders` có 22 mục (`_SUCCESS` và 21 thư mục `purchase_month=`), dung lượng 680.4 K, còn `/data/olist/sqoop/orders_full` là 1.0 M (Hình 4.14).



Hình 4.12. Spark UI, trang Jobs của ứng dụng `L04-jdbc-full`.



Stage Id ▾	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
35	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	12 ms	1/1			59.0 B	
33	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	23 ms	1/1				59.0 B
32	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/30 10:50:32	0.1 s	1/1		250.4 KiB		
31	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	6 ms	1/1			59.0 B	
29	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	21 ms	1/1				59.0 B
28	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/30 10:50:32	0.2 s	1/1		384.4 KiB		
27	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	6 ms	1/1			59.0 B	
25	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	20 ms	1/1				59.0 B
24	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/30 10:50:31	0.6 s	1/1		397.4 KiB		
23	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:31	19 ms	1/1			59.0 B	
20	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:31	53 ms	1/1			1564.0 B	59.0 B
18	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:30	0.1 s	12/12	75.2 KiB			1564.0 B
17	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:30	10 ms	1/1			708.0 B	
15	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:30	0.2 s	12/12	75.2 KiB			708.0 B
14	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/30 10:50:30	0.1 s	1/1				
13	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/30 10:50:27	2 s	4/4		680.4 KiB		
12	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:26	40 ms	1/1			248.0 B	
10	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:26	0.1 s	4/4				248.0 B
9	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:26	79 ms	1/1			248.0 B	
7	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:25	0.3 s	4/4				248.0 B
6	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:25	9 ms	1/1			236.0 B	
4	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:25	85 ms	4/4				236.0 B
3	first at /lab/src/spark_jdbc_ingest.py:47 +details	2026/10/30 10:50:24	50 ms	1/1				
2	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:24	0.1 s	1/1			59.0 B	
0	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:24	0.2 s	1/1				59.0 B

Hình 4.13. Spark UI, trang Stages: các stage đọc JDBC song song và stage ghi Parquet đều có 4 task.

```
L04 — Parquet partitionBy(purchase_month) trên HDFS

$ hdfs dfs -ls /data/olist/spark/orders | head
Found 22 items
-rw-r--r--   1 spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/_SUCCESS
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-01
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-02
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-03
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-04
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-05
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-06
...
$ hdfs dfs -du -s -h /data/olist/spark/orders /data/olist/sqoop/orders_full
680.4 K  680.4 K  /data/olist/spark/orders
1.0 M    1.0 M    /data/olist/sqoop/orders_full
```

Hình 4.14. Đầu ra Parquet phân vùng theo tháng trên HDFS và so sánh dung lượng với đầu ra văn bản của Sqoop.

4.2.5 Spark nạp gia tăng bằng watermark

Bước `incremental` chốt cận trên bằng `now()` của cơ sở dữ liệu trước khi đọc, đọc cửa sổ `(wm_old, wm_new]` theo `updated_at`, nối vào change log `orders_changes`, dựng snapshot `orders_current` bằng `row_number()` theo `order_id` (sắp `updated_at DESC, _ingested_at DESC`), và chỉ ghi watermark sau cùng:

```
wm_new = jdbc(spark, query="SELECT to_char(now(), 'YYYY-MM-DD
 ↪  HH24:MI:SS.US') AS ts").first().ts
changed_sql = (f"(SELECT * FROM orders WHERE updated_at > timestamp
 ↪  '{wm_old}'"
               f" AND updated_at <= timestamp '{wm_new}') AS
                ↪  changed_orders")
# ... ghi changes (kèm cột _ingested_at) vào orders_changes, rồi dựng
 ↪  snapshot:
log = spark.read.parquet(f"{BASE}/orders_changes")
w = Window.partitionBy("order_id") \
    .orderBy(F.col("updated_at").desc(), F.col("_ingested_at").desc())
current = log.withColumn("_rn", F.row_number().over(w)).filter("_rn =
 ↪  1").drop("_rn")
current.write.mode("overwrite").parquet(f"{BASE}/orders_current")
spark.createDataFrame([(wm_new,)], "watermark string").coalesce(1) \
    .write.mode("overwrite").json(f"{BASE}/_state/orders_watermark")
```

Kết quả ba lần chạy ở Bảng 4.9 và Hình 4.15.

Bảng 4.9. Kết quả ba lần chạy Spark nạp gia tăng (thời điểm ngày 2026-10-30).

Lần chạy	Cửa sổ watermark	Dòng đọc được	Change log	Snapshot
1 (chưa có watermark)	`(1970-01-01 00:00:00, 2026-10-30 10:51:08.121235]`	5,003	5,003	5,003
`simulate_changes` lần 2	tạo 5004–5006, cập nhật đơn 337 và 355			
2	`(10:51:08.121235, 10:51:26.913767]`	**5**	5,008	5,006
3 (không có thay đổi)	`(10:51:26.913767, 10:51:44.317579]`	0	5,008	5,006

Change log giữ mọi phiên bản ($5003 + 5 = 5008$), còn snapshot chỉ giữ phiên bản mới nhất của mỗi đơn ($5003 + 3 = 5006$), nên hai đơn được cập nhật không bị nhân đôi. Lần thứ ba đọc 0 dòng: watermark được lưu đúng và chạy lại không làm sai kết quả. Cận trên của lần trước trùng

khít với cận dưới của lần sau.

L04 — Spark incremental (watermark updated_at) × 3 lần

```
### run 1 (no watermark yet)
$ spark-submit spark_jdbc_ingest.py --step incremental
watermark: (1970-01-01 00:00:00, 2026-10-30 10:51:08.121235]
changed rows since last run: 5003
+---------+--------------------------------+------------+-------------------+
|order_seq|order_id                        |order_status|updated_at         |
+---------+--------------------------------+------------+-------------------+
|13       |8fcda0cbaf5fbdf74b44ec00ad898b16|invoiced    |2017-01-04 08:28:34|
|9        |3e72f2038b375f7cffb84634e058a90d|delivered   |2017-01-08 03:33:57|
|10       |04e531031c4460209ab391b1ecd08b73|delivered   |2017-01-09 14:43:54|
|14       |b9215ed78670f5fcdd902dd20d25ddfa|delivered   |2017-01-10 08:30:04|
|1        |7fde1c0d7fe0422f9f39c260b2a00fe0|delivered   |2017-01-11 06:01:41|
|7        |4dc8028f24056a2745258e340b6f0acb|delivered   |2017-01-11 17:59:16|
|16       |a8b568029838e24b62999f248b689a46|delivered   |2017-01-12 10:25:07|
|5        |45358698bf0058ceccd0ba051b389c3e|delivered   |2017-01-12 18:46:29|
|11       |78565a07e33b7cee0300e9a0dd9b165d|delivered   |2017-01-12 20:29:04|
|4        |94210b260a31271221a4b2385b509571|delivered   |2017-01-13 14:17:16|
+---------+--------------------------------+------------+-------------------+
only showing top 10 rows
change-log rows: 5003 | current snapshot rows: 5003
saved watermark 2026-10-30 10:51:08.121235

### $ psql -f sql/simulate_changes.sql   (#2)
INSERT 0 3
UPDATE 2
 order_seq |             order_id             | order_status |         updated_at
-----------+----------------------------------+--------------+----------------------------
      5004 | 7fcdf5479024a8742073bb0ef6935142 | created      | 2026-10-30 10:51:17.171709
      5005 | 074912012cd67af7286f9d65fda97044 | created      | 2026-10-30 10:51:17.171709
      5006 | 87aa76620ff952a3c59e109b75a627fb | created      | 2026-10-30 10:51:17.171709
       337 | 9914cbc8175f775efa350b76cb42c257 | delivered    | 2026-10-30 10:51:17.175487
       355 | d95630b6b34b9e5fc509a5d750b0fa66 | delivered    | 2026-10-30 10:51:17.175487
(5 rows)

### run 2 (after simulate_changes)
$ spark-submit spark_jdbc_ingest.py --step incremental
watermark: (2026-10-30 10:51:08.121235, 2026-10-30 10:51:26.913767]
changed rows since last run: 5
+---------+--------------------------------+------------+--------------------------+
|order_seq|order_id                        |order_status|updated_at                |
+---------+--------------------------------+------------+--------------------------+
|5004     |7fcdf5479024a8742073bb0ef6935142|created     |2026-10-30 10:51:17.171709|
|5005     |074912012cd67af7286f9d65fda97044|created     |2026-10-30 10:51:17.171709|
|5006     |87aa76620ff952a3c59e109b75a627fb|created     |2026-10-30 10:51:17.171709|
|337      |9914cbc8175f775efa350b76cb42c257|delivered   |2026-10-30 10:51:17.175487|
|355      |d95630b6b34b9e5fc509a5d750b0fa66|delivered   |2026-10-30 10:51:17.175487|
+---------+--------------------------------+------------+--------------------------+
change-log rows: 5008 | current snapshot rows: 5006
saved watermark 2026-10-30 10:51:26.913767

### run 3 (no changes)
$ spark-submit spark_jdbc_ingest.py --step incremental
watermark: (2026-10-30 10:51:26.913767, 2026-10-30 10:51:44.317579]
changed rows since last run: 0
+---------+--------+------------+----------+
|order_seq|order_id|order_status|updated_at|
+---------+--------+------------+----------+
+---------+--------+------------+----------+
change-log rows: 5008 | current snapshot rows: 5006
saved watermark 2026-10-30 10:51:44.317579
```

Hình 4.15. Ba lần chạy Spark `--step incremental`.

4.2.6 So sánh Sqoop và Spark JDBC

Bảng so sánh của Phần 3 trong đề, kèm thời gian đo được, nằm ở Bảng 4.10.

Bảng 4.10. So sánh Sqoop và Spark JDBC khi nạp bảng `orders`.

Tiêu chí	Sqoop 1.4.7 (MapReduce)	Spark JDBC
Tham số song song	`-m 4` + `--split-by`	`numPartitions=4` + `partitionColumn`
Cách tính khoảng	Sqoop tự chạy `SELECT MIN, MAX` (BoundingValsQuery)	người dùng truyền `lowerBound`/`upperBound` (chỉ là stride)
Nạp gia tăng	Có sẵn: `--incremental append/lastmodified`, saved job lưu last value	Tự viết: truy vấn watermark + file trạng thái
Xử lý UPDATE	`--merge-key` (thêm một MR job)	`row_number()` khử trùng lặp, hoặc MERGE vào table format
Định dạng đầu ra	Văn bản CSV, 1.0 MB	Parquet Snappy phân vùng theo tháng, 680.4 KB
Biến đổi khi nạp	Không (chỉ SQL qua `--query`)	Đầy đủ DataFrame API (`withColumn`, `partitionBy`, …)
Thời gian nạp `orders`	25.2 s MR job (28.5 s cả lệnh)	3.10 s ghi Parquet (21.2 s cả `spark-submit` 6 bảng, gồm khởi động JVM)
Trạng thái dự án	Apache Attic (ngừng từ 2021)	Đang phát triển

Sqoop chậm chủ yếu vì mỗi lệnh phải sinh mã, biên dịch lớp record và dựng một MapReduce job riêng. Spark dùng một SparkSession cho cả 6 bảng và ghi thẳng ra định dạng cột có nén, nên đầu ra nhỏ hơn khoảng một phần ba. Tuy vậy mọi thứ chạy trên một máy với khoảng 1 MB dữ liệu, thời gian đo phản ánh chi phí khởi động nhiều hơn thông lượng, nên chênh lệch thời gian chỉ nên hiểu theo nghĩa định tính.

4.3 Trả lời câu hỏi checkpoint

4.3.1 Sqoop chạy câu SQL nào trước khi import, và từ đó ra 4 split bằng cách nào?

Sqoop chạy bounding query `SELECT MIN("order_seq"), MAX("order_seq") FROM "orders"` và nhận (1, 5000) (Hình 4.3). Bộ chia số nguyên cắt đoạn này thành 4 khoảng bằng

nhau về giá trị, rộng khoảng $(5000 - 1)/4 \approx 1250$; mỗi map chạy `WHERE order_seq >= lo AND order_seq < hi`, khoảng cuối dùng <= để lấy cả MAX (Bảng 4.3). Vì chia theo giá trị chứ không theo số dòng, các split chỉ đều khi cột chia phân bố đều.

4.3.2 Vì sao các file `customers` không đều, và nếu dùng `--split-by customer_id` thì sao?

Sqoop chia đều theo giá trị, nhưng zip code của khách hàng Olist dồn về São Paulo, nên khoảng đầu [1014, 24536) nhận gần nửa bảng còn khoảng [48057, 71578) rất ít (Bảng 4.4, Bảng 4.5). Đây là data skew: task nặng nhất quyết định thời gian của cả job. Còn `customer_id` là chuỗi hash kiểu text; mặc định Sqoop từ chối chia theo cột text và báo lỗi, trừ khi bật `-Dorg.apache.sqoop.splitter.allow_text_splitter=true`. Kể cả khi bật, việc chia theo thứ tự chuỗi phụ thuộc collation nên split vẫn có thể lệch hoặc rỗng. Cột chia tốt nhất là cột số, phân bố đều và có chỉ mục, như `order_seq`.

4.3.3 Với `lowerBound=1, upperBound=5003, numPartitions=4`, Spark có đọc dòng `order_seq` = 9,999 không?

Có đọc. `lowerBound` và `upperBound` chỉ dùng để tính bước chia, không phải bộ lọc: partition cuối là `order_seq >= 3752` không có cận trên (Hình 4.10), nên dòng 9,999 rơi vào partition cuối; giá trị nhỏ hơn `lowerBound` và NULL rơi vào partition đầu. Cận sai không làm mất dữ liệu, nhưng partition hai đầu sẽ phình to và gây skew. Vì vậy script lấy MIN/MAX thật trước khi đọc, nhờ đó 4 partition gần đều (Bảng 4.7).

4.3.4 Vì sao phải chốt cận trên trước khi đọc, và chỉ lưu watermark sau khi ghi xong?

Script lấy `wm_new = now()` trước khi đọc, rồi truy vấn `updated_at > wm_old AND updated_at <= wm_new` (Mục 4.2.5). Nếu lấy `now()` sau khi đọc, dòng được cập nhật trong lúc đang đọc sẽ có `updated_at` nhỏ hơn watermark mới mà không nằm trong dữ liệu vừa đọc, và lần sau bị bỏ qua luôn. Cận trên cố định còn bảo đảm các partition JDBC song song cùng nhìn một cửa sổ, và các cửa sổ liên tiếp liền nhau mà không chồng lên nhau, đúng như cận của ba lần chạy ở Bảng 4.9.

Về thứ tự lưu, nếu job chết giữa chừng thì watermark cũ vẫn còn và lần chạy lại đọc lại cả cửa sổ. Dữ liệu có thể trùng nhưng không mất (*at-least-once*), và phần trùng được loại bằng `row_number()` khi dựng snapshot. Ngược lại, nếu lưu watermark trước rồi bước ghi thất bại, cửa sổ đó sẽ không bao giờ được đọc lại.

4.3.5 Chế độ append hay `lastmodified` bắt được UPDATE, và chế độ nào bắt được DELETE?

`append` chỉ dựa vào `order_seq > last_value` nên chỉ bắt được INSERT: saved job chỉ lấy các đơn mới (Bảng 4.6). `lastmodified` dựa vào `updated_at` nên bắt được cả INSERT và UPDATE (thêm đơn 139 và 278); khi đó cần `--merge-key` để gộp bản cũ với bản mới. Không chế độ nào bắt được DELETE, vì dòng đã xóa không còn trong bảng để truy vấn.

4.3.6 Watermark trên `updated_at` không giải quyết được gì, và CDC khác ở đâu?

Thứ nhất là DELETE: dòng bị xóa không bao giờ xuất hiện trong truy vấn, nên bản sao trên HDFS giữ lại dữ liệu đã không còn ở nguồn, trừ khi ứng dụng dùng soft delete hoặc so sánh cả snapshot. Thứ hai là các lần UPDATE trung gian: nếu một đơn đổi từ `shipped` sang `delivered` rồi `canceled` trong cùng một chu kỳ, chỉ trạng thái cuối được thấy. Ngoài ra `updated_at` có thể không đáng tin: ứng dụng quên cập nhật cột, đồng hồ máy chủ lệch, hoặc một transaction dài commit sau khi watermark đã vượt qua timestamp của nó và dòng đó bị sót. Cách này cũng tạo tải quét lên nguồn (cần chỉ mục trên `updated_at`), và độ trễ bằng chu kỳ lập lịch.

CDC (Debezium, sẽ làm ở L08) không truy vấn bảng mà đọc write-ahead log (WAL) của PostgreSQL qua logical decoding [3], [4]. Mọi INSERT, UPDATE và DELETE thành sự kiện theo thứ tự commit, kèm ảnh trước và sau, không phụ thuộc `updated_at`, gần như không tạo tải truy vấn và trễ chỉ vài giây.

4.3.7 Điều gì xảy ra với cơ sở dữ liệu nguồn khi đặt `-m 50` hoặc `numPartitions=50`?

Nguồn nhận 50 kết nối đồng thời, mỗi kết nối một range scan. Con số này dễ chạm `max_connections` (PostgreSQL mặc định 100) khiến ứng dụng production không lấy được kết nối, và các truy vấn tranh CPU, I/O, buffer cache với tải OLTP; nếu cơ sở dữ liệu đã là nút thắt thì tốc độ gần như không tăng. Với bảng 5,000 dòng, mỗi task chỉ còn khoảng 100 dòng, chi phí khởi tạo lớn hơn lợi ích. Thực tế nên giữ khoảng 4–8 kết nối, thống nhất với DBA, đọc từ read replica hoặc ngoài giờ cao điểm.

4.4 Kết luận

Bài đã hoàn thành đủ phần Sqoop (nạp toàn phần, cột chia lệch, `append`, saved job, `lastmodified` kèm merge, export), phần Spark JDBC (đọc song song, Parquet phân vùng theo tháng, nạp gia tăng bằng watermark) và bảng so sánh. Nhật ký SQL cho thấy cả hai công cụ chia bảng theo khoảng giá trị, nên `order_seq` cho các task đều còn zip code và timestamp thì

lệch. `append` chỉ bắt INSERT, `lastmodified` và watermark bắt thêm UPDATE, còn DELETE và phiên bản trung gian cần CDC.

Tài liệu tham khảo

[1] The Apache Software Foundation, "Sqoop User Guide (v1.4.7)," 2017. https://sqoop.apache.org/docs/1.4

[2] The Apache Software Foundation, "JDBC To Other Databases," Spark SQL Guide. https://spark.apache.org/docs/latest/sql-data-sources-jdbc.html

[3] The PostgreSQL Global Development Group, "Logical Decoding," PostgreSQL Documentation. https://www.postgresql.org/docs/current/logicaldecoding.html

[4] Debezium Community, "Debezium Connector for PostgreSQL," Debezium Documentation. https://debezium.io/documentation/reference/stable/connectors/postgresql.html

Phụ lục: Tệp nộp kèm

Đoạn mã tự viết chính đã trích trong Mục 4.2.5; tệp đầy đủ ở Bảng 4.11.

Bảng 4.11. Danh mục tệp nộp kèm báo cáo L04.

Tệp	Nội dung
`sqoop_part1.sh`, `spark_jdbc_ingest.py`	Lệnh Sqoop phần 1; script Spark JDBC đã hoàn thành
`01_*.txt` đến `09_*.txt`, `img/`	Log terminal, nhật ký SQL của PostgreSQL, ảnh chụp terminal và UI

Chương 5

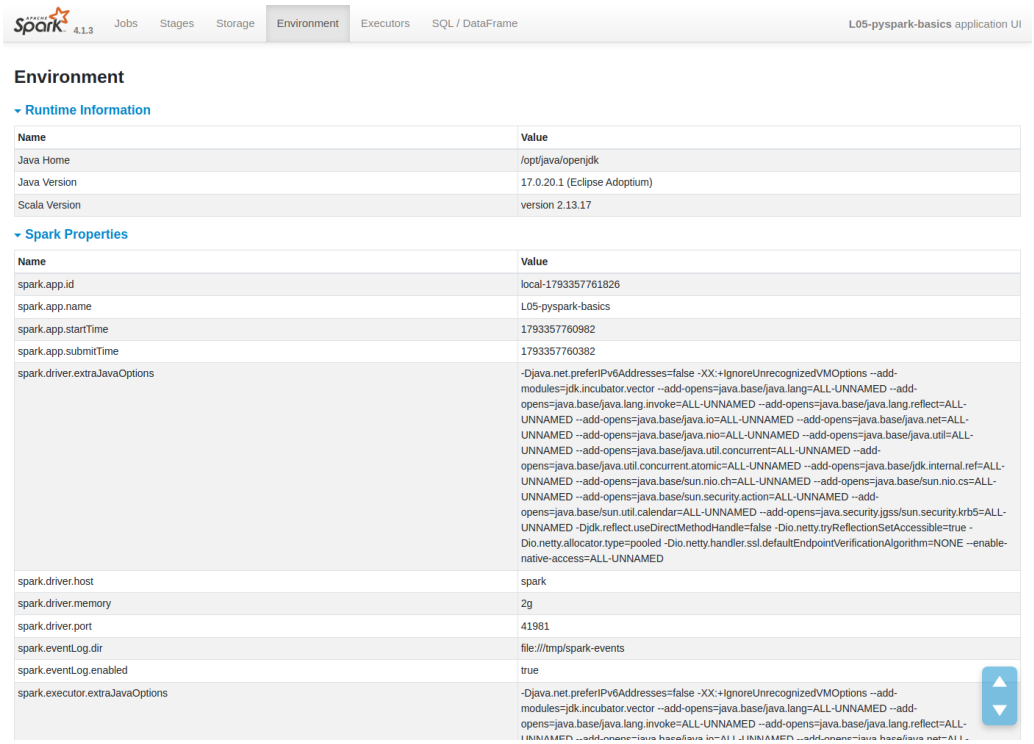
PySpark cơ bản: DataFrame, Spark SQL, kế hoạch thực thi và Spark UI

5.1 Mục tiêu và môi trường

Bài này dùng API DataFrame của PySpark để xử lý dữ liệu taxi vàng New York tháng 01/2024 [5]: đọc Parquet và CSV, kiểm chứng lazy evaluation, làm sạch dữ liệu (kèm chế độ ANSI của Spark 4), join với bảng khu vực, tổng hợp, dùng hàm cửa sổ, viết lại bằng Spark SQL, đọc kế hoạch vật lý cùng Spark UI và ghi Parquet phân vùng theo ngày. Cấu hình ở Bảng 5.1 lấy từ `conf/spark-defaults.conf` và được kiểm tra lại trong tab Environment (Hình 5.1); dữ liệu đầu vào ở Bảng 5.2.

Bảng 5.1. Môi trường thực nghiệm và cấu hình Spark.

Thành phần	Giá trị
Phiên bản Spark	4.1.3 (image Docker `spark:4.1.3-python3`)
Java / Scala / Python	Java 17 (17.0.20.1, Eclipse Adoptium) / Scala 2.13.17 / Python 3.10
Chế độ chạy	`local[4]`: driver và 4 luồng task trong một JVM
Bộ nhớ driver	2 GB (`spark.driver.memory=2g`)
`spark.sql.shuffle.partitions`	8 (thay cho mặc định 200)
Khác	`spark.sql.session.timeZone=UTC`, `spark.sql.ansi.enabled=true` (mặc định Spark 4)
Event log, UI	event log ghi vào `file:///tmp/spark-events`; UI cổng 4040, giữ sống bằng `--hold 300`



Hình 5.1. Tab Environment của Spark UI: Java 17.0.20.1, Scala 2.13.17, `spark.driver.memory=2g`, event log được bật.

Bảng 5.2. Dữ liệu đầu vào.

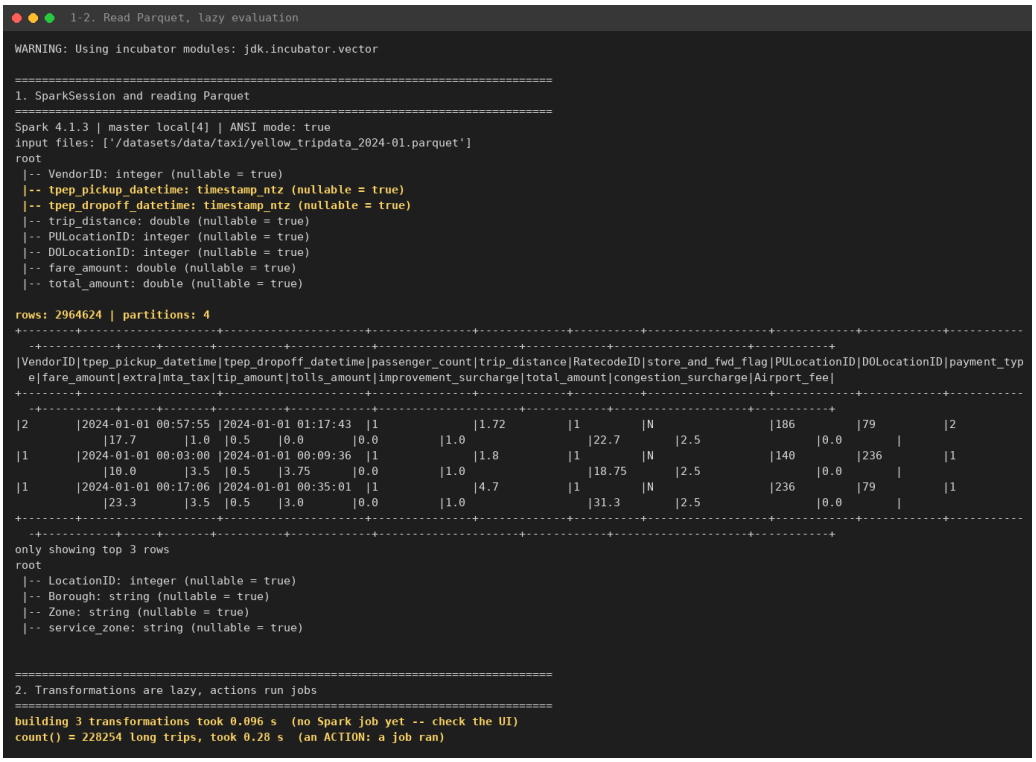
Tệp	Định dạng	Kích thước	Số dòng	Số cột
`yellow_tripdata_2024-01.parquet`	Parquet	khoảng 48 MB	2,964,624	19
`taxi_zone_lookup.csv`	CSV có header	12.0 KiB	265	4

Toàn bộ các bước nằm trong `l05_basics.py`, chạy bằng `spark-submit` trong container. Để so sánh hai cách ghi Parquet mà không sửa mã, chúng tôi thêm cờ `--no-repartition`. Lần đầu chạy với cờ này (`01_run_no_repartition.txt`), lần thứ hai là lần chạy chính có `repartition` và `--hold 300` để chụp Spark UI (`03_run_with_repartition.txt`). Số liệu bên dưới lấy từ lần chạy chính nếu không ghi chú khác; hai lần cho kết quả giống hệt, trừ phần ghi Parquet và thời gian đo. Lần chạy chính kết thúc với mã 137 vì tiến trình bị dừng trong lúc `--hold`, khi mọi kết quả đã in xong, nên số liệu không bị ảnh hưởng.

5.2 Thực hiện và kết quả

5.2.1 Đọc dữ liệu và lazy evaluation

Spark đọc được 19 cột (hai cột thời gian kiểu `timestamp_ntz`), 2,964,624 dòng, chia thành 4 partition đầu vào (Hình 5.2). Ba transformation (`filter`, `withColumn`, `select`) được dựng trong 0.096 s và không sinh job nào; `count()` ngay sau đó trả về 228,254 chuyến dài hơn 10 dặm sau 0.28 s. Ở lần chạy không `repartition`, hai con số là 0.076 s và 0.26 s, cùng bậc độ lớn.



Hình 5.2. Đọc dữ liệu: schema 19 cột, 2,964,624 dòng, 4 partition; dựng ba transformation mất 0.096 s, `count()` mất 0.28 s.

5.2.2 Làm sạch dữ liệu và chế độ ANSI

Số NULL của mọi cột được đếm trong một phép `select` duy nhất: 14 cột không có NULL, còn đúng năm cột `passenger_count`, `RatecodeID`, `store_and_fwd_flag`, `congestion_surcharge`, `Airport_fee` cùng có 140,162 NULL (Hình 5.3). Số dòng vi phạm từng luật được đếm bằng tổng hợp có điều kiện trong một lần quét (Bảng 5.3).

Bảng 5.3. Số dòng vi phạm từng luật làm sạch (một dòng có thể vi phạm nhiều luật).

Luật	Số dòng vi phạm
Thời điểm đón nằm ngoài tháng của tệp (2024-01)	18
Giá cước `fare_amount` ≤ 0	38,341

Luật	Số dòng vi phạm
Tổng tiền `total_amount` < 0	35,504
Quãng đường ≤ 0 hoặc > 100 dặm	60,430
Thời lượng ≤ 0 hoặc > 6 giờ	2,642
Giữ lại sau làm sạch	**2,867,786 / 2,964,624 (96.73 %)**

Tổng số dòng bị loại là 96,838, ít hơn tổng cột bên phải vì các luật chồng lên nhau: chuyến có giá cước âm thường cũng có tổng tiền âm. Dòng có NULL không bị loại mà được điền mặc định `passenger_count=1`, `RatecodeID=1`, `congestion_surcharge=0.0`, `Airport_fee=0.0`. DataFrame `clean` sau đó được `cache()`.

```
3. TODO 1-2: NULL counts and cleaning rules

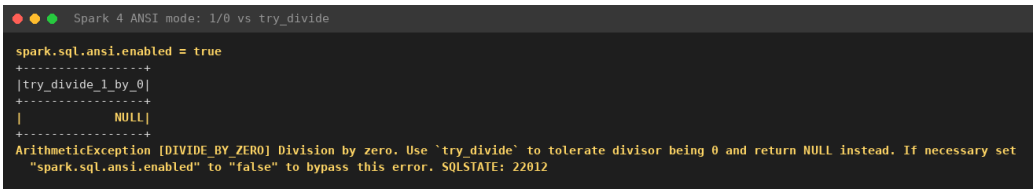
3. Data quality and cleaning
================================================================================
-RECORD 0-----------------------
 VendorID              | 0
 tpep_pickup_datetime  | 0
 tpep_dropoff_datetime | 0
 passenger_count       | 140162
 trip_distance         | 0
 RatecodeID            | 140162
 store_and_fwd_flag    | 140162
 PULocationID          | 0
 DOLocationID          | 0
 payment_type          | 0
 fare_amount           | 0
 extra                 | 0
 mta_tax               | 0
 tip_amount            | 0
 tolls_amount          | 0
 improvement_surcharge | 0
 total_amount          | 0
 congestion_surcharge  | 140162
 Airport_fee           | 140162

-RECORD 0----------------------------------
 pickup outside the file's month(s) | 18
 negative or zero fare              | 38341
 negative total                     | 35504
 distance <= 0 or > 100 miles       | 60430
 duration <= 0 or > 6 h             | 2642

kept 2,867,786 of 2,964,624 rows (96.73 %)
```

Hình 5.3. Đếm NULL theo cột và số dòng vi phạm từng luật làm sạch; giữ lại 2,867,786 dòng (96.73 %).

Từ Spark 4.0, ANSI mode bật mặc định nên chia cho 0 sẽ báo lỗi thay vì trả NULL như Spark 3 [3]; phép chia `tip_amount / fare_amount` thông thường sẽ làm hỏng cả job nếu gặp mẫu số bằng 0, nên tỷ lệ tip được tính bằng `F.try_divide`. Đoạn thử riêng (Hình 5.4) xác nhận `spark.sql.ansi.enabled = true`: `try_divide(1, 0)` trả NULL, còn `1/0` ném `ArithmeticException [DIVIDE_BY_ZERO]` với SQLSTATE 22012.



Hình 5.4. Chế độ ANSI: `try_divide` trả NULL, phép chia thường ném lỗi `[DIVIDE_BY_ZERO]`.

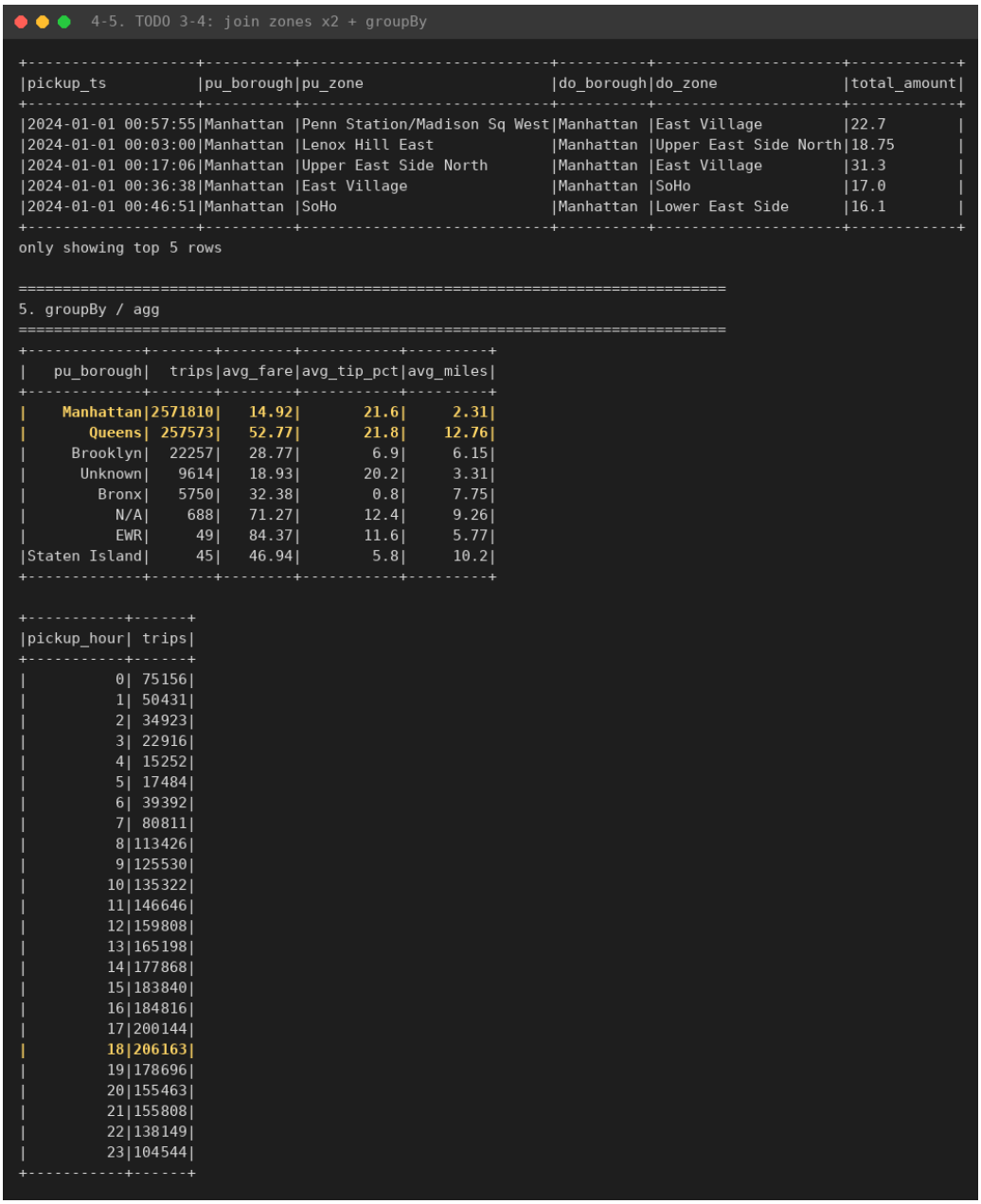
5.2.3 Join với bảng khu vực và tổng hợp

Bảng khu vực được join hai lần, theo điểm đón và điểm trả. Trước mỗi lần join, cột được đổi tên (`PULocationID`/`pu_borough`/`pu_zone` và `DOLocationID`/`do_borough`/`do_zone`); nếu không, hai cột `Borough` trùng tên sẽ gây lỗi `AMBIGUOUS_REFERENCE`. Kết quả ở Hình 5.5 và Bảng 5.4.

Bảng 5.4. Tổng hợp theo quận đón sau làm sạch (`by_borough`).

Quận đón	Số chuyến	Giá cước TB	Tip TB (%)	Quãng đường TB (dặm)
Manhattan	2,571,810	14.92	21.6	2.31
Queens	257,573	52.77	21.8	12.76
Brooklyn	22,257	28.77	6.9	6.15
Unknown	9,614	18.93	20.2	3.31
Bronx	5,750	32.38	0.8	7.75
N/A	688	71.27	12.4	9.26
EWR	49	84.37	11.6	5.77
Staten Island	45	46.94	5.8	10.2

Manhattan chiếm khoảng 89.7 % số chuyến, toàn chuyến ngắn và rẻ. Giá cước trung bình ở Queens cao gấp khoảng 3.5 lần Manhattan, dễ hiểu vì JFK và LaGuardia đều nằm ở Queens. Tip ở Bronx chỉ 0.8 %; chúng tôi đoán phần lớn tiền boa ở đây trả bằng tiền mặt nên không được ghi nhận, nhưng dữ liệu không đủ để khẳng định. Theo giờ đón, khung 18:00 đông nhất với 206,163 chuyến, khung 04:00 vắng nhất với 15,252 chuyến.




```
+-------------------+----------+--------------------------+----------+--------------------+------------+
|pickup_ts          |pu_borough|pu_zone                   |do_borough|do_zone             |total_amount|
+-------------------+----------+--------------------------+----------+--------------------+------------+
|2024-01-01 00:57:55|Manhattan |Penn Station/Madison Sq West|Manhattan |East Village        |22.7        |
|2024-01-01 00:03:00|Manhattan |Lenox Hill East           |Manhattan |Upper East Side North|18.75       |
|2024-01-01 00:17:06|Manhattan |Upper East Side North     |Manhattan |East Village        |31.3        |
|2024-01-01 00:36:38|Manhattan |East Village              |Manhattan |SoHo                |17.0        |
|2024-01-01 00:46:51|Manhattan |SoHo                      |Manhattan |Lower East Side     |16.1        |
+-------------------+----------+--------------------------+----------+--------------------+------------+
only showing top 5 rows

================================================================================
5. groupBy / agg
================================================================================
+-------------+-------+--------+-----------+---------+
|   pu_borough|  trips|avg_fare|avg_tip_pct|avg_miles|
+-------------+-------+--------+-----------+---------+
|    Manhattan|2571810|   14.92|       21.6|     2.31|
|       Queens| 257573|   52.77|       21.8|    12.76|
|     Brooklyn|  22257|   28.77|        6.9|     6.15|
|      Unknown|   9614|   18.93|       20.2|     3.31|
|        Bronx|   5750|   32.38|        0.8|     7.75|
|          N/A|    688|   71.27|       12.4|     9.26|
|          EWR|     49|   84.37|       11.6|     5.77|
|Staten Island|     45|   46.94|        5.8|     10.2|
+-------------+-------+--------+-----------+---------+

+-----------+------+
|pickup_hour| trips|
+-----------+------+
|          0| 75156|
|          1| 50431|
|          2| 34923|
|          3| 22916|
|          4| 15252|
|          5| 17484|
|          6| 39392|
|          7| 80811|
|          8|113426|
|          9|125530|
|         10|135322|
|         11|146646|
|         12|159808|
|         13|165198|
|         14|177868|
|         15|183840|
|         16|184816|
|         17|200144|
|         18|206163|
|         19|178696|
|         20|155463|
|         21|155808|
|         22|138149|
|         23|104544|
+-----------+------+
```

Hình 5.5. Kết quả join hai lần với bảng khu vực, tổng hợp theo quận và theo giờ đón.

5.2.4 Hàm cửa sổ

Top-3 khu vực đón mỗi quận dùng `dense_rank()` trên cửa sổ `partitionBy("pu_borough")` (Bảng 5.5). Riêng Staten Island trả về hơn 3 dòng vì `dense_rank` để nhiều khu vực đồng hạng: ba khu vực cùng 4 chuyến ở hạng 2 và bốn khu vực cùng 2 chuyến ở hạng 3.

Bảng 5.5. Top-3 khu vực đón tại Manhattan và Queens.

Quận	Hạng 1	Hạng 2	Hạng 3
Manhattan	Midtown Center (140,065)	Upper East Side South (140,059)	Upper East Side North (133,891)
Queens	JFK Airport (138,296)	LaGuardia Airport (87,657)	East Elmhurst (11,861)

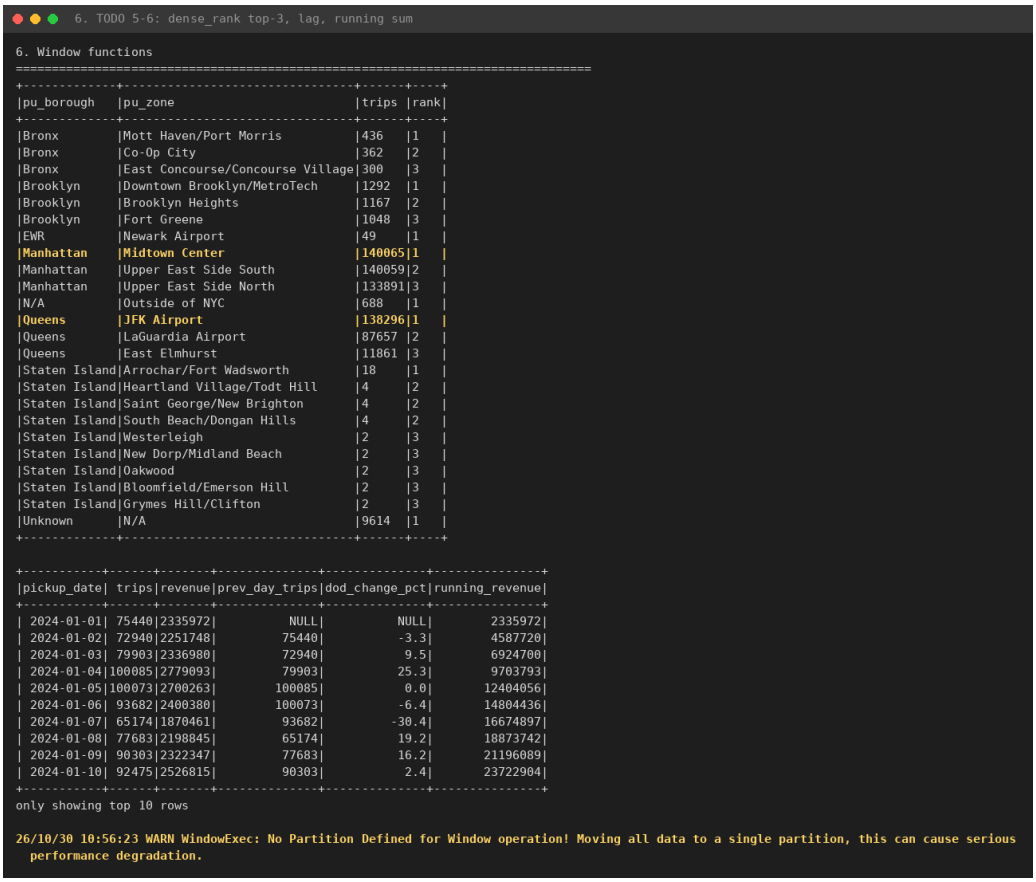
Chuỗi theo ngày dùng `lag` và tổng lũy kế trên `Window.orderBy("pickup_date")` (Bảng 5.6, Hình 5.6). Ngày 2024-01-07 (Chủ nhật) giảm 30.4 % so với hôm trước, còn 2024-01-04 tăng 25.3 %.

Bảng 5.6. Số chuyến và doanh thu theo ngày với `lag` và tổng lũy kế (10 ngày đầu).

Ngày	Số chuyến	Doanh thu	Ngày trước	Thay đổi (%)	Doanh thu lũy kế
2024-01-01	75,440	2,335,972	NULL	NULL	2,335,972
2024-01-02	72,940	2,251,748	75,440	-3.3	4,587,720
2024-01-03	79,903	2,336,980	72,940	9.5	6,924,700
2024-01-04	100,085	2,779,093	79,903	25.3	9,703,793
2024-01-05	100,073	2,700,263	100,085	0.0	12,404,056
2024-01-06	93,682	2,400,380	100,073	-6.4	14,804,436
2024-01-07	65,174	1,870,461	93,682	-30.4	16,674,897
2024-01-08	77,683	2,198,845	65,174	19.2	18,873,742
2024-01-09	90,303	2,322,347	77,683	16.2	21,196,089
2024-01-10	92,475	2,526,815	90,303	2.4	23,722,904

Log in ra cả chục dòng cảnh báo *“No Partition Defined for Window operation! Moving all data to a single partition”*. Lúc đầu chúng tôi tưởng đó là lỗi, nhưng ở đây cảnh báo vô hại vì cửa sổ chỉ áp lên `daily` đã tổng hợp còn 31 dòng. Áp cùng cửa sổ lên 2.87 triệu chuyến gốc thì toàn

bộ dữ liệu dồn vào một task, mất song song và có thể tràn bộ nhớ.



Hình 5.6. Hàm cửa sổ: top-3 khu vực đón mỗi quận và chuỗi theo ngày với `lag`, tổng lũy kế.

5.2.5 Spark SQL và kế hoạch vật lý

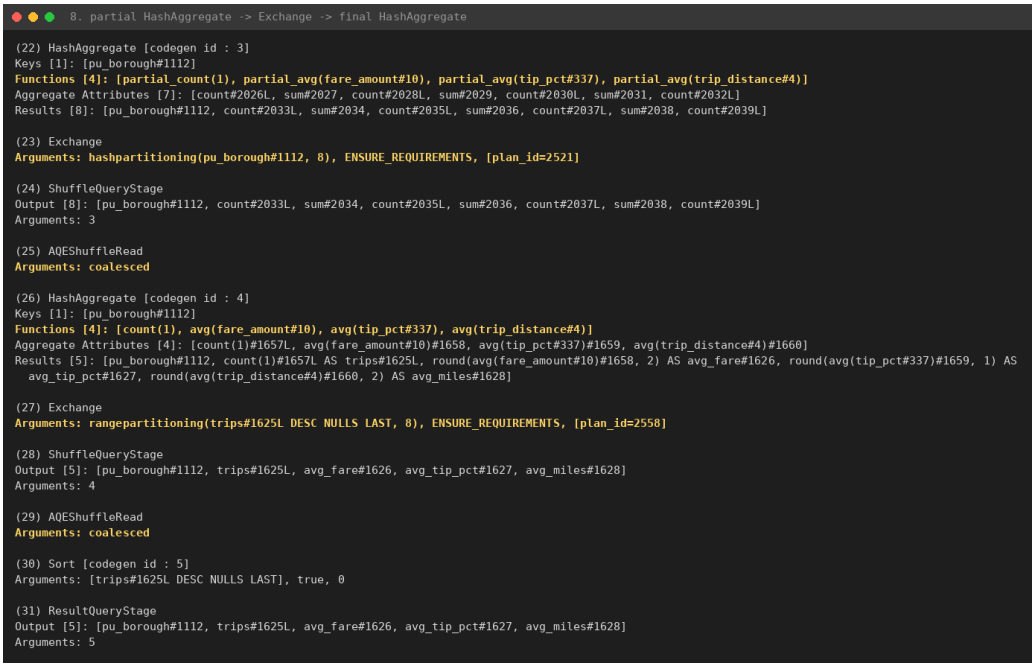
Truy vấn SQL trên temp view `trips` cho đúng bảng ở Bảng 5.4, script in ra `SQL result == DataFrame result: True` (Hình 5.7). Hai API đi qua cùng bộ tối ưu Catalyst [1] nên sinh cùng kế hoạch vật lý.



Hình 5.7. Kết quả Spark SQL trùng khớp DataFrame và cây kế hoạch vật lý của `by_borough`.

Kế hoạch `explain("formatted")` của `by_borough` (Hình 5.7, Hình 5.8) đọc từ dưới lên như sau. `Scan parquet` (3) mang `PushedFilters: [IsNotNull(tpep_pickup_datetime), IsNotNull(fare_amount), IsNotNull(total_amount), IsNotNull(trip_distance), GreaterThan(fare_amount,0.0), GreaterThanOrEqual(total_amount,0.0), GreaterThan(trip_distance,0.0), LessThanOrEqual(trip_distance,100.0)]`, nghĩa là các luật làm sạch đơn giản đã được đẩy xuống reader Parquet. Luật theo tháng và theo thời lượng cần `date_format`, `unix_timestamp` nên vẫn nằm ở nút `Filter` (5). `InMemoryTableScan` (1) đọc từ cache với `rowCount=2.87E+6`. Hai nhánh `Scan csv` đi qua `BroadcastExchange` (`Statistics(sizeInBytes=1026.1 KiB, rowCount=265)`) vào `BroadcastHashJoin LeftOuter BuildRight` (13) và (20). `HashAggregate` (22) với `partial_count(1), partial_avg(...)` phát ra các cặp sum/count, qua `Exchange hashpartitioning(pu_borough, 8)` (23) có `ShuffleQueryStage` thống kê `sizeInBytes=1968.0 B, rowCount=24`, qua `AQEShuffleRead coalesced` (25), tức Adaptive Query

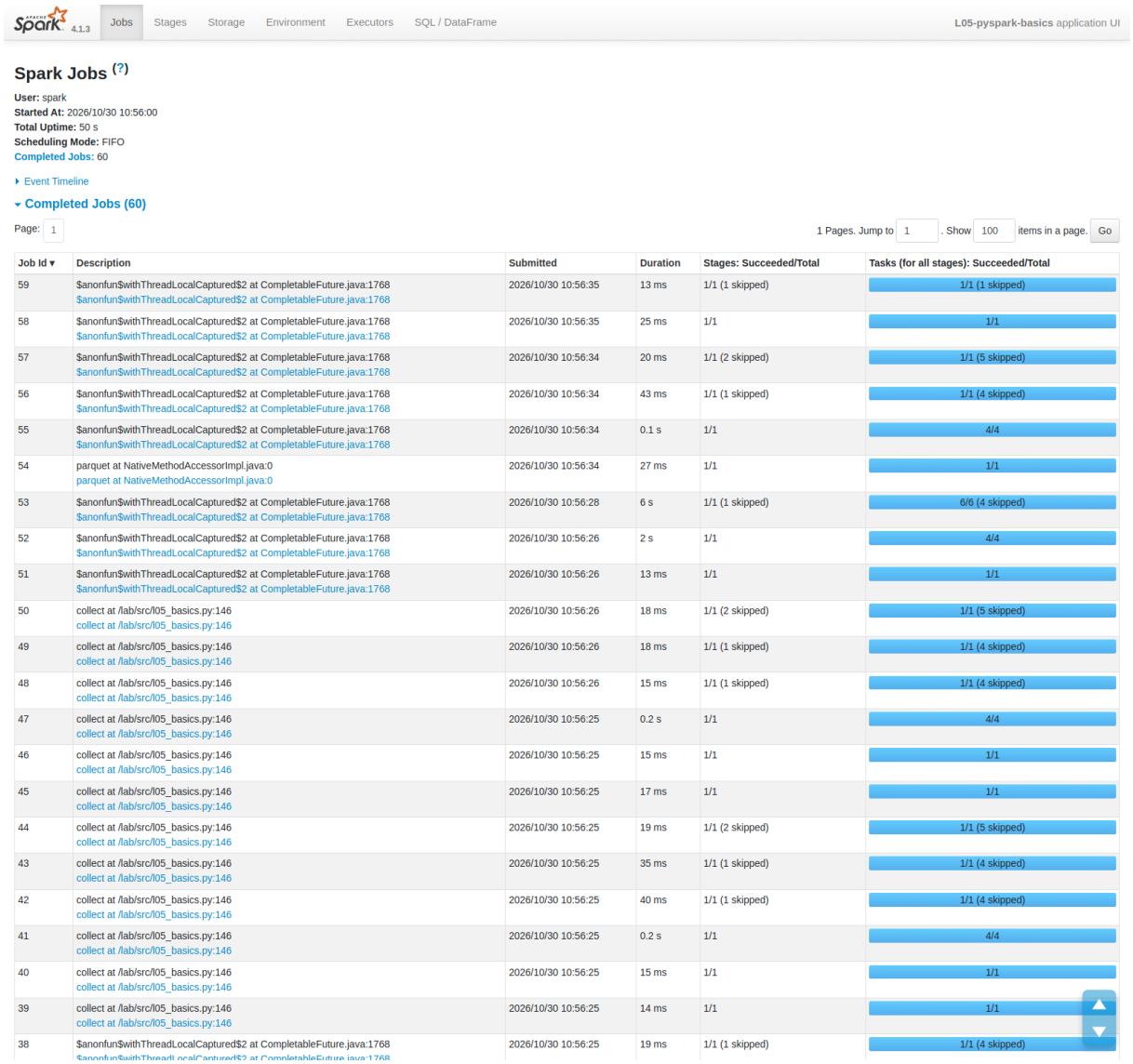
Execution gộp các partition shuffle nhỏ dựa trên thống kê lúc chạy [2], rồi tới `HashAggregate` (26) final. Cuối cùng là `Exchange rangepartitioning(trips DESC NULLS LAST, 8)` (27) với `rowCount=8`, `AQEShuffleRead` (29) và `Sort` (30) cho `orderBy`.



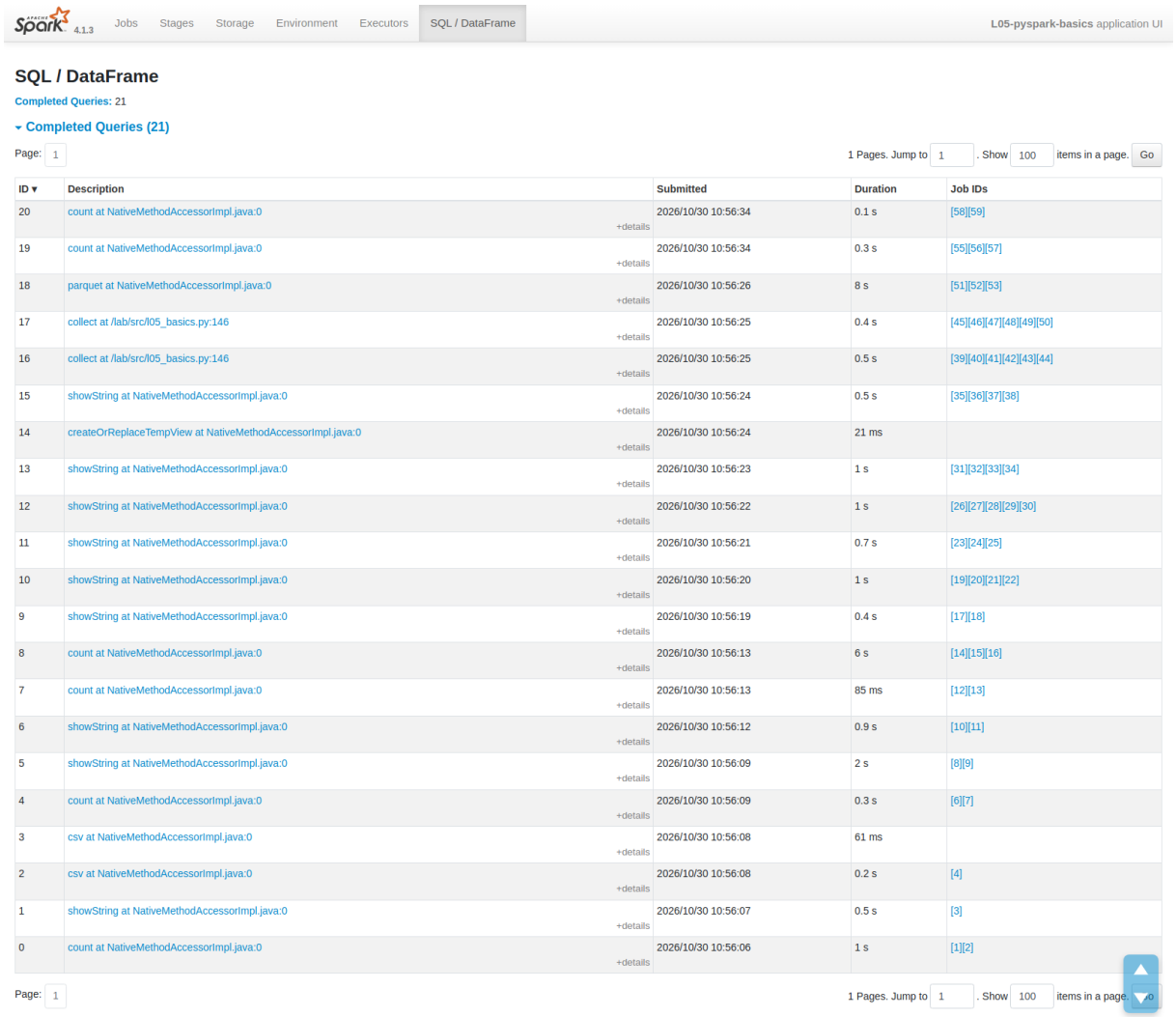
Hình 5.8. Phần kế hoạch thể hiện `HashAggregate` partial, `Exchange hashpartitioning` và `HashAggregate` final.

5.2.6 Spark UI

Trong tab Jobs (Hình 5.9), mỗi action (`count`, `showString`, `collect`, `parquet`) sinh một hoặc nhiều job. Tab SQL/DataFrame (Hình 5.10) liệt kê 21 truy vấn; Bảng 5.7 chọn ra các truy vấn đáng xem.



Hình 5.9. Tab Jobs: mỗi action sinh một hoặc nhiều job.



Hình 5.10. Tab SQL/DataFrame: 21 truy vấn đã hoàn thành cùng thời lượng và danh sách job.

Bảng 5.7. Một số truy vấn trong tab SQL (log `ui_sql_queries.txt`).

Query	Action	Thời lượng (ms)	Ý nghĩa
0	`count`	1,477	đếm số dòng đầu vào
4	`count`	268	`count()` ở bước lazy evaluation (228,254 chuyến dài)
8	`count`	6,225	`clean.count()` lần đầu, materialize cache
10	`showString`	1,473	`by_borough.show()`

Query	Action	Thời lượng (ms)	Ý nghĩa
16, 17	`collect`	511, 418	so sánh SQL và DataFrame
18	`parquet`	8,127	ghi Parquet phân vùng có `repartition`
19	`count`	336	đọc lại ngày 2024-01-15

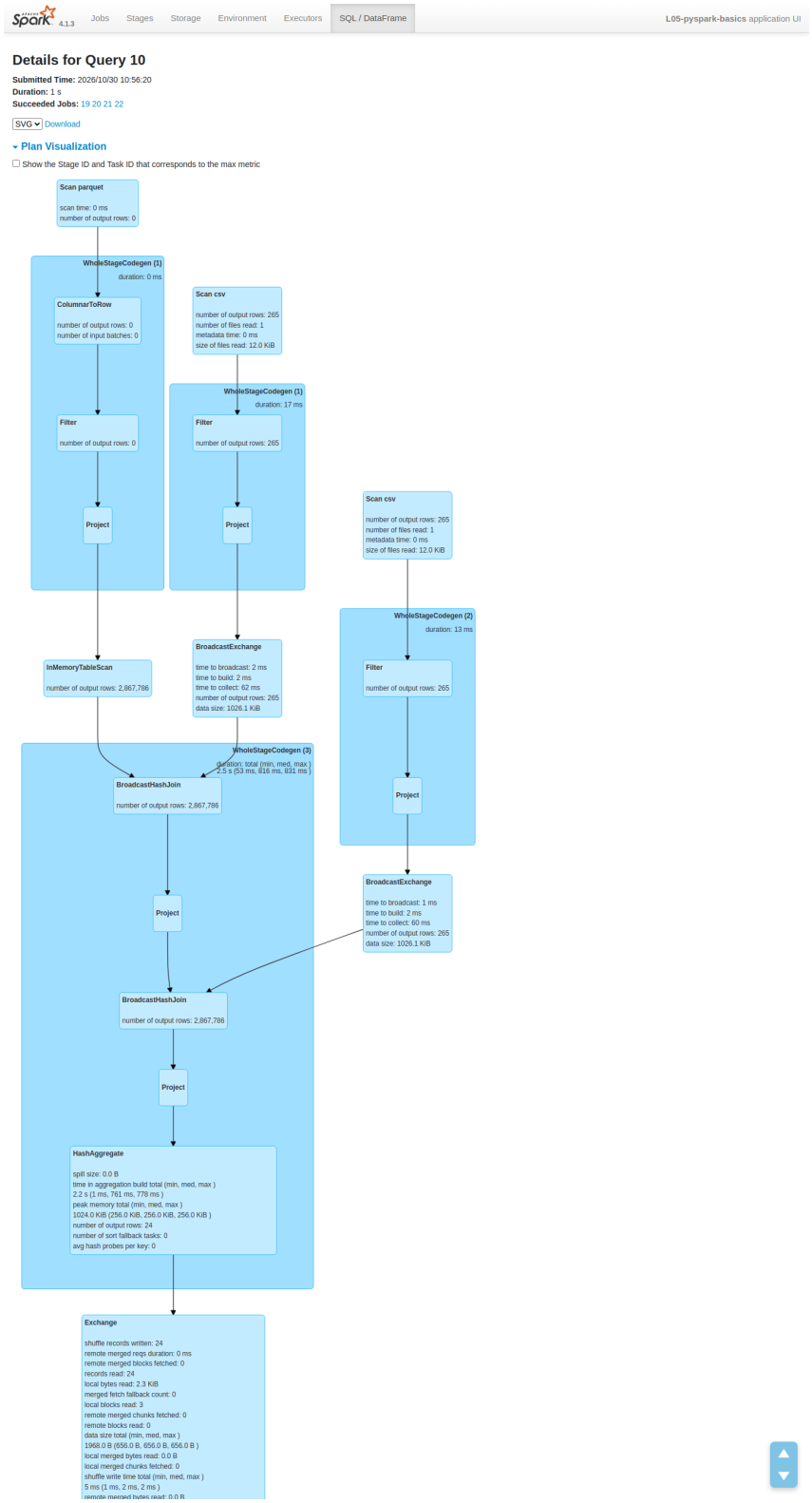
Truy vấn 10 (`by_borough.show()`) sinh bốn job 19–22 (Bảng 5.8, lấy từ `ui_by_borough_jobs.txt`).

Bảng 5.8. Các job và stage của truy vấn 10 (`by_borough.show()`).

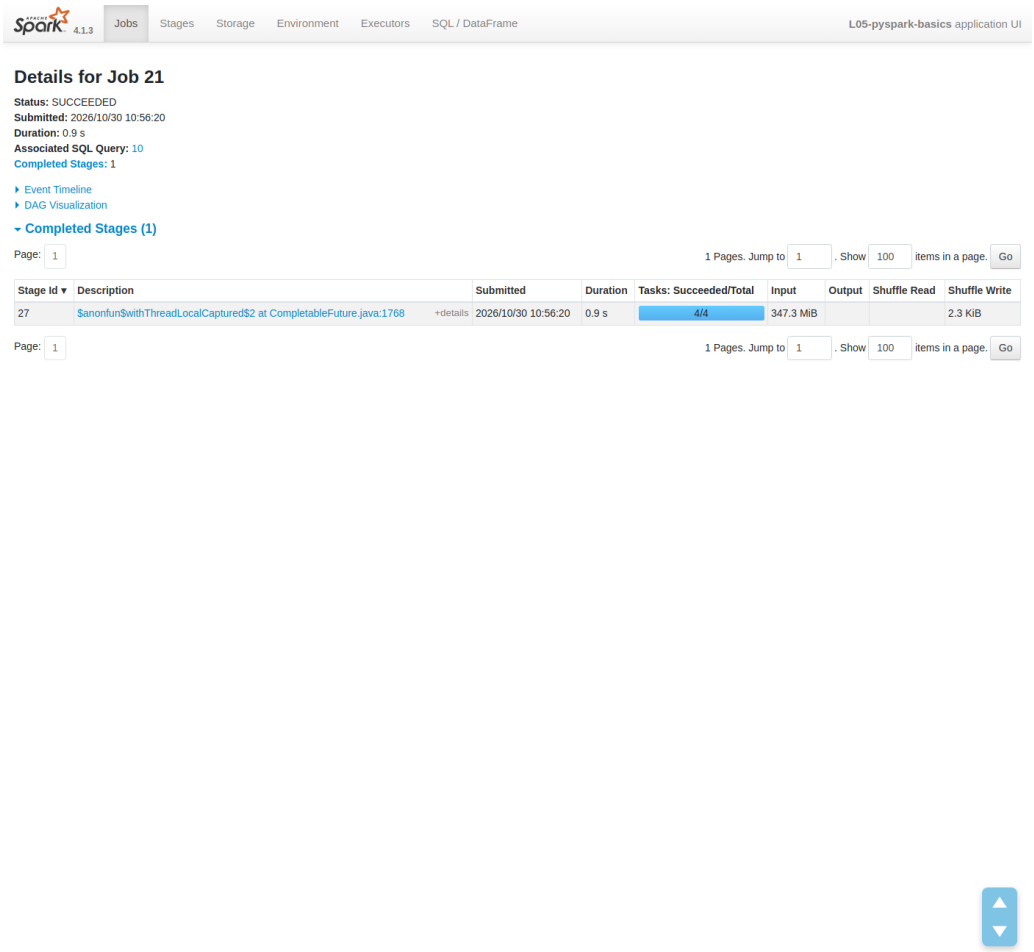
Job	Stage	Số task	Công việc
19	25	1	dựng broadcast bảng khu vực cho điểm đón (`BroadcastExchange`)
20	26	1	dựng broadcast bảng khu vực cho điểm trả
21	27	4	`InMemoryTableScan`, 2 × `BroadcastHashJoin`, `HashAggregate` partial, ghi shuffle
22	28 (skipped), 29	1	`AQEShuffleRead coalesced`, `HashAggregate` final, `TakeOrderedAndProject`

Trên đồ thị của truy vấn 10 (Hình 5.11), `InMemoryTableScan` xuất 2,867,786 dòng, mỗi nhánh `BroadcastExchange` 265 dòng (1026.1 KiB), hai `BroadcastHashJoin` giữ nguyên số dòng, còn `HashAggregate` partial chỉ xuất 24 dòng và `Exchange` ghi 24 bản ghi shuffle (1968.0 B). Job 21 (Hình 5.12) chạy 4/4 task trong 0.9 s, đọc 347.3 MiB từ cache và chỉ ghi 2.3 KiB shuffle.

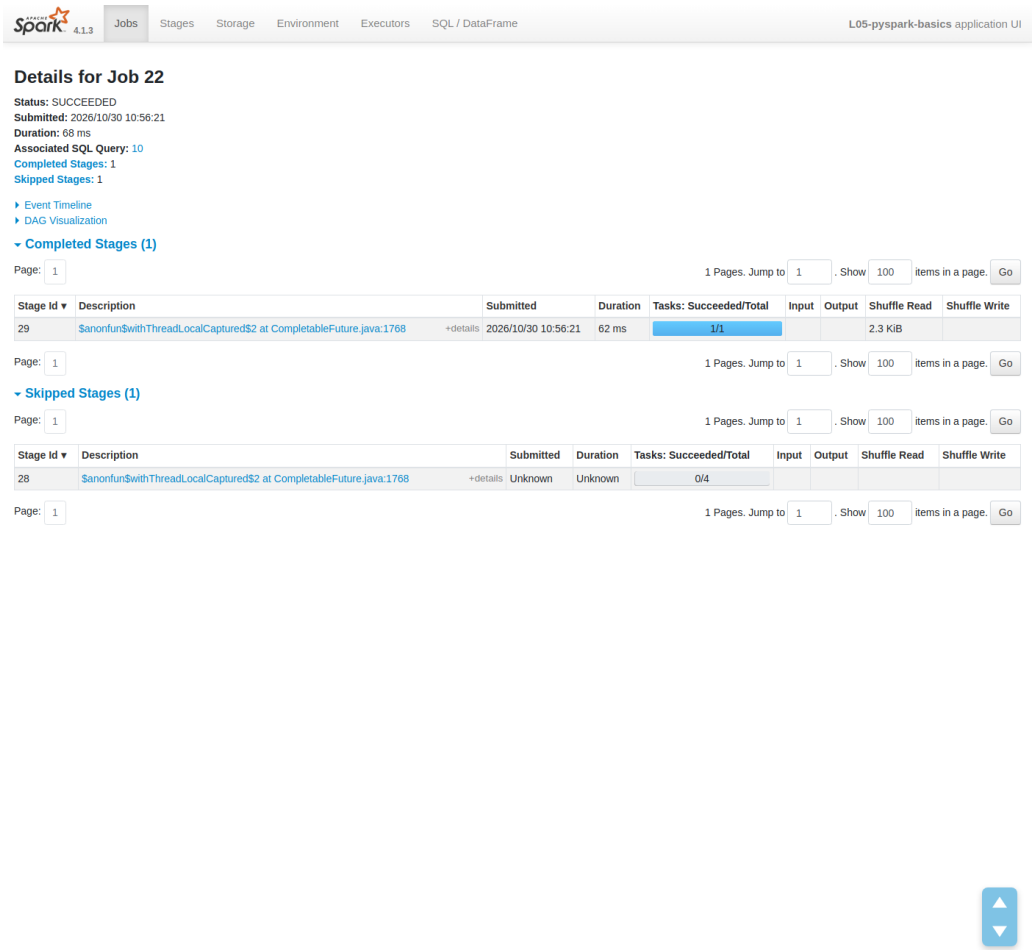
Ở job 22 (Hình 5.13), stage 28 bị skipped (0/4 task, vì output shuffle của stage 27 đã có sẵn), stage 29 chạy 1 task trong 62 ms với Shuffle Read 2.3 KiB.



Hình 5.11. Đồ thị kế hoạch truy vấn 10 trong tab SQL: 2,867,786 dòng vào, 24 dòng ra khỏi `HashAggregate` partial.

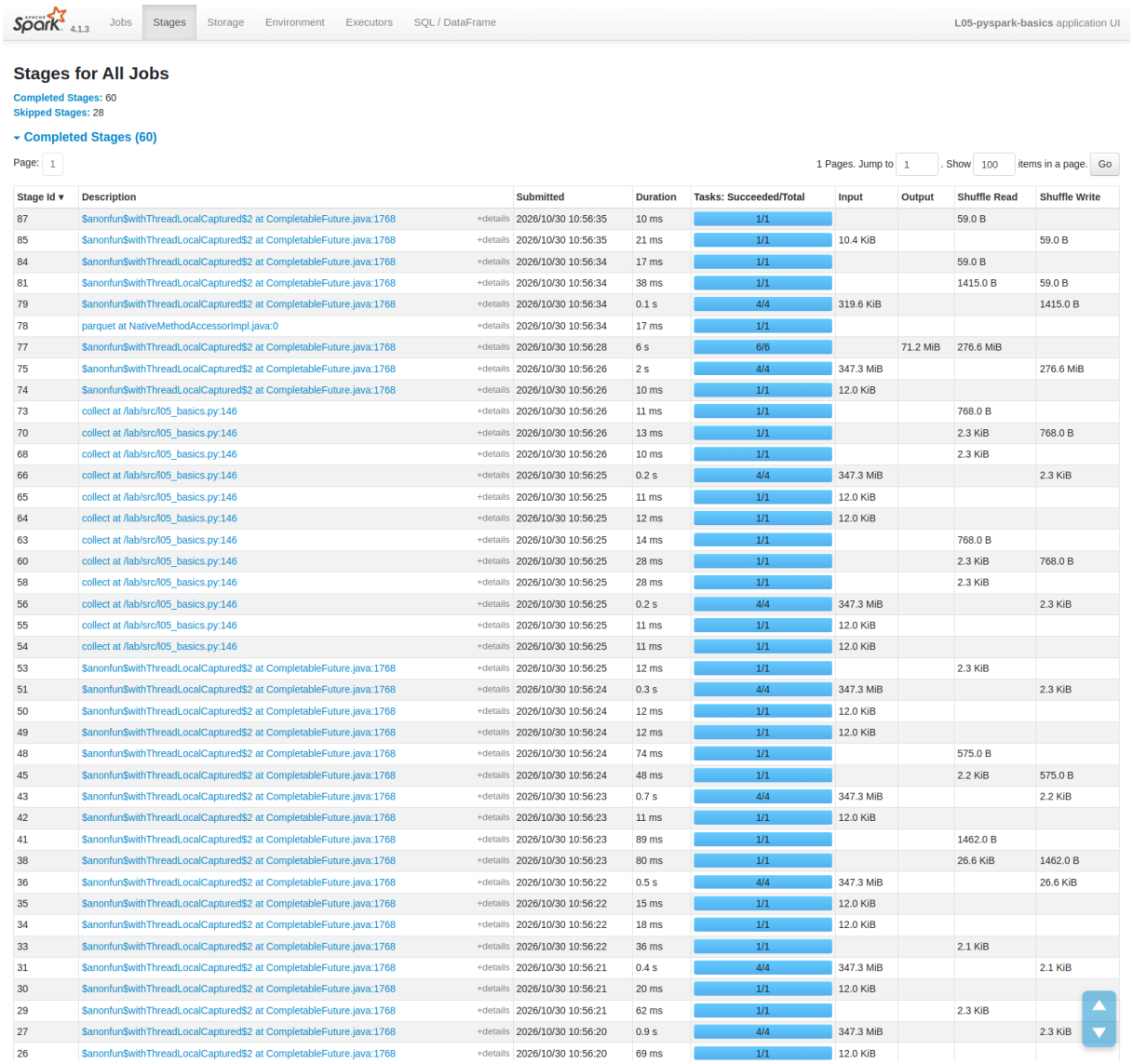


Hình 5.12. Job 21: một stage, 4/4 task, Input 347.3 MiB từ cache, Shuffle Write 2.3 KiB.



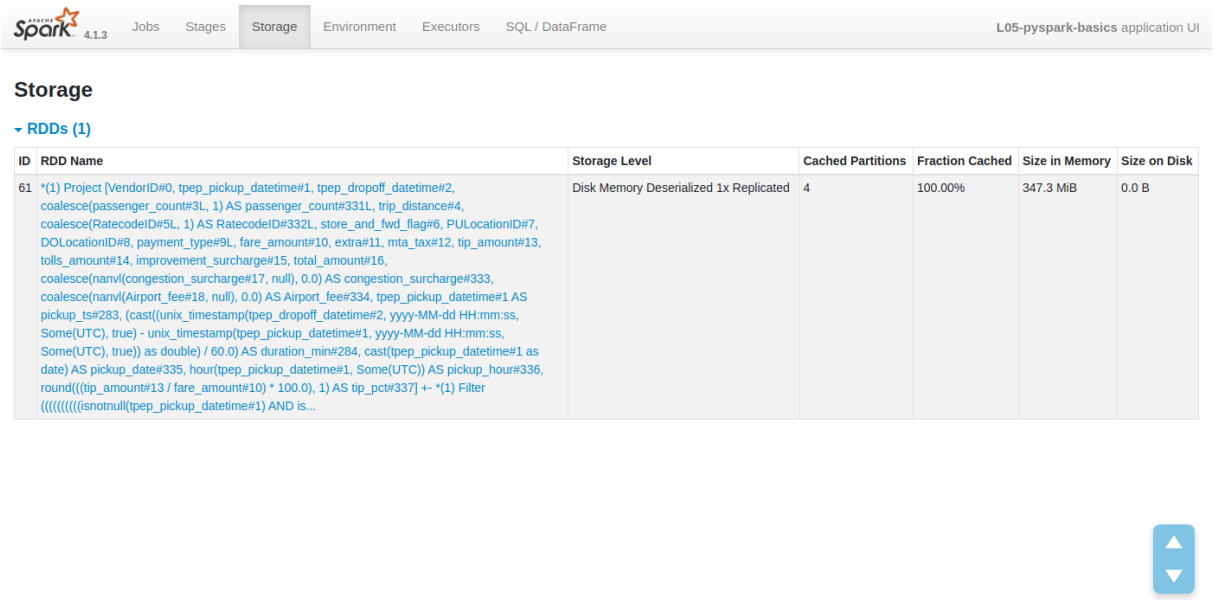
Hình 5.13. Job 22: stage 28 bị bỏ qua, stage 29 chạy 1 task với Shuffle Read 2.3 KiB.

Tab Stages (Hình 5.14) có 60 stage hoàn thành và 28 stage bị bỏ qua. Các stage 4 task đọc 347.3 MiB từ cache, các stage 1 task đọc 12.0 KiB từ tệp CSV khu vực. Tab Storage (Hình 5.15) cho thấy `clean` được cache ở mức *Disk Memory Deserialized 1x Replicated*, 4/4 partition (100 %), 347.3 MiB trong bộ nhớ, 0.0 B trên đĩa.



Stage Id ▾	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
87	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:35	10 ms	1/1			59.0 B	
85	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:35	21 ms	1/1	10.4 KiB			59.0 B
84	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:34	17 ms	1/1			59.0 B	
81	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:34	38 ms	1/1			1415.0 B	59.0 B
79	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:34	0.1 s	4/4	319.6 KiB			1415.0 B
78	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/30 10:56:34	17 ms	1/1				
77	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:28	6 s	6/6		71.2 MiB	276.6 MiB	
75	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:26	2 s	4/4	347.3 MiB			276.6 MiB
74	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:26	10 ms	1/1	12.0 KiB			
73	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:26	11 ms	1/1			768.0 B	
70	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:26	13 ms	1/1			2.3 KiB	768.0 B
68	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:26	10 ms	1/1			2.3 KiB	
66	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	0.2 s	4/4	347.3 MiB			2.3 KiB
65	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	11 ms	1/1	12.0 KiB			
64	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	12 ms	1/1	12.0 KiB			
63	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	14 ms	1/1			768.0 B	
60	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	28 ms	1/1			2.3 KiB	768.0 B
58	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	28 ms	1/1			2.3 KiB	
56	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	0.2 s	4/4	347.3 MiB			2.3 KiB
55	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	11 ms	1/1	12.0 KiB			
54	collect at /lab/src/l05_basics.py:146 +details	2026/10/30 10:56:25	11 ms	1/1	12.0 KiB			
53	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:25	12 ms	1/1			2.3 KiB	
51	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:24	0.3 s	4/4	347.3 MiB			2.3 KiB
50	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:24	12 ms	1/1	12.0 KiB			
49	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:24	12 ms	1/1	12.0 KiB			
48	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:24	74 ms	1/1			575.0 B	
45	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:24	48 ms	1/1			2.2 KiB	575.0 B
43	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:23	0.7 s	4/4	347.3 MiB			2.2 KiB
42	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:23	11 ms	1/1	12.0 KiB			
41	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:23	89 ms	1/1			1462.0 B	
38	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:23	80 ms	1/1			26.6 KiB	1462.0 B
36	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:22	0.5 s	4/4	347.3 MiB			26.6 KiB
35	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:22	15 ms	1/1	12.0 KiB			
34	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:22	18 ms	1/1	12.0 KiB			
33	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:22	36 ms	1/1			2.1 KiB	
31	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:21	0.4 s	4/4	347.3 MiB			2.1 KiB
30	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:21	20 ms	1/1	12.0 KiB			
29	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:21	62 ms	1/1			2.3 KiB	
27	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:20	0.9 s	4/4	347.3 MiB			2.3 KiB
26	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:56:20	69 ms	1/1	12.0 KiB			

Hình 5.14. Tab Stages: 60 stage hoàn thành, 28 stage bị bỏ qua, kèm Input và Shuffle Read/Write.



Hình 5.15. Tab Storage: `clean` được cache 4/4 partition, 100 %, 347.3 MiB trong bộ nhớ.

5.2.7 Ghi Parquet phân vùng và partition pruning

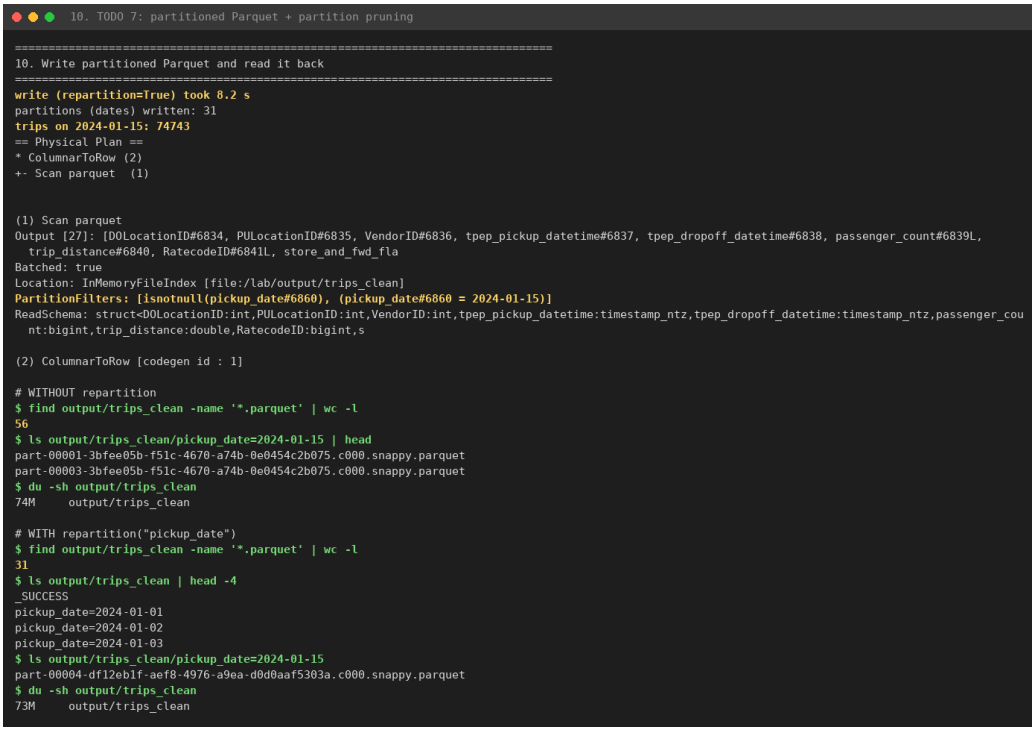
`enriched` được ghi bằng `partitionBy("pickup_date")`, một lần không và một lần có `repartition("pickup_date")` (Bảng 5.9, theo `02_files_no_repartition.txt`, `04_files_with_repartition.txt` và hai log chạy).

Bảng 5.9. So sánh ghi Parquet phân vùng theo ngày có và không có `repartition`.

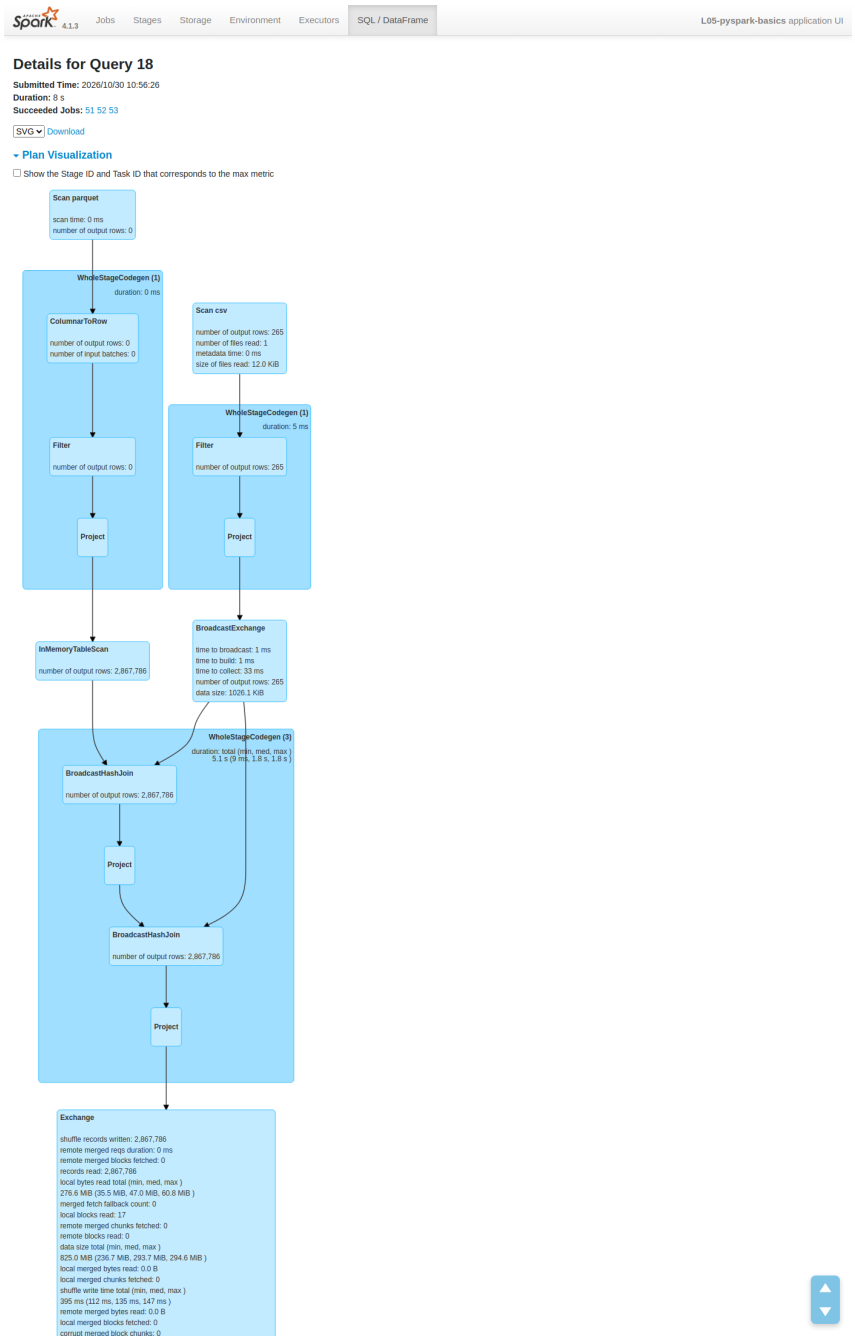
Chỉ số	Không `repartition`	`repartition("pickup_date")`
Số tệp `.parquet`	56	31 (1 tệp/ngày)
Số tệp trong `pickup_date=2024-01-15`	2 (`part-00001`, `part-00003`)	1 (`part-00004`)
Dung lượng thư mục output	74 MB	73 MB
Thời gian ghi	6.8 s	8.2 s
Tổng thời gian chạy script	35.70 s	không so sánh được (có `--hold 300`)

Cả hai lần đều ra 31 thư mục ngày. Đọc lại với `filter(pickup_date == '2024-01-15')` trả về 74,743 chuyến, và nút `Scan parquet` có `PartitionFilters: [isnotnull(pickup_date), (pickup_date = 2024-01-15)]` (Hình 5.16), tức chỉ một thư mục được mở. Ở đồ thị truy vấn 18 (Hình 5.17), `repartition` thêm một `Exchange` mà cả 2,867,786 dòng phải đi qua; trong tab Stages, stage 75 ghi 276.6 MiB shuffle, stage 77 (6 task sau khi AQE gộp partition)

đọc lại lượng đó và xuất 71.2 MiB Parquet (Hình 5.14). Mỗi cấu hình chỉ chạy một lần, nên chênh lệch thời gian ghi mới là một mẫu, chưa đủ để kết luận chắc về chi phí của `repartition`.



Hình 5.16. Ghi Parquet phân vùng, đọc lại ngày 2024-01-15 (74,743 chuyến) và `PartitionFilters` trong kế hoạch.



Hình 5.17. Đồ thị truy vấn 18 (ghi Parquet): `Exchange` do `repartition` shuffle toàn bộ 2,867,786 dòng.

5.3 Trả lời câu hỏi checkpoint

5.3.1 Vì sao tệp 48 MB lại có 4 partition đầu vào?

Spark cắt tệp thành các split có kích thước tối đa maxSplitBytes $=$ min(maxPartitionBytes, max(openCostIn
với `spark.sql.files.maxPartitionBytes` mặc định 128 MB, `openCostInBytes` mặc định 4 MB, còn `bytesPerCore` là tổng dung lượng (cộng chi phí mở tệp) chia cho `defaultParallelism`. Ở `local[4]`, `defaultParallelism` bằng 4 nên split bị hạ xuống khoảng 48 MB / 4 thay vì 128 MB, ra 4 split và 4 partition (Hình 5.2). Có điều Parquet chỉ cắt

được ở ranh giới row group: task nào nhận điểm bắt đầu của row group thì đọc cả row group đó. Chúng tôi không tự kiểm tra số row group, nhưng theo đáp án tham khảo của bài lab tệp này có 3 row group, nên một partition có thể rỗng. Quan sát ở Bảng 5.9 khớp với điều này: các tệp của ngày 2024-01-15 mang số `part-00001` và `part-00003`, không có task 2.

5.3.2 Dòng nào là transformation, dòng nào là action, và sao dựng plan nhanh vậy?

`filter`, `withColumn`, `select` là transformation: chúng chỉ thêm nút vào kế hoạch logic, không chạm dữ liệu, nên tab Jobs không có job mới (Mục 5.2.1). `count()` là action: Catalyst phân tích và tối ưu kế hoạch, DAG scheduler tạo stage và task, rồi mới đọc tệp; thời gian đo được khớp với truy vấn 4 trong Bảng 5.7. `show`, `collect`, `take`, `write` cũng là action. Dựng plan nhanh vì đó chỉ là thao tác trên cây biểu thức trong bộ nhớ driver, không có I/O và không cần liên lạc với executor.

5.3.3 `by_borough.show()` sinh bao nhiêu job, stage, và stage bị cắt ở đâu?

Truy vấn 10 sinh **4 job (19–22) với 5 stage (25–29)**, trong đó stage 28 bị bỏ qua nên chỉ 4 stage thực sự chạy (Bảng 5.8). Job 22 đọc shuffle đã được AQE gộp từ 8 xuống 1 partition, tổng hợp final và lấy kết quả có thứ tự cho `show()`.

Ranh giới stage nằm ở các `Exchange`: giữa stage 27 và stage 29 là `Exchange hashpartitioning(pu_borough, 8)`. Broadcast phải chạy thành job riêng vì `BroadcastExchange` cần thu dữ liệu về driver rồi phát lại trước khi phía lớn join được. Với AQE, mỗi query stage được nộp như một job riêng để Spark tối ưu lại khi đã có thống kê thật, vì thế stage 28 (bản sao của stage 27 trong DAG của job 22) hiện là skipped (Hình 5.13). Với `show()`, Spark dùng `TakeOrderedAndProject` thay cho sort toàn cục. Nếu tính trọn `by_borough` (ví dụ `collect`), kế hoạch có thêm `Exchange rangepartitioning(trips DESC, 8)` và `Sort`, tức thêm một stage, như trong `explain` (Hình 5.7). Trước khi gọi `show()`, các phép join và `groupBy` chưa chạy gì, chúng chỉ dựng DAG.

5.3.4 Vì sao năm cột có cùng 140,162 NULL, và nên xử lý thế nào?

Năm cột trùng số NULL (Hình 5.3) vì đó là cùng các chuyến: các bản ghi này thiếu cả nhóm trường metadata của chuyến (thường từ một nhà cung cấp, hoặc chuyến đón vẫy tay ngoài đường), chứ không phải năm lỗi độc lập. Nếu độc lập thì gần như không thể trùng số tuyệt đối ở cả năm cột. Có ba cách xử lý: (i) giữ dòng và điền mặc định như script đã làm (Mục 5.2.2); (ii) giữ NULL và chỉ bỏ các dòng này khỏi những chỉ số cần đến các cột đó; (iii) tách sang vùng quarantine để kiểm tra sau. Chúng tôi chọn (i) cho đơn giản, nhưng điền `passenger_count=1` có thể làm lệch chỉ số số khách trung bình nếu sau này cần tới.

5.3.5 `HashAggregate` partial trước `Exchange` làm gì, ứng với khái niệm nào ở L02?

`HashAggregate` (22) trước `Exchange` là pha partial: mỗi task gộp trước các dòng của mình theo `pu_borough`. Hàm `avg` bị tách thành cặp `(sum, count)` (thấy trong `Aggregate Attributes` gồm `count#...`, `sum#...`), vì trung bình của các trung bình không bằng trung bình chung, còn tổng và số đếm thì cộng dồn được. Mỗi task chỉ phát một dòng cho mỗi quận nó gặp, nên 4 task cộng lại chỉ có **24 dòng** qua shuffle thay vì 2,867,786 dòng (Hình 5.11). Sau `Exchange` (ranh giới stage), `HashAggregate` (26) final hợp nhất các kết quả partial (Hình 5.8).

Đây là combiner của MapReduce ở L02 [4]: gộp phía map để giảm dữ liệu trung gian trước khi gửi sang reducer. Việc tách `avg` thành `(sum, count)` giống bài mở rộng L02 (combiner tính giá cước trung bình); lúc đó chúng tôi phải tự nghĩ ra cặp này, còn ở Spark Catalyst tự làm.

5.3.6 Vì sao bảng khu vực được broadcast, và khi nào Spark chuyển sang sort-merge join?

Bảng khu vực chỉ có 265 dòng (tệp 12 KB, khoảng 1 MB trong bộ nhớ theo `Statistics(sizeInBytes= 1026.1 KiB)`), nhỏ hơn nhiều so với ngưỡng `spark.sql.autoBroadcastJoinThreshold` mặc định 10 MB [2]. Spark gửi một bản sao tới mọi task nên 2,867,786 chuyến không phải shuffle; trên Hình 5.11, `BroadcastHashJoin` nằm chung một khối WholeStageCodegen với phép quét cache. Khi cả hai phía đều vượt ngưỡng, khi ngưỡng đặt bằng -1, hoặc khi thống kê ước lượng vượt ngưỡng, Spark shuffle cả hai phía theo khóa join và dùng SortMergeJoin.

5.3.7 `cache()` thay đổi gì trong plan và UI, và khi nào không nên cache?

Sau `cache()`, kế hoạch đọc từ `InMemoryRelation`/`InMemoryTableScan` thay vì quét và lọc lại Parquet. Nhánh `Scan parquet` vẫn còn trên đồ thị nhưng ra 0 dòng (Hình 5.11). Tab Storage hiện RDD được cache (Hình 5.15). Theo Bảng 5.7, `clean.count()` lần đầu (truy vấn 8) chậm hẳn vì phải materialize cache, còn các truy vấn dùng cache sau đó (9–17) chỉ mất 396 ms đến 1,473 ms.

Không nên cache khi dữ liệu chỉ dùng một lần, hoặc khi nó không vừa bộ nhớ và gây spill, eviction, áp lực GC. Ở đây cache chiếm 347 MiB so với khoảng 48 MB Parquet nén, gấp khoảng 7 lần, nên với dữ liệu cả năm thì driver 2 GB khó mà chứa nổi. Cache cũng có thể thừa khi nguồn đọc vốn đã rẻ: Parquet có column pruning đôi khi nhanh hơn một cache dạng hàng rộng. Cuối cùng, quên `unpersist()` thì bộ nhớ bị giữ vô ích.

5.3.8 Vì sao cần `repartition("pickup_date")` trước `partitionBy("pickup_date")`?

`partitionBy` ghi một tệp cho mỗi cặp (task, giá trị phân vùng) mà task đó chứa. Không `repartition` thì mỗi task có thể chứa dòng của nhiều ngày, số tệp tối đa là $N \times 31$ với N task, ở đây là $4 \times 31 = 124$. Số tệp đo được ở Bảng 5.9 ít hơn giới hạn khá nhiều, vì tệp nguồn gần như đã sắp theo thời gian: mỗi task chỉ chứa một dải ngày liền nhau và chỉ vài ngày bị tách giữa hai task (như 2024-01-15). Với dữ liệu không có thứ tự và hàng trăm task trên cluster thật, con số sẽ lên tới hàng nghìn tệp nhỏ, gây áp lực cho NameNode và làm chậm việc đọc.

`repartition("pickup_date")` shuffle trước để mọi dòng của một ngày về cùng một task, nên mỗi ngày đúng một tệp. Cái giá là một lần shuffle toàn bộ dữ liệu (Mục 5.2.7, Hình 5.17) và thời gian ghi tăng (Bảng 5.9). Với dữ liệu lệch (skew), một ngày quá lớn sẽ dồn vào một task và sinh một tệp khổng lồ; khi đó có thể dùng `repartition(n, "pickup_date", salt)` hoặc `maxRecordsPerFile`.

5.4 Kết luận

Script chạy đủ các bước của đề và cho đúng các số liệu tham khảo: 2,964,624 dòng trong 4 partition, 228,254 chuyến dài, 2,867,786 dòng giữ lại sau làm sạch, Manhattan 2,571,810 chuyến, giờ cao điểm 18:00, 31 phân vùng ngày và 74,743 chuyến ngày 2024-01-15. Trên log và UI, chúng tôi quan sát được transformation chỉ dựng kế hoạch còn action mới chạy job, stage bị cắt ở `Exchange`, tổng hợp hai pha chỉ đưa 24 dòng qua shuffle, bảng nhỏ được broadcast tự động, và DataFrame với SQL cho cùng kết quả. Ghi có `repartition` cho đúng một tệp mỗi ngày, đổi lại một lần shuffle toàn bộ dữ liệu.

Tài liệu tham khảo

[1] M. Armbrust *et al.*, “Spark SQL: Relational Data Processing in Spark,” in *Proc. ACM SIGMOD*, 2015.

[2] Apache Software Foundation, “Performance Tuning,” Apache Spark Documentation. https://spark.apache.org/docs/latest/sql-performance-tuning.html

[3] Apache Software Foundation, “ANSI Compliance,” Apache Spark Documentation. https://spark.apache.org/docs/latest/sql-ref-ansi-compliance.html

[4] J. Dean and S. Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters,” in *Proc. 6th USENIX OSDI*, 2004.

[5] NYC Taxi and Limousine Commission, “TLC Trip Record Data.” https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

Phụ lục: Trích đoạn mã nguồn

Đếm NULL của mọi cột trong một lần quét và tính tỷ lệ tip an toàn với ANSI mode (phần làm sạch trong `l05_basics.py`):

```
null_counts = trips.select([F.sum(F.col(c).isNull().cast("int")).alias(c)
                            for c in trips.columns])

# ANSI mode: try_divide trả NULL khi fare_amount = 0 thay vì làm hỏng job
.withColumn("tip_pct",
            F.round(F.try_divide(F.col("tip_amount"), F.col("fare_amount"))
 ↪  * 100, 1))
```

Join hai lần với bảng khu vực, đổi tên cột trước mỗi lần để tránh `AMBIGUOUS_REFERENCE`:

```
pu = zones.select(F.col("LocationID").alias("PULocationID"),
                  F.col("Borough").alias("pu_borough"),
 ↪  F.col("Zone").alias("pu_zone"))
do = zones.select(F.col("LocationID").alias("DOLocationID"),
                  F.col("Borough").alias("do_borough"),
 ↪  F.col("Zone").alias("do_zone"))
enriched = clean.join(pu, "PULocationID", "left").join(do, "DOLocationID",
 ↪  "left")
```

Ghi Parquet phân vùng; cờ `--no-repartition` cho phép chạy cả hai cấu hình mà không sửa mã:

```
out = enriched.drop("pickup_ts")
if not args.no_repartition:
    out = out.repartition("pickup_date")
out.write.mode("overwrite").partitionBy("pickup_date").parquet(OUTPUT)
```

Chương 6

Tối ưu hiệu năng Spark: shuffle, chiến lược join, data skew, AQE, phân vùng và cache

6.1 Mục tiêu và môi trường

Lab L06 đo tác dụng của các tham số tinh chỉnh Spark qua năm thí nghiệm: E1 số partition shuffle và cơ chế gộp partition của *Adaptive Query Execution* (AQE); E2 broadcast hash join so với sort-merge join; E3 data skew trong join, xử lý bằng salting và AQE skew join; E4 `repartition` so với `coalesce` trước khi ghi; E5 khi nào `cache`/`persist` có lợi. Mọi thứ chạy trong một container Docker ở chế độ local 4 core (Bảng 6.1).

Bảng 6.1. Môi trường thực nghiệm.

Thành phần	Giá trị
Máy chủ	Laptop 12 luồng CPU, 32 GB RAM
Ảo hóa	Docker Desktop trên WSL2, cấp 31 GB
Spark	4.1.3 (image `spark:4.1.3-python3`), PySpark
Chế độ chạy	`local[4]` (4 core, `defaultParallelism` = 4), driver 2 GB
Thu thập số liệu	`time.time()` quanh mỗi action; Spark REST API (`/sql`, `/stages/.../taskSummary`)
Ngày thực hiện	2026-10-30

Dữ liệu là tập chuyến taxi New York tháng 01/2024 và hai bảng tổng hợp cho E3 (Bảng 6.2).

Bảng 6.2. Dữ liệu thực nghiệm.

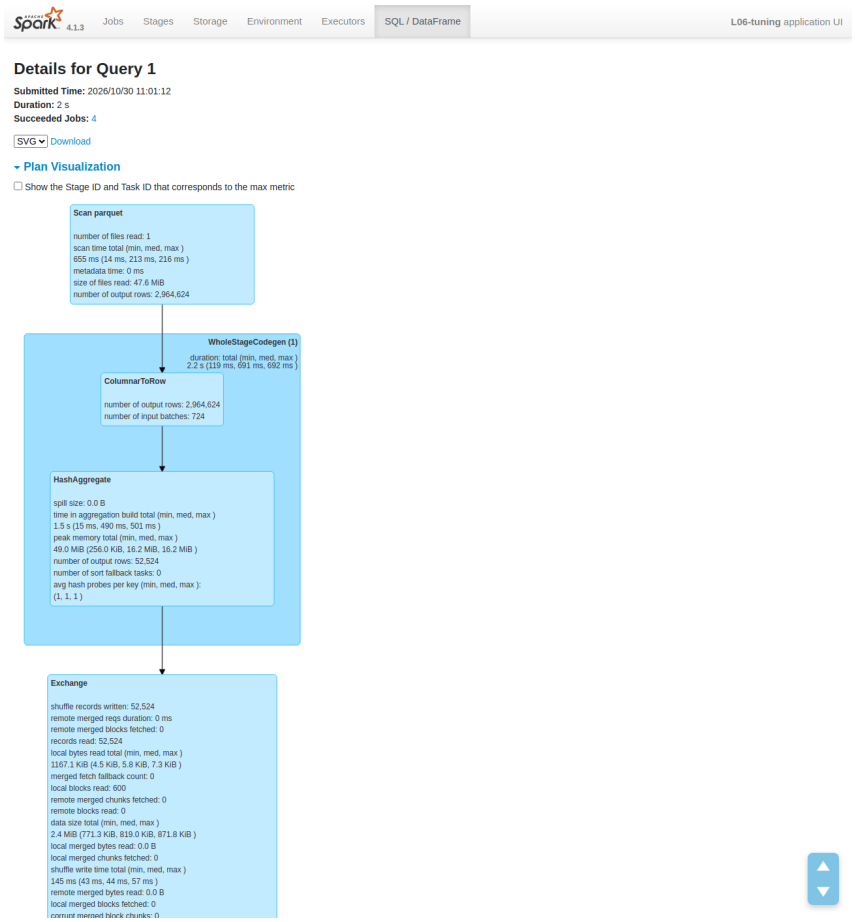
Tập dữ liệu	Nguồn	Kích thước	Dùng trong
Trips	`yellow_tripdata_2024-01.parquet`	2,964,624 dòng, 47.6 MiB, 3 row group	E1, E2, E4, E5
Zones	`taxi_zone_lookup.csv`	265 dòng	E2
Fact tổng hợp	`spark.range(10,000,000)`, `HOT_SHARE = 0.6` (key = 0), còn lại rải đều trên 200,000 khóa	10,000,000 dòng (5,998,366 hot, 4,001,634 thường)	E3
Dim tổng hợp	`spark.range(200,000)`, cột `segment` = `seg_{key % 50}`	200,000 dòng	E3

Script `l06_experiments.py` chạy một `count()` khởi động không tính giờ, rồi với mỗi biến thể thì đặt lại cấu hình (`reset_conf`), đổi tham số và ép thực thi bằng `df.write.format("noop")` (tính đủ nhưng không ghi file); E4 ghi Parquet thật để đếm file, E5 dùng `collect()`. Sau mỗi action, hàm `last_sql_summary()` gọi REST API [2] lấy các nút của kế hoạch cuối (sau AQE), tổng số task, và median/max thời gian task ở stage lệch nhất (stage có hiệu max trừ median lớn nhất). Chúng tôi chạy cả script hai lần (34 dòng trong `results.csv`) và dùng lần thứ hai; cờ `--hold` giữ Spark UI để chụp kế hoạch và Summary Metrics.

6.2 Thực hiện và kết quả

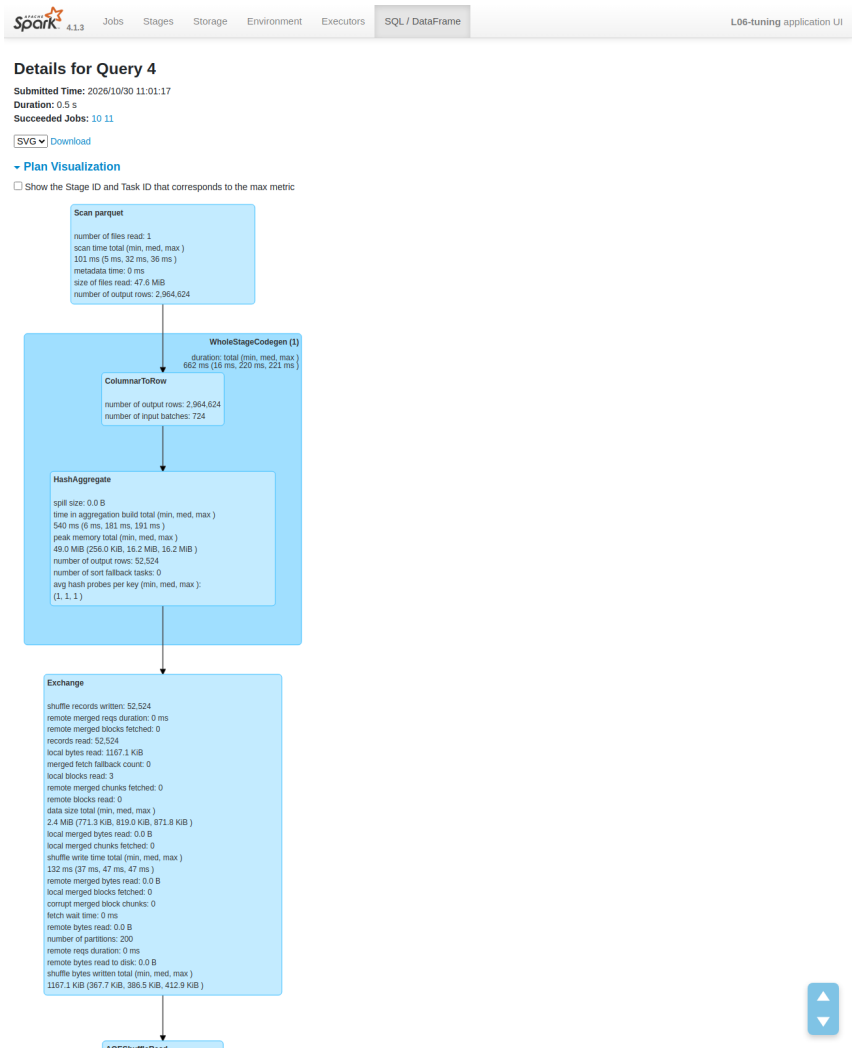
6.2.1 E1: số partition shuffle và AQE coalescing

Truy vấn `groupBy(PULocationID, DOLocationID).agg(count, avg)` đọc 47.6 MiB nhưng chỉ ghi khoảng 1167.1 KiB dữ liệu shuffle. Script chạy nó với AQE tắt và `shuffle.partitions` $\in \{200, 8, 1\}$; phần chúng tôi tự viết là biến thể bật AQE, giữ 200 partition. Với 200 partition và AQE tắt, truy vấn sinh 204 task (4 map, 200 reduce), mỗi task reduce chạy khoảng 6 ms, chậm nhất trong E1; kế hoạch chỉ có một `Exchange` 200 partition (Hình 6.1).



Hình 6.1. Kế hoạch E1 khi AQE tắt: `Exchange` với 200 partition shuffle.

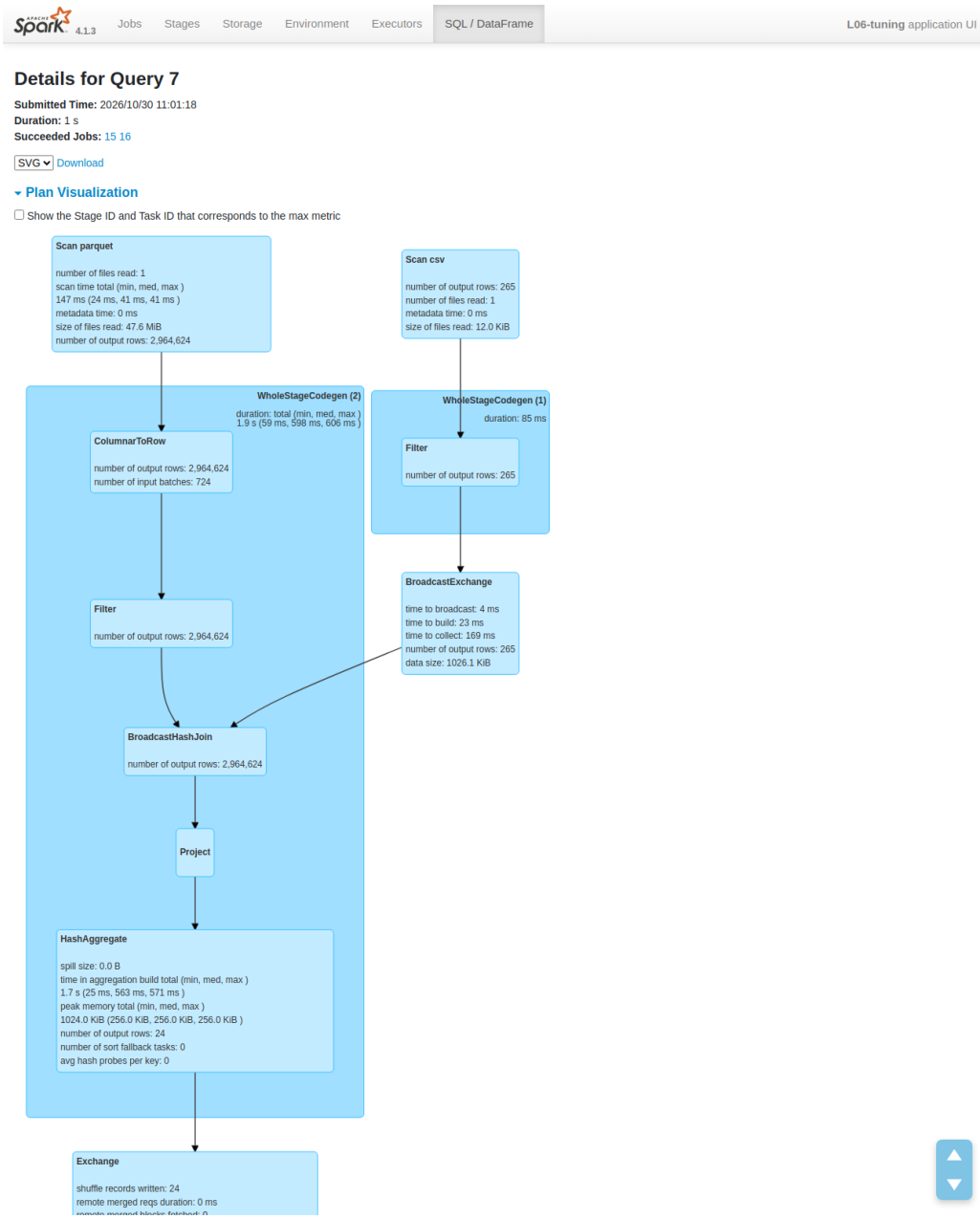
AQE tối ưu lại kế hoạch trong lúc chạy, dựa trên thống kê thật của stage shuffle đã xong thay vì ước lượng [1]. Bật AQE mà vẫn để 200 partition thì sau `Exchange` xuất hiện `AQEShuffleRead` (Hình 6.2), các partition nhỏ được gộp lại, tổng chỉ còn 5 task, thời gian xấp xỉ bản tinh chỉnh tay 8 partition (Bảng 6.4). Bản 1 partition cũng nhanh, nhưng chỉ vì kết quả quá nhỏ (khoảng 20 nghìn nhóm), xem Mục 6.3.1.



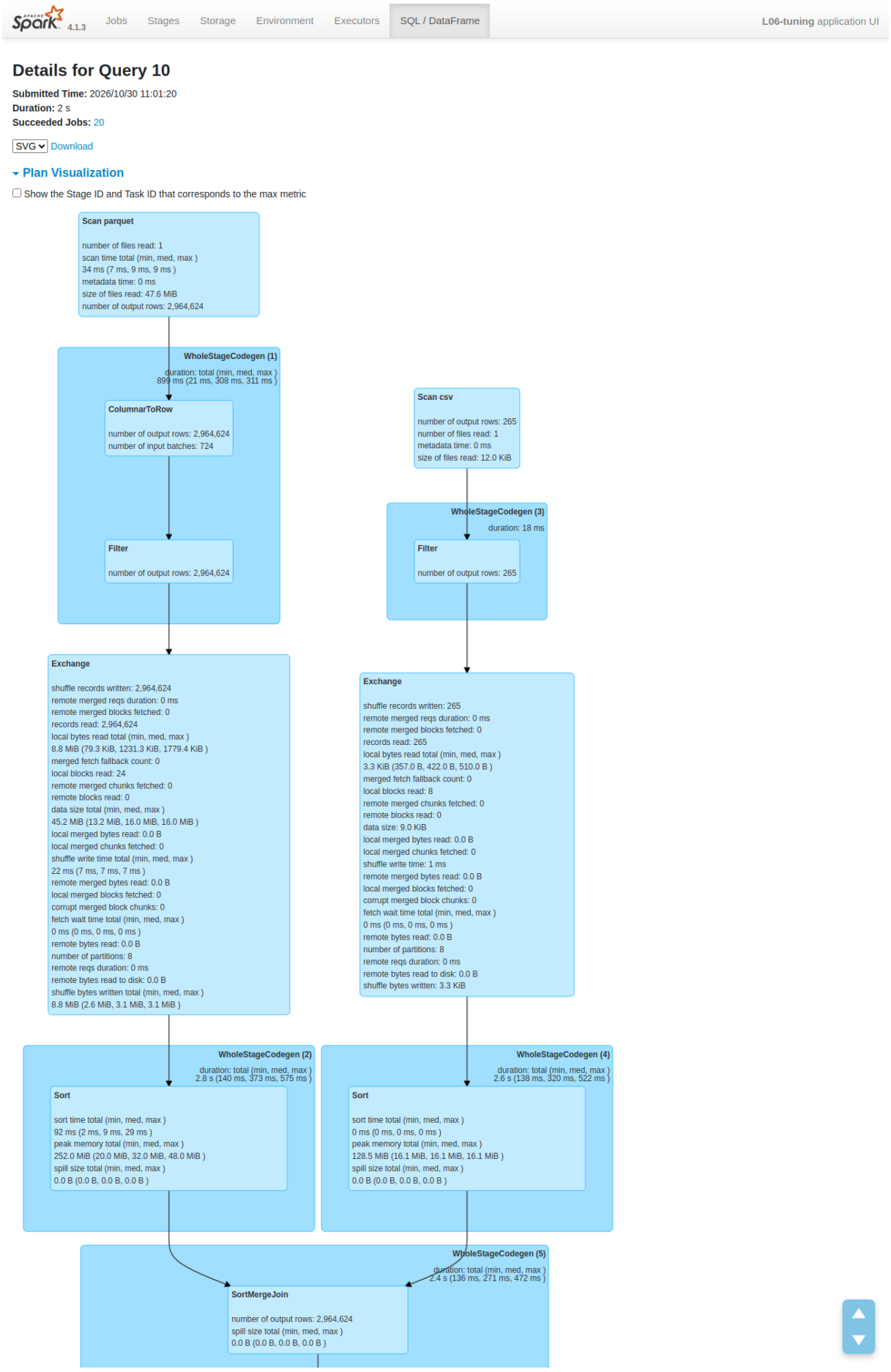
Hình 6.2. Kế hoạch E1 khi AQE bật: `Exchange` 200 partition theo sau là `AQEShuffleRead` ở chế độ coalesced; truy vấn đọc 2,964,624 dòng, 47.6 MiB.

6.2.2 E2: broadcast hash join và sort-merge join

Phép join trips ⋈ zones được chạy với `autoBroadcastJoinThreshold` 10 MB, với ngưỡng -1, và (phần tự viết) với ngưỡng -1 kèm gợi ý `F.broadcast(zones_df())`. Khi ngưỡng là -1, Spark phải dùng `SortMergeJoin`: khoảng 3 triệu chuyến bị shuffle cùng bảng zone rồi sort ở cả hai phía (Hình 6.4). Khi broadcast, chỉ 265 dòng được gửi tới mọi executor và phía trips không có `Exchange` trước join (Hình 6.3).



Hình 6.3. Kế hoạch E2 với broadcast: `BroadcastExchange` và `BroadcastHashJoin`, phía trips không có `Exchange` trước join.



Hình 6.4. Kế hoạch E2 với ngưỡng -1: `Exchange` và `Sort` ở cả hai phía trước `SortMergeJoin`.

Đặt ngưỡng -1 chỉ tắt việc chọn broadcast tự động của planner [3]; gợi ý `broadcast()` vẫn có hiệu lực, cho kế hoạch `BroadcastHashJoin` và là biến thể nhanh nhất của E2 (0.40 s, 6 task). Hơi bất ngờ là broadcast tắt AQE (1.20 s) chỉ nhanh hơn SMJ (1.51 s) một chút, có lẽ vì script mặc định chỉ dùng 8 partition nên SMJ ở đây khá nhẹ, cộng thêm dao động giữa các lần chạy (Bảng 6.5). Khác biệt thật nằm ở kế hoạch: broadcast bỏ được `Exchange` và `Sort` ở phía bảng lớn, và khoản này sẽ lớn dần theo kích thước dữ liệu.

6.2.3 E3: data skew, salting và AQE skew join

Shuffle băm theo khóa nên mọi bản ghi của khóa 0 (60 % bảng fact) dồn vào một partition. Ba biến thể được chạy: sort-merge join với AQE tắt, salting với 64 giá trị salt, và AQE skew join. Phân vị thời gian task và số bản ghi shuffle đọc mỗi task tại stage join của từng biến thể được lấy qua REST API (Bảng 6.3, Hình 6.5).

Bảng 6.3. Phân vị min / 25 % / median / 75 % / max tại stage join của E3 (lần chạy thứ hai).

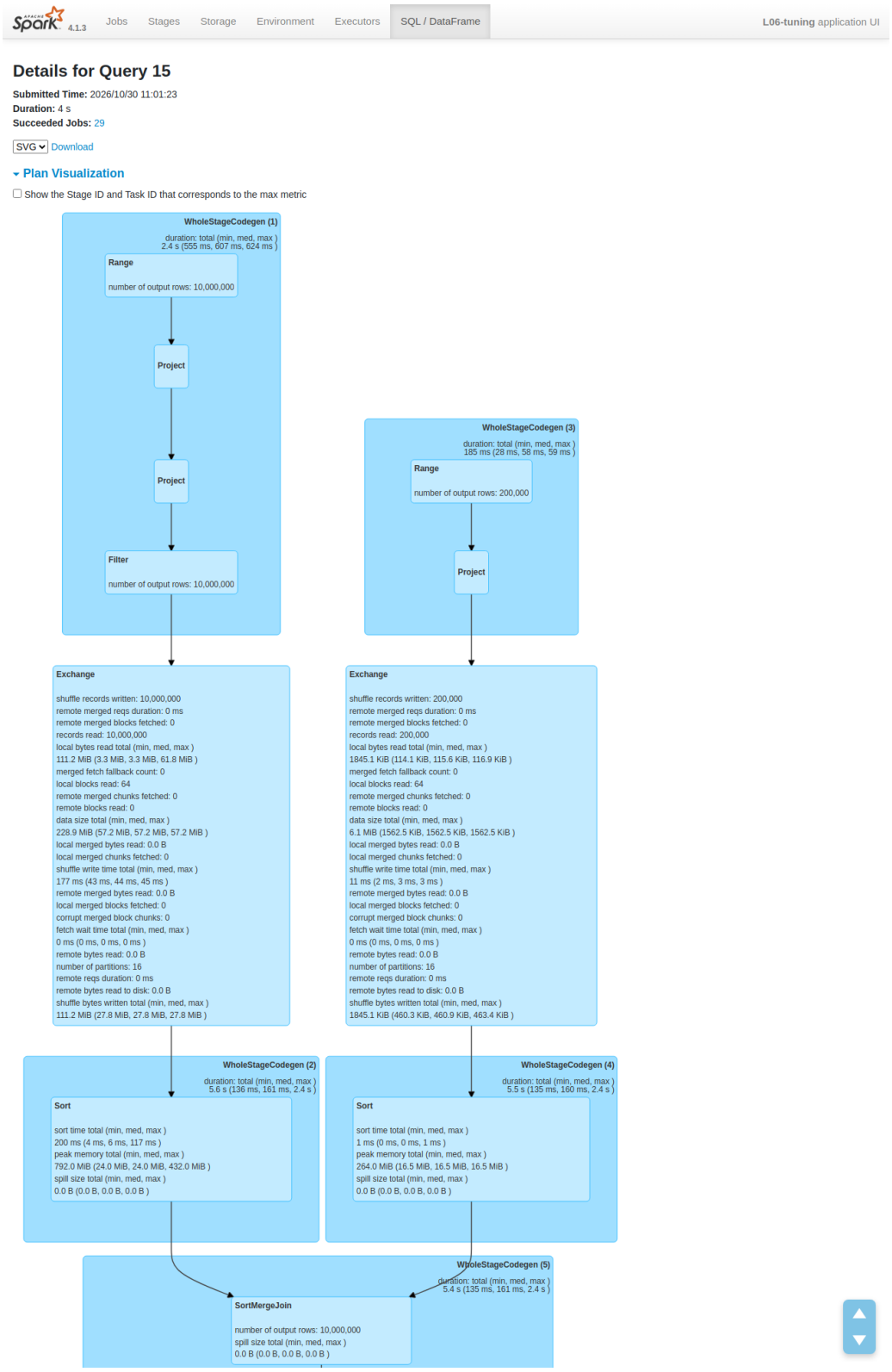
Stage (biến thể)	Thời gian task (ms)	Bản ghi shuffle đọc / task
42: baseline, AQE off	140 / 157 / 165 / 379 / 2,366	260,166 / 261,225 / 262,619 / 264,416 / 6,264,438
46: salting 64	308 / 484 / 590 / 901 / 1,197	359,285 / 543,450 / 638,133 / 823,338 / 917,267
52: AQE skew join	129 / 144 / 186 / 199 / 682	260,166 / 261,225 / 262,677 / 265,121 / 1,576,365

```
E3: task time and shuffle records per task

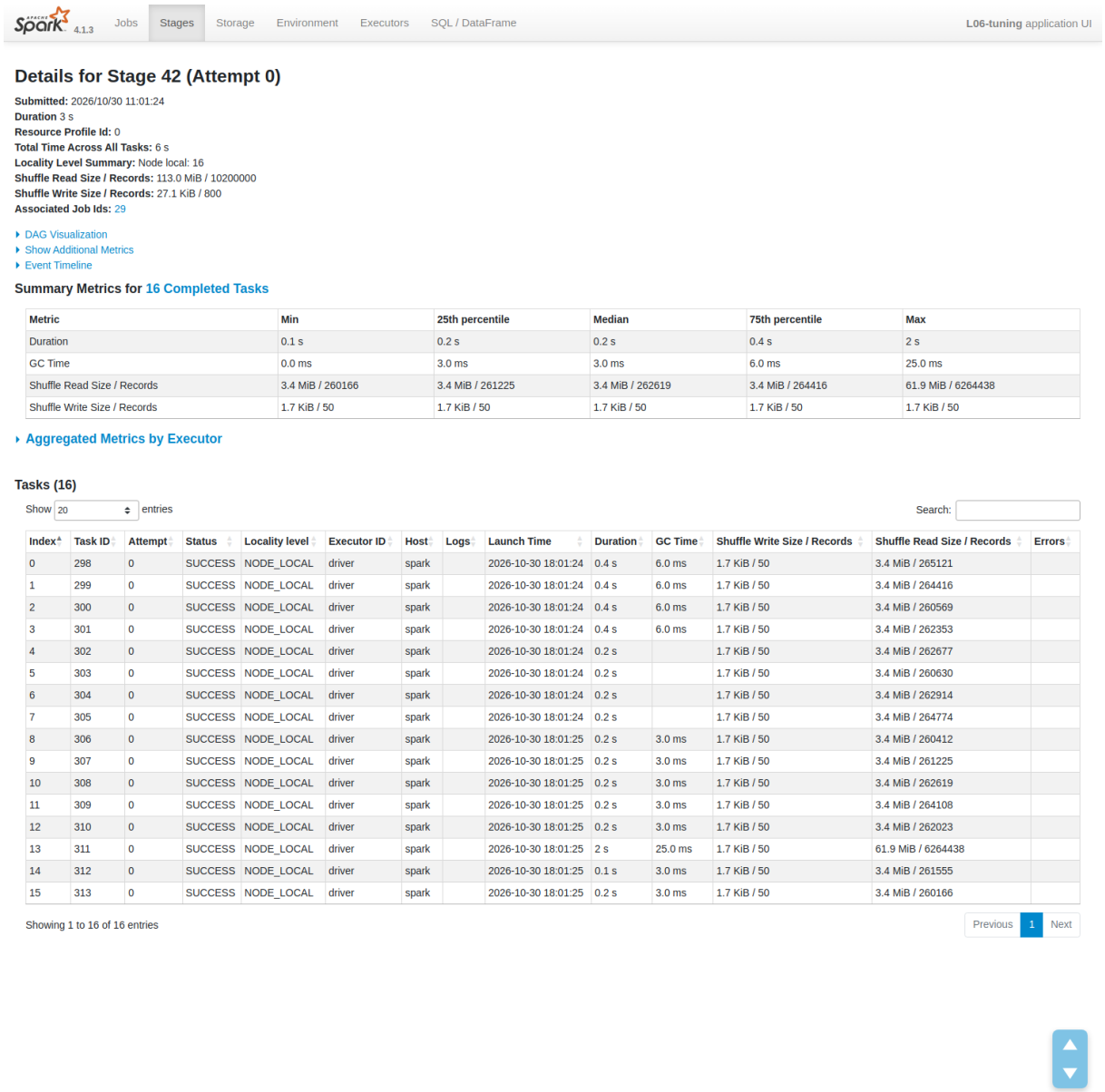
# task quantiles [min, p25, median, p75, max] from /api/v1/.../taskSummary
stage 42: runTime [140.0, 157.0, 165.0, 379.0, 2366.0] shuffleReadRecords [260166.0, 261225.0, 262619.0, 264416.0, 6264438.0]
stage 46: runTime [308.0, 484.0, 590.0, 901.0, 1197.0] shuffleReadRecords [359285.0, 543450.0, 638133.0, 823338.0, 917267.0]
stage 52: runTime [129.0, 144.0, 186.0, 199.0, 682.0] shuffleReadRecords [260166.0, 261225.0, 262677.0, 265121.0, 1576365.0]
```

Hình 6.5. Phân vị thời gian task và số bản ghi shuffle mỗi task của ba stage join E3, lấy từ Spark REST API.

Baseline. Kế hoạch là `Exchange` + `SortMergeJoin` với 16 partition (Hình 6.6). Ở stage 42, task chậm nhất lâu hơn median khoảng 14 lần. Theo Summary Metrics (Hình 6.7), task này đọc 61.9 MiB, trong khi các task khác chỉ khoảng 3.4 MiB. Cả stage đọc 113.0 MiB / 10,200,000 bản ghi qua 16 task. Trong lúc task đó chạy, ba core còn lại không có việc.



Hình 6.6. Kế hoạch E3 baseline (AQE tắt): sort-merge join trên dữ liệu lệch.



Spark 4.1.3 Jobs Stages Storage Environment Executors SQL / DataFrame L06-tuning application UI

Details for Stage 42 (Attempt 0)

Submitted: 2026/10/30 11:01:24
Duration 3 s
Resource Profile Id: 0
Total Time Across All Tasks: 6 s
Locality Level Summary: Node local: 16
Shuffle Read Size / Records: 113.0 MiB / 10200000
Shuffle Write Size / Records: 27.1 KiB / 800
Associated Job Ids: 29

▸ DAG Visualization
▸ Show Additional Metrics
▸ Event Timeline

Summary Metrics for 16 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	0.1 s	0.2 s	0.2 s	0.4 s	2 s
GC Time	0.0 ms	3.0 ms	3.0 ms	6.0 ms	25.0 ms
Shuffle Read Size / Records	3.4 MiB / 260166	3.4 MiB / 261225	3.4 MiB / 262619	3.4 MiB / 264416	61.9 MiB / 6264438
Shuffle Write Size / Records	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50

▸ Aggregated Metrics by Executor

Tasks (16)

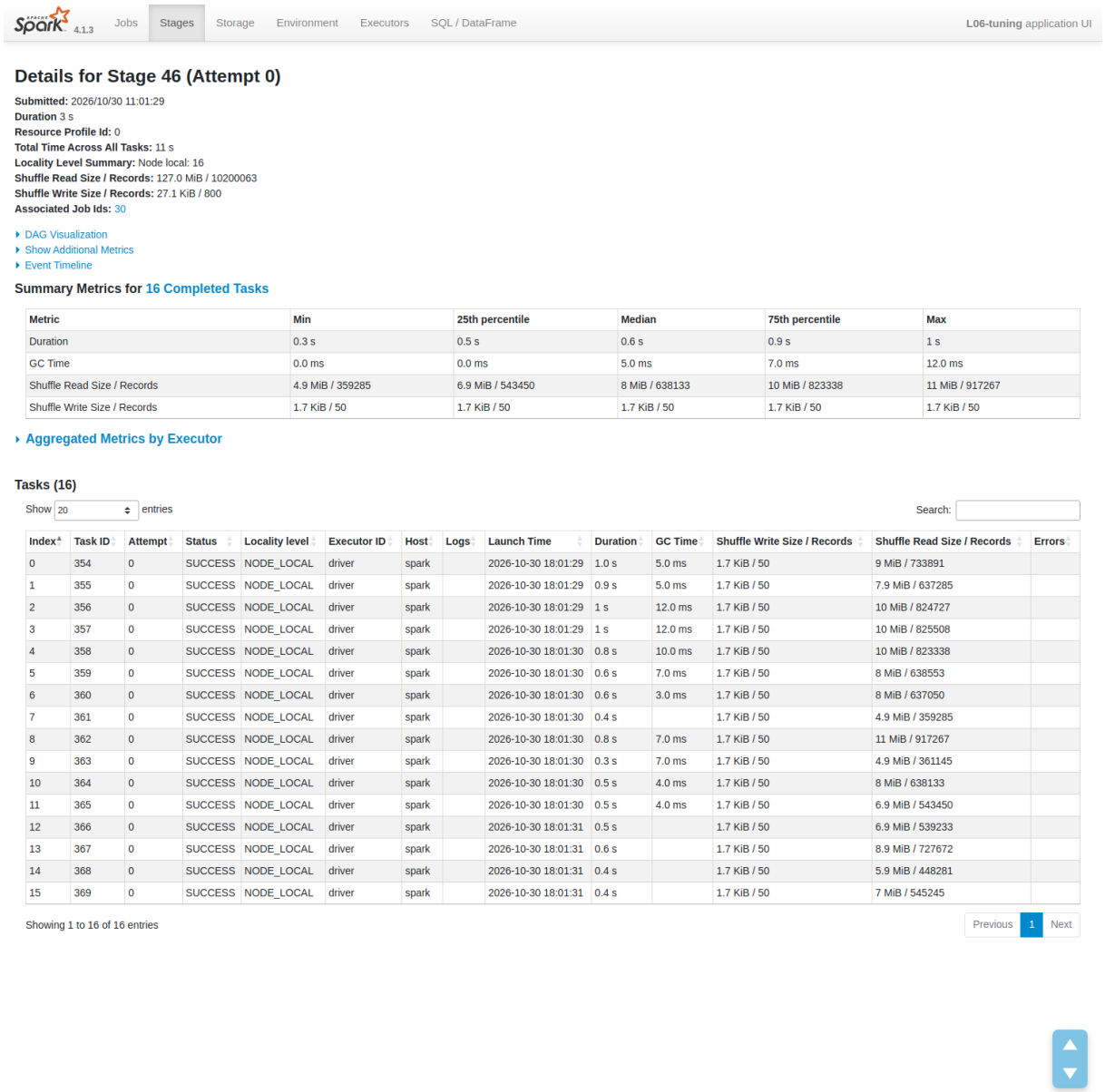
Show 20 entries Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Write Size / Records	Shuffle Read Size / Records	Errors
0	298	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.4 s	6.0 ms	1.7 KiB / 50	3.4 MiB / 265121	
1	299	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.4 s	6.0 ms	1.7 KiB / 50	3.4 MiB / 264416	
2	300	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.4 s	6.0 ms	1.7 KiB / 50	3.4 MiB / 260569	
3	301	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.4 s	6.0 ms	1.7 KiB / 50	3.4 MiB / 262353	
4	302	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.2 s		1.7 KiB / 50	3.4 MiB / 262677	
5	303	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.2 s		1.7 KiB / 50	3.4 MiB / 260630	
6	304	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.2 s		1.7 KiB / 50	3.4 MiB / 262914	
7	305	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:24	0.2 s		1.7 KiB / 50	3.4 MiB / 264774	
8	306	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 260412	
9	307	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 261225	
10	308	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 262619	
11	309	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 264108	
12	310	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 262023	
13	311	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	2 s	25.0 ms	1.7 KiB / 50	61.9 MiB / 6264438	
14	312	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.1 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 261555	
15	313	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:25	0.2 s	3.0 ms	1.7 KiB / 50	3.4 MiB / 260166	

Showing 1 to 16 of 16 entries Previous 1 Next

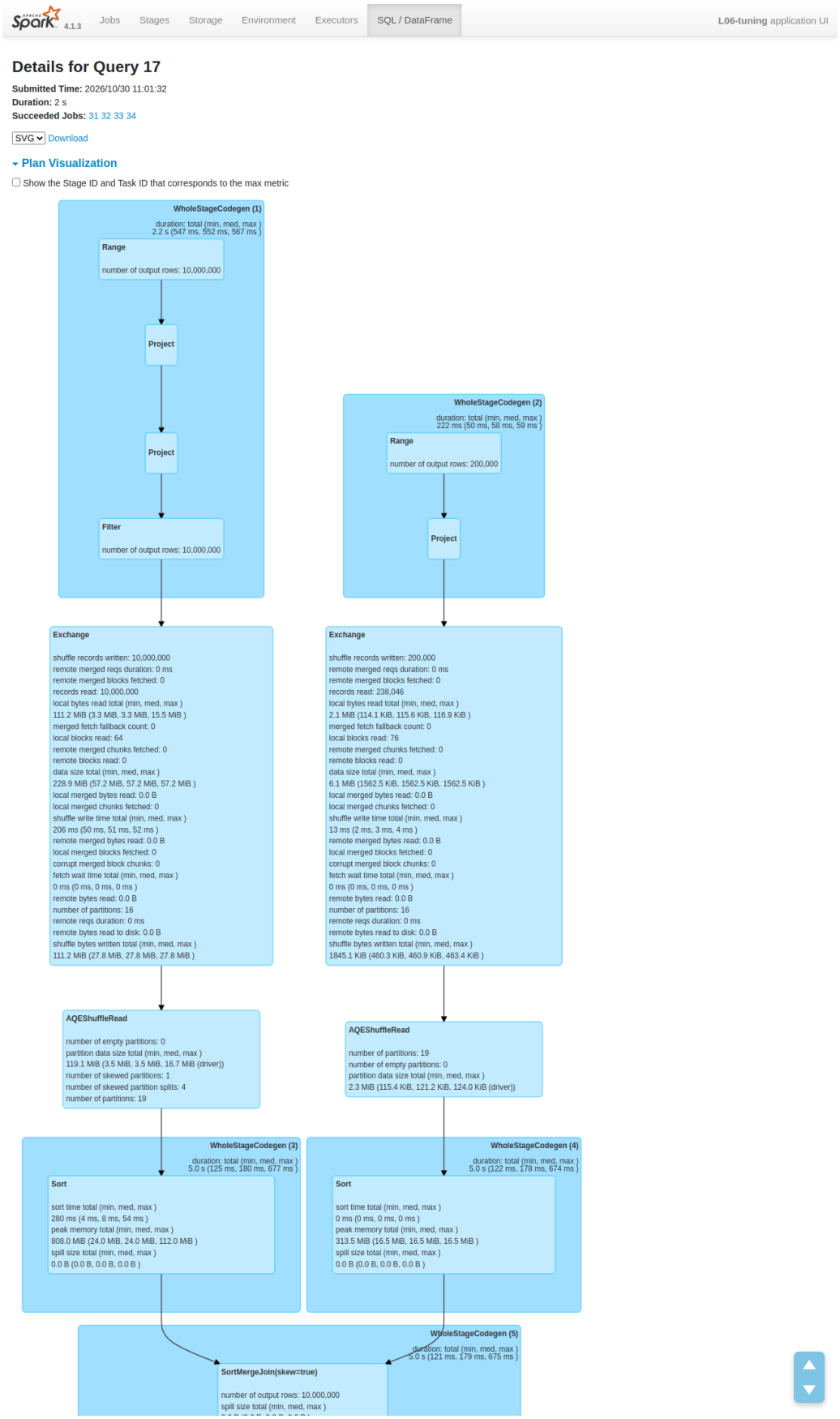
Hình 6.7. Summary Metrics của stage 42 (baseline): max 61.9 MiB / 6,264,438 bản ghi so với median 3.4 MiB / 262,619 bản ghi; task index 13 mất 2 s.

Salting. Chỉ hot key được gắn salt ngẫu nhiên 0..63, dòng dim tương ứng được nhân lên 64 bản, rồi join theo `(key, salt)`. Task max giảm gần một nửa và số bản ghi lớn nhất mỗi task chỉ còn chưa tới một triệu (Hình 6.8). Bù lại, median tăng lên 590 ms (lý do ở Mục 6.3.3) và thời gian tổng gần như đứng yên (4.01 s so với 4.10 s). Tổng `amount` vẫn đúng (500061629), nên salting không làm sai kết quả.

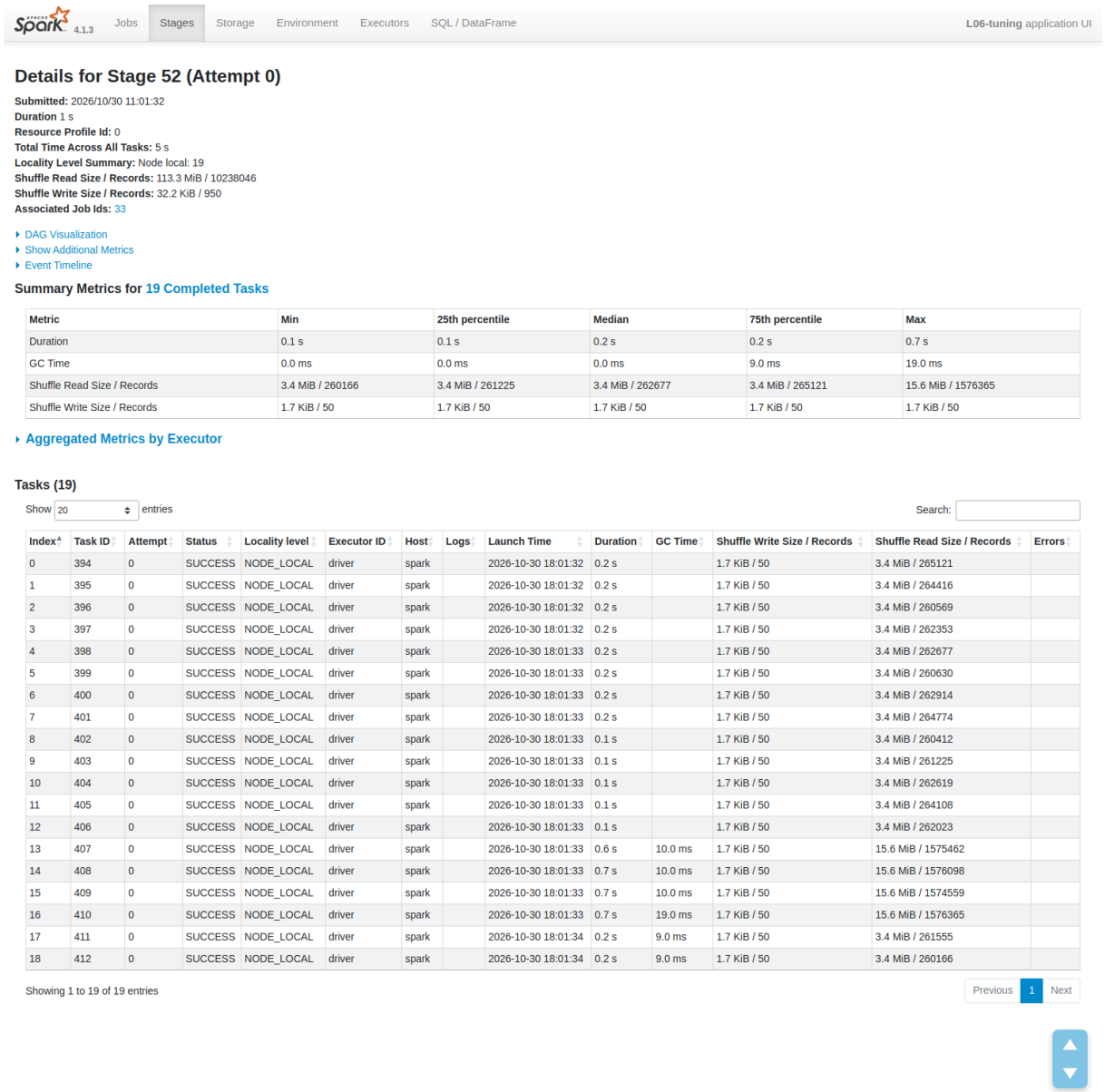


Hình 6.8. Summary Metrics của stage 46 (salting 64): phân bố bản ghi đều hơn, max 917,267 bản ghi, nhưng median thời gian task tăng lên 590 ms.

AQE skew join. Cấu hình bật AQE và skew join, đặt `advisoryPartitionSizeInBytes=4m` , `skewedPartitionThresholdInBytes=16m`, `skewedPartitionFactor=3` và tắt coalesce. Kế hoạch cuối có `SortMergeJoin(skew=true)`, và `AQEShuffleRead` báo *number of skewed partitions: 1, skewed partition splits: 4, number of partitions: 19* (Hình 6.9). Phía fact ghi 10,000,000 bản ghi shuffle (111.2 MiB); sau khi đọc, partition có median 3.5 MiB và max 16.7 MiB. Ở stage 52, số bản ghi lớn nhất mỗi task xấp xỉ một phần tư của baseline, đúng với 4 split (Hình 6.10). Thời gian tổng còn **2.22 s, nhanh gần gấp đôi baseline**, mà không phải sửa dòng truy vấn nào. Lúc đầu chúng tôi để ngưỡng mặc định thì AQE không tách gì, vì partition nóng chỉ có 61.9 MiB, thấp hơn nhiều so với mức 256 MB mặc định, nên phải hạ ngưỡng như trên.



Hình 6.9. Kế hoạch E3 với AQE skew join: `AQEShuffleRead` báo 1 partition lệch được tách thành 4 split (16 thành 19 partition), sau đó là `SortMergeJoin(skew=true)`.



Spark 4.1.3 Jobs Stages Storage Environment Executors SQL / DataFrame L06-tuning application UI

Details for Stage 52 (Attempt 0)

Submitted: 2026/10/30 11:01:32
Duration 1 s
Resource Profile Id: 0
Total Time Across All Tasks: 5 s
Locality Level Summary: Node local: 19
Shuffle Read Size / Records: 113.3 MiB / 10238046
Shuffle Write Size / Records: 32.2 KiB / 950
Associated Job Ids: 33

▸ DAG Visualization
▸ Show Additional Metrics
▸ Event Timeline

Summary Metrics for 19 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	0.1 s	0.1 s	0.2 s	0.2 s	0.7 s
GC Time	0.0 ms	0.0 ms	0.0 ms	9.0 ms	19.0 ms
Shuffle Read Size / Records	3.4 MiB / 260166	3.4 MiB / 261225	3.4 MiB / 262677	3.4 MiB / 265121	15.6 MiB / 1576365
Shuffle Write Size / Records	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50

▸ Aggregated Metrics by Executor

Tasks (19)

Show 20 entries Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Write Size / Records	Shuffle Read Size / Records	Errors
0	394	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:32	0.2 s		1.7 KiB / 50	3.4 MiB / 265121	
1	395	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:32	0.2 s		1.7 KiB / 50	3.4 MiB / 264416	
2	396	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:32	0.2 s		1.7 KiB / 50	3.4 MiB / 260569	
3	397	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:32	0.2 s		1.7 KiB / 50	3.4 MiB / 262353	
4	398	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.2 s		1.7 KiB / 50	3.4 MiB / 262677	
5	399	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.2 s		1.7 KiB / 50	3.4 MiB / 260630	
6	400	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.2 s		1.7 KiB / 50	3.4 MiB / 262914	
7	401	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.2 s		1.7 KiB / 50	3.4 MiB / 264774	
8	402	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 260412	
9	403	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 261225	
10	404	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 262619	
11	405	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 264108	
12	406	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.1 s		1.7 KiB / 50	3.4 MiB / 262023	
13	407	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.6 s	10.0 ms	1.7 KiB / 50	15.6 MiB / 1575462	
14	408	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.7 s	10.0 ms	1.7 KiB / 50	15.6 MiB / 1576098	
15	409	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.7 s	10.0 ms	1.7 KiB / 50	15.6 MiB / 1574559	
16	410	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:33	0.7 s	19.0 ms	1.7 KiB / 50	15.6 MiB / 1576365	
17	411	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:34	0.2 s	9.0 ms	1.7 KiB / 50	3.4 MiB / 261555	
18	412	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:34	0.2 s	9.0 ms	1.7 KiB / 50	3.4 MiB / 260166	

Showing 1 to 19 of 19 entries Previous 1 Next

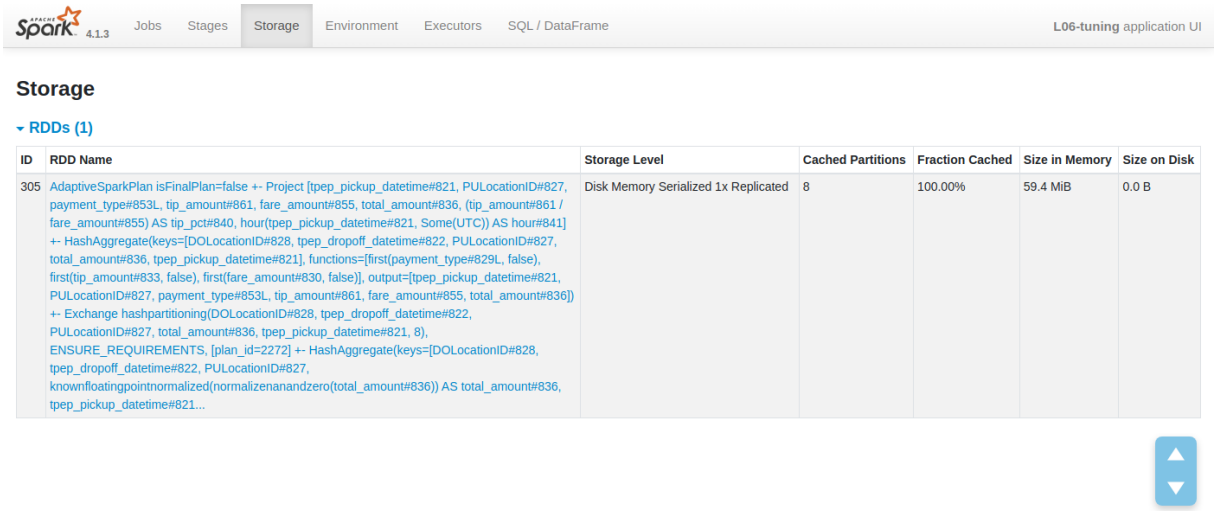
Hình 6.10. Summary Metrics của stage 52 (AQE skew join): max 682 ms và 1,576,365 bản ghi mỗi task.

6.2.4 E4: repartition và coalesce trước khi ghi

Trips được ghi ra Parquet theo bốn cách: `repartition(64)`, `repartition(4)`, `coalesce(4)`, `coalesce(1)`. `repartition(64)` chậm nhất vì vừa có một shuffle đầy đủ vừa ghi 64 file nhỏ. `coalesce(4)` không shuffle nên nhanh nhất, nhưng chỉ ra 3 file chứ không phải 4. Ban đầu chúng tôi nghĩ có lỗi; xem lại thì file Parquet đầu vào chỉ có 3 row group nên một trong 4 partition đầu vào rỗng. `coalesce(1)` dồn cả đọc lẫn ghi vào một task và mất hết song song, còn `repartition(4)` cho 4 file đều nhau với giá một shuffle (Bảng 6.4).

6.2.5 E5: cache

DataFrame "cleaned" gồm một phép lọc và `dropDuplicates` trên năm cột, tức là shuffle toàn bộ khoảng 3 triệu dòng. Nó được dùng cho ba phép tổng hợp (theo giờ, theo `PULocationID`, theo `payment_type`), lần đầu không cache, lần sau với `persist(MEMORY_AND_DISK)`. Không cache thì dedup chạy lại ba lần. Có cache thì `count()` để materialize mất 3.42 s và trả về 2,869,714 dòng, sau đó ba truy vấn chỉ còn **0.99 s**. Tab Storage (Hình 6.11) cho thấy 8/8 partition đã được cache (100 %), chiếm 59.4 MiB bộ nhớ và 0.0 B đĩa, mức `Disk Memory Serialized 1x Replicated`; REST API báo `memoryUsed` = 62,240,847 byte. Tính cả thời gian materialize, cache vẫn lời (khoảng 4.4 s so với 6.4 s).



Hình 6.11. Tab Storage của Spark UI: DataFrame `cleaned` được cache 8/8 partition, 100 %, 59.4 MiB trong bộ nhớ.

6.2.6 Bảng kết quả và độ ổn định

Theo đề, bảng ghi nhận là của lần chạy thứ hai. Hình 6.12 là đầu ra console của lần chạy thứ hai, Hình 6.13 là bảng Markdown script in ra ở cuối, và Bảng 6.4 gom đủ 17 biến thể; hai cột median/max là thời gian task ở stage lệch nhất. Tab SQL của phiên có giữ UI liệt kê 31 truy vấn đã xong (Hình 6.14), chúng tôi dùng danh sách này để tìm kế hoạch của từng biến thể.

```
spark-submit l06_experiments.py (run 2)

Spark 4.1.3 | cores 4 | input ['/datasets/data/taxi/yellow_tripdata_2024-01.parquet']

===== E1: e1_shuffle_partitions
[E1] AQE off, shuffle.partitions=200           2.20 s | tasks=204 skewiest stage: median=6.0 ms max=60.0 ms | Exchange
[E1] AQE off, shuffle.partitions=8             0.33 s | tasks=12 skewiest stage: median=177.0 ms max=178.0 ms | Exchange
[E1] AQE off, shuffle.partitions=1             0.44 s | tasks=5 skewiest stage: median=232.0 ms max=233.0 ms | Exchange
[E1] AQE on,  shuffle.partitions=200           0.53 s | tasks=5 skewiest stage: median=271.0 ms max=274.0 ms | AQEShuffleRead Exchange

===== E2: e2_broadcast_vs_smj
[E2] AQE off, broadcast (threshold 10MB)       1.20 s | tasks=13 skewiest stage: median=626.0 ms max=635.0 ms | BroadcastExchange BroadcastHashJoin
  Exchange
[E2] AQE off, sort-merge (threshold -1)        1.51 s | tasks=21 skewiest stage: median=386.0 ms max=591.0 ms | Exchange SortMergeJoin
[E2] AQE on,  threshold -1 + broadcast() hint  0.40 s | tasks=6 skewiest stage: median=157.0 ms max=168.0 ms | AQEShuffleRead BroadcastExchange
  BroadcastHashJoin Exchange

===== E3: e3_skew
+----------+-------+
|is_hot_key|  count|
+----------+-------+
|      true|5998366|
|     false|4001634|
+----------+-------+

[E3] skewed SMJ, AQE off                       4.10 s | tasks=40 skewiest stage: median=165.0 ms max=2366.0 ms | Exchange SortMergeJoin
[E3] salted SMJ (salt=64), AQE off             4.01 s | tasks=56 skewiest stage: median=590.0 ms max=1197.0 ms | Exchange SortMergeJoin
[E3] skewed SMJ, AQE skewJoin on               2.22 s | tasks=27 skewiest stage: median=186.0 ms max=682.0 ms | AQEShuffleRead Exchange
  SortMergeJoin(skew=true)
same result with and without salting: True (500061629.0)

===== E4: e4_repartition_vs_coalesce
[E4] repartition(64)                         7.25 s | files=64 | tasks=68 | Exchange
[E4] repartition(4)                          4.40 s | files=4 | tasks=8 | Exchange
[E4] coalesce(4)                             3.12 s | files=3 | tasks=4 | Coalesce
[E4] coalesce(1)                             6.95 s | files=1 | tasks=1 | Coalesce

===== E5: e5_cache
[E5] no cache                                6.40 s
[E5] with cache (MEMORY_AND_DISK)            0.99 s
```

Hình 6.12. Đầu ra console của lần chạy thứ hai: thời gian, số task, median/max task và các nút kế hoạch của từng biến thể.

```
results table (run 2)

 experiment | variant | seconds | total tasks | task median ms* | task max ms* | plan (final, after AQE)
---|---|---|---|---|---|---
 E1 | AQE off, shuffle.partitions=200 | 2.2 | 204 | 6.0 | 60.0 | Exchange
 E1 | AQE off, shuffle.partitions=8 | 0.33 | 12 | 177.0 | 178.0 | Exchange
 E1 | AQE off, shuffle.partitions=1 | 0.44 | 5 | 232.0 | 233.0 | Exchange
 E1 | AQE on,  shuffle.partitions=200 | 0.53 | 5 | 271.0 | 274.0 | AQEShuffleRead Exchange
 E2 | AQE off, broadcast (threshold 10MB) | 1.2 | 13 | 626.0 | 635.0 | BroadcastExchange BroadcastHashJoin Exchange
 E2 | AQE off, sort-merge (threshold -1) | 1.51 | 21 | 386.0 | 591.0 | Exchange SortMergeJoin
 E2 | AQE on,  threshold -1 + broadcast() hint | 0.4 | 6 | 157.0 | 168.0 | AQEShuffleRead BroadcastExchange BroadcastHashJoin Exchange
 E3 | skewed SMJ, AQE off | 4.1 | 40 | 165.0 | 2366.0 | Exchange SortMergeJoin
 E3 | salted SMJ (salt=64), AQE off | 4.01 | 56 | 590.0 | 1197.0 | Exchange SortMergeJoin
 E3 | skewed SMJ, AQE skewJoin on | 2.22 | 27 | 186.0 | 682.0 | AQEShuffleRead Exchange SortMergeJoin(skew=true)
 E4 | repartition(64) | 7.25 | 68 | 152.0 | 980.0 | Exchange
 E4 | repartition(4) | 4.4 | 8 | 2185.0 | 2210.0 | Exchange
 E4 | coalesce(4) | 3.12 | 4 | 2907.0 | 2970.0 | Coalesce
 E4 | coalesce(1) | 6.95 | 1 | 6800.0 | 6800.0 | Coalesce
 E5 | no cache | 6.4 |  |  |  |
 E5 | cache materialisation (count) | 3.42 |  |  |  |
 E5 | with cache (MEMORY_AND_DISK) | 0.99 |  |  |  |
* median / max task run time of the most skewed stage of that query
```

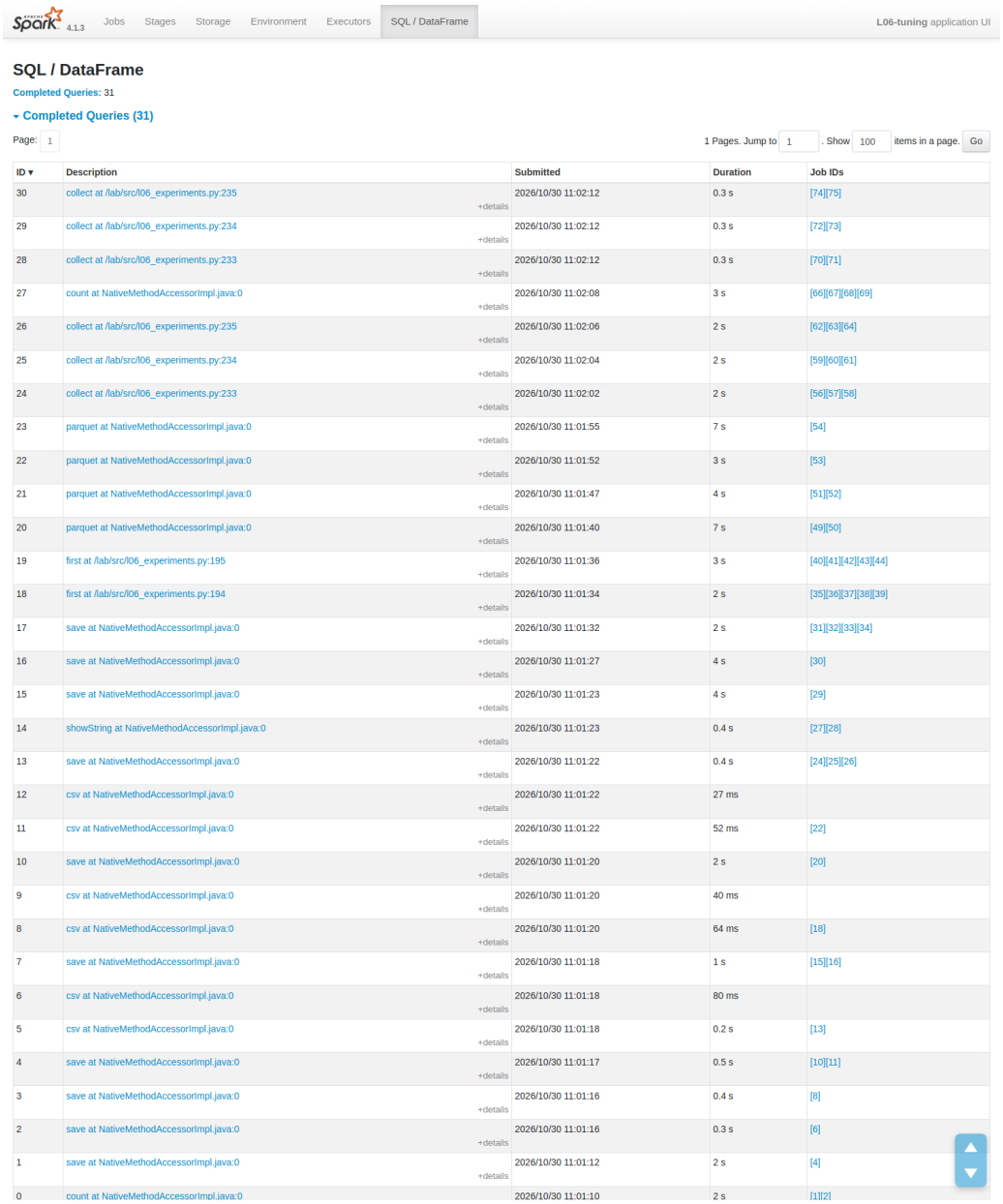
Hình 6.13. Bảng kết quả Markdown do script in ra ở cuối lần chạy thứ hai.

Bảng 6.4. Kết quả đầy đủ của lần chạy thứ hai (17 biến thể).

Thí nghiệm	Biến thể	Thời gian (s)	Tổng task	Median (ms)	Max (ms)	Kế hoạch cuối	Ghi chú
E1	AQE off, shuffle.partitions=200	2.20	204	6	60	Exchange	

Thí nghiệm	Biến thể	Thời gian (s)	Tổng task	Median (ms)	Max (ms)	Kế hoạch cuối	Ghi chú
E1	AQE off, shuffle.partitions=8	0.33	12	177	178	Exchange	
E1	AQE off, shuffle.partitions=1	0.44	5	232	233	Exchange	
E1	AQE on, shuffle.partitions=200	0.53	5	271	274	AQEShuffleRead Exchange	AQE đã gộp partition nhỏ
E2	AQE off, broadcast (threshold 10MB)	1.20	13	626	635	BroadcastExchange BroadcastHashJoin Exchange	
E2	AQE off, sort-merge (threshold -1)	1.51	21	386	591	Exchange SortMergeJoin	
E2	AQE on, threshold -1 + broadcast() hint	0.40	6	157	168	AQEShuffleRead BroadcastExchange BroadcastHashJoin Exchange	Read chắc chắn thỏa threshold
E3	skewed SMJ, AQE off	4.10	40	165	**2366**	Exchange SortMergeJoin	
E3	salted SMJ (salt=64), AQE off	4.01	56	590	1197	Exchange SortMergeJoin	cùng kết quả (500061629)
E3	skewed SMJ, AQE skewJoin on	2.22	27	186	682	AQEShuffleRead Exchange SortMergeJoin(skew)	skewed=true) partition split
E4	repartition(64)	7.25	68	152	980	Exchange	64 file
E4	repartition(4)	1.40	8	2185	2210	Exchange	4 file
E4	coalesce(4)	3.12	4	2907	2970	Coalesce	3 file
E4	coalesce(1)	6.95	1	6800	6800	Coalesce	1 file

Thí nghiệm	Biến thể	Thời gian (s)	Tổng task	Median (ms)	Max (ms)	Kế hoạch cuối	Ghi chú
E5	no cache (3 queries)	6.40					3 aggregation
E5	cache materialisation (count)	3.42					2,869,714 dòng, 59.4 MiB
E5	with cache (3 queries)	0.99					3 aggregation



Hình 6.14. Tab SQL / DataFrame của Spark UI với 31 truy vấn đã hoàn tất trong phiên chạy.

Lần chạy đầu (tổng 71.71 s) cho cùng xu hướng ở mọi thí nghiệm, phần lớn chênh lệch nằm trong khoảng ±20 % mà đề bài cho phép (Bảng 6.5). Ngoại lệ là các biến thể rất ngắn như E2 broadcast tắt AQE và E1 một partition: với truy vấn dưới một giây, vài trăm mili giây dao động đã là tỉ lệ lớn, nên chúng tôi không rút kết luận từ chênh lệch nhỏ giữa các biến thể loại này. Vì chạy local một máy, không có chi phí mạng, cột max/median đáng tin hơn thời gian tuyệt đối.

Bảng 6.5. So sánh một số chỉ số giữa hai lần chạy.

Chỉ số	Lần 1	Lần 2
E1, 200 partition, AQE off	2.15 s	2.20 s
E1, 1 partition, AQE off	0.30 s	0.44 s

Chỉ số	Lần 1	Lần 2
E2, broadcast, AQE off	0.77 s	1.20 s
E3 baseline, task max	2,686 ms	2,366 ms
E3 salting	4.28 s	4.01 s
E3 AQE skew join	2.28 s	2.22 s
E5 không cache	6.07 s	6.40 s
E5 với cache	0.88 s	0.99 s

So với bảng tham chiếu của đề (Apple M-series, Docker 4 GB, `local[4]`), thời gian trên máy chúng tôi cao hơn khoảng 1.3 đến 1.9 lần, có thể do khác phần cứng và thêm lớp WSL2, nhưng mọi xu hướng đều khớp, kể cả việc `coalesce(4)` ra 3 file (Bảng 6.6). Chỗ khác là số task của AQE skew join, vì số partition sau khi tách phụ thuộc kích thước partition thực tế lúc chạy.

Bảng 6.6. So sánh với bảng kết quả tham chiếu của bài lab.

Chỉ số	Tham chiếu	Lần 2
E1, 200 partition, AQE off	1.29 s	2.20 s
E1, AQE on	0.21 s	0.53 s
E2, broadcast hint	0.24 s	0.40 s
E3 baseline (median / max task)	110 / 1,263 ms	165 / 2,366 ms
E3 salting	2.97 s	4.01 s
E3 AQE skew join (thời gian, task)	1.73 s, 43 task	2.22 s, 27 task
E4 coalesce(4)	1.85 s, 3 file	3.12 s, 3 file
E5 không cache / có cache	4.11 / 0.70 s	6.40 / 0.99 s

6.3 Trả lời câu hỏi checkpoint

6.3.1 Vì sao 200 partition chậm, và vì sao không nên đặt 1 partition dù nó chạy nhanh?

Dữ liệu shuffle của E1 chưa tới 1.2 MiB (Mục 6.2.1). Chia ra 200 phần thì mỗi task reduce chỉ làm việc khoảng 6 ms, còn chi phí lập lịch, serialize và khởi tạo của 204 task lớn hơn phần việc thật, nên bản này chậm hơn bản 8 partition khoảng 6.7 lần (Bảng 6.4). Một partition chạy nhanh ở đây chỉ vì shuffle quá nhỏ. Với dữ liệu thật, một reducer phải ôm toàn bộ dữ liệu shuffle: không có song song, dễ spill ra đĩa và dễ OOM. Thực tế nên nhắm khoảng 100–200 MB mỗi partition shuffle, hoặc để AQE tự quyết.

6.3.2 `AQEShuffleRead coalesced` làm gì, và AQE biết gì mà planner tĩnh không biết?

Khi stage map của shuffle chạy xong, AQE có kích thước thật của từng partition shuffle (*map output statistics*). `AQEShuffleRead coalesced` cho một reducer đọc gộp nhiều partition nhỏ liền kề cho tới khi đạt `advisoryPartitionSizeInBytes`; trong E1, 200 partition chỉ còn một task reduce và tổng số task giảm từ 204 xuống 5 (Hình 6.2). Planner tĩnh chỉ có ước lượng dựa trên kích thước file, và ước lượng này hay sai sau filter, join hoặc aggregate.

6.3.3 Con số nào cho thấy skew, vì sao salting không rút ngắn thời gian tổng trên 4 core, và khi nào salting thắng?

Bằng chứng là chênh lệch giữa task max và median ở stage join của baseline: **2,366 ms so với 165 ms**, và task chậm nhất đọc 6,264,438 bản ghi trong khi median là 262,619 (Bảng 6.3, Hình 6.7).

Salting cắt được task max (Hình 6.8) nhưng thời gian tổng gần như không đổi. Một phần vì salting thêm việc ở mọi chỗ: `rand()` cho từng dòng, thêm cột join, nhân dòng dim lên 64 lần. Phần lớn hơn là khoảng 6 triệu dòng hot vẫn phải chia trên 16 partition chạy trên 4 core: tổng khối lượng việc không giảm, chỉ được chia lại, nên median tăng. Lần chạy đầu cũng vậy, salting chỉ nhanh hơn baseline một chút (4.28 s so với 4.62 s), còn bảng tham chiếu của đề thậm chí ghi salting chậm hơn (2.97 s so với 2.45 s). Salting sẽ thắng rõ khi có nhiều core và nhiều partition, vì lúc đó một straggler giữ hàng trăm executor rảnh; nó cũng có lợi khi mỗi dòng xử lý nặng (dòng rộng, join many-to-many, có spill) hoặc khi skew nặng hơn, ví dụ 90 % hot.

6.3.4 AQE nhận ra partition lệch thế nào, xử lý ra sao, và kém salting ở đâu?

Một partition bị coi là lệch khi nó vừa lớn hơn `skewedPartitionFactor` lần median, vừa lớn hơn `skewedPartitionThresholdInBytes` [1]. Mặc định là 5 lần và 256 MB; bài lab hạ xuống 3 lần và 16 MB cho vừa dữ liệu nhỏ. Partition hot ở đây là 61.9 MiB so với median 3.4 MiB, thỏa cả hai điều kiện. AQE tách partition đó theo các khoảng map output (4 split, nên 16 partition thành 19), rồi nhân bản partition tương ứng ở phía còn lại; kế hoạch hiện `SortMergeJoin(skew=true)` (Hình 6.9), và số bản ghi lớn nhất mỗi task xấp xỉ một phần tư baseline (Bảng 6.3).

AQE có mấy giới hạn mà salting không có. Nó chỉ áp dụng cho join có shuffle ở cả hai phía (sort-merge hoặc shuffled hash join), và không tách được phía phải giữ nguyên của outer join. Ngưỡng phải hợp với dữ liệu, nếu không nó sẽ không kích hoạt, đúng như chúng tôi gặp ở E3. Quan trọng nhất, AQE không xử lý skew trong `groupBy` hay skew phát sinh sau join, trong khi salting hoặc aggregate hai pha làm được.

6.3.5 Khi nào không nên broadcast dù một phía "nhỏ"?

Không nên broadcast khi bảng vẫn lớn lúc nằm trong bộ nhớ, vì driver phải collect rồi gửi bản sao tới mọi executor, dễ OOM ở driver hoặc broadcast timeout. Cũng không nên khi thống kê đánh giá thấp kích thước thật (sau filter hoặc UDF), khi join là full outer join hoặc phía cần broadcast lại là phía được giữ của outer join, và khi join chạy lặp lại trên dữ liệu thay đổi nên mỗi lần phải dựng lại broadcast. Bảng zones trong E2 chỉ có 265 dòng và không rơi vào trường hợp nào ở trên, nên broadcast là đúng (Hình 6.3).

6.3.6 `repartition(n)` hay `coalesce(n)`: cái nào shuffle, cái nào tạo file lệch, cái nào làm giảm song song stage trước?

`repartition(n)` luôn shuffle, nhờ vậy partition cân bằng (4 file đều nhau) và stage trước giữ được mức song song, cái giá là thêm một shuffle. `coalesce(n)` gộp partition có sẵn qua phụ thuộc hẹp, không shuffle nên rẻ hơn, nhưng file ra có thể lệch: ở E4 chỉ có 3 file vì đầu vào chỉ có 3 row group. Cũng vì là phụ thuộc hẹp, `coalesce` kéo luôn stage trước về cùng số task [4]; `coalesce(1)` đọc và ghi trong một task duy nhất (Bảng 6.4).

6.3.7 Vì sao cache có ích trong E5, còn cache một DataFrame rẻ thì thường không?

DataFrame được cache trong E5 là kết quả của một bước đắt (`dropDuplicates`, shuffle toàn bộ khoảng 3 triệu dòng) và được dùng lại ba lần, nên cache giúp bước này chỉ chạy một lần thay vì ba (Mục 6.2.5, Hình 6.11). Một phép quét Parquet chỉ lấy vài cột vốn đã rẻ nhờ đọc vectorized, column pruning và predicate pushdown; cache nó chỉ tốn thêm bộ nhớ và thời gian materialize. Nguyên tắc là cache cái gì vừa đắt vừa được dùng lại, xong thì gọi `unpersist()`.

6.3.8 Nên tinh chỉnh gì trước khi thêm phần cứng?

1. Đọc kế hoạch và Spark UI: tìm stage chậm nhất, spill, thời gian GC, và tỉ lệ task max/median để phát hiện skew (như Bảng 6.3).
2. Lọc và chọn cột càng sớm càng tốt: partition pruning, predicate/column pushdown, dùng định dạng cột (Parquet/ORC).
3. Để partition shuffle có kích thước hợp lý: bật AQE (coalesce và skew join), nhắm khoảng 100–200 MB mỗi partition.
4. Chọn đúng kiểu join: broadcast bảng nhỏ; nếu có skew thì dùng AQE skew join, salting, hoặc tách hot key.
5. Tránh UDF Python (dùng hàm built-in hoặc pandas UDF), chỉ cache khi cần, và tránh sinh file nhỏ khi ghi (repartition hoặc compaction).

6.4 Khuyến nghị cho job production trên cluster 20 node

Mỗi khuyến nghị ứng với một thí nghiệm, đặt trong bối cảnh nhiều core hơn hẳn `local[4]`:

1. E1: bật AQE, đặt `shuffle.partitions` ban đầu khoảng 2–3 lần tổng số core (400–800) và để `coalescePartitions` gộp về khoảng 64–128 MB mỗi partition; không đặt cứng 1 hay 8.
2. E2: broadcast bảng dimension nhỏ, có thể nâng `autoBroadcastJoinThreshold` lên khoảng 50–100 MB nếu executor đủ bộ nhớ, dùng `broadcast()` khi thống kê sai.
3. E3: để AQE skew join chạy với ngưỡng mặc định; với hot key đã biết hoặc skew trong `groupBy`/outer join thì dùng salting hoặc tách hot key.
4. E4: ghi bằng `repartition(n)` hoặc `repartition(col)` để mỗi file khoảng 128 MB–1 GB; tránh `coalesce(1)` trên dữ liệu lớn.
5. E5: chỉ `persist(MEMORY_AND_DISK)` kết quả đắt được dùng lại ít nhất hai lần, rồi `unpersist()` ngay.

6.5 Kết luận

Cả năm thí nghiệm đã chạy hai lần, các phần mã tự viết (AQE trong E1, gợi ý broadcast trong E2, salting và AQE skew join trong E3) đều cho kế hoạch như đề yêu cầu. Ngay trên một máy, khác biệt đã đo được: số partition shuffle không hợp làm truy vấn chậm khoảng 6.7 lần và AQE tự sửa được; skew lộ rõ qua tỉ lệ task max/median khoảng 14 lần; AQE skew join giảm gần một nửa thời gian tổng mà không sửa truy vấn, còn salting chỉ chia lại việc khi số core có hạn; `coalesce` rẻ nhưng có thể sinh file lệch; cache chỉ có lợi khi kết quả đắt và được dùng lại.

Tài liệu tham khảo

[1] The Apache Software Foundation, “Performance Tuning – Spark SQL, DataFrames and Datasets,” Apache Spark Documentation. https://spark.apache.org/docs/latest/sql-performance-tuning.html

[2] The Apache Software Foundation, “Monitoring and Instrumentation (REST API),” Apache Spark Documentation. https://spark.apache.org/docs/latest/monitoring.html

[3] M. Armbrust *et al.*, “Spark SQL: Relational Data Processing in Spark,” in *Proc. ACM SIGMOD International Conference on Management of Data*, 2015.

[4] B. Chambers and M. Zaharia, *Spark: The Definitive Guide*. Sebastopol, CA: O’Reilly Media, 2018.

Phụ lục: Trích đoạn mã nguồn

Các tệp nộp kèm được liệt kê ở Bảng 6.7.

Bảng 6.7. Danh mục tệp nộp kèm.

Tệp	Nội dung
`l06_experiments.py`	Script thí nghiệm đã hoàn thành
`results.csv`, `RESULTS.md`	34 dòng (2 lần × 17 biến thể); bảng lần chạy thứ hai và nhận xét
`01_run1.txt`, `02_run2.txt`, `03_e3_task_quantiles.txt`	Log hai lần chạy; phân vị task ba stage join E3

Ở E1, chúng tôi bật lại AQE nhưng giữ 200 partition để xem AQE tự gộp:

```
set_conf(spark__sql__adaptive__enabled=True,
 ↪  spark__sql__shuffle__partitions=200)
run("E1", "AQE on,  shuffle.partitions=200", q(), "AQE coalesces small
 ↪  shuffle partitions")
```

Ở E2, ngưỡng được để ở -1 nhưng gắn gợi ý broadcast cho bảng zones:

```
set_conf(spark__sql__adaptive__enabled=True,
 ↪  spark__sql__autoBroadcastJoinThreshold=-1)
run("E2", "AQE on,  threshold -1 + broadcast() hint",
    trips_df().join(F.broadcast(zones_df()), F.col("PULocationID") ==
 ↪  F.col("LocationID"))
    .groupBy("Borough").agg(F.count("*").alias("trips")),
    "an explicit hint beats the threshold")
```

Salting ở E3 chỉ gắn salt cho hot key; phía dim nhân dòng hot lên `SALT` = 64 bản:

```
hot = F.col("key") == 0
fact_s = fact.withColumn(
    "salt", F.when(hot, (F.rand(seed=5) *
     ↪  SALT).cast("int")).otherwise(F.lit(0)))
salts = spark.range(SALT).select(F.col("id").cast("int").alias("salt"))
dim_s = (dim.filter(~hot).withColumn("salt", F.lit(0))
         .unionByName(dim.filter(hot).crossJoin(salts)))
salted = (fact_s.join(dim_s, ["key", "salt"])
          .groupBy("segment").agg(F.sum("amount").alias("amount")))
run("E3", f"salted SMJ (salt={SALT}), AQE off", salted)
```

Biến thể cuối của E3 bật AQE skew join và hạ ngưỡng cho vừa dữ liệu cỡ lab, đồng thời tắt coalesce để thấy rõ phần tách:

```
set_conf(spark__sql__adaptive__enabled=True,
         spark__sql__adaptive__skewJoin__enabled=True,
         spark__sql__adaptive__coalescePartitions__enabled=False,
         spark__sql__adaptive__advisoryPartitionSizeInBytes="4m",
 ↪  spark__sql__adaptive__skewJoin__skewedPartitionThresholdInBytes="16m",
         spark__sql__adaptive__skewJoin__skewedPartitionFactor=3)
run("E3", "skewed SMJ, AQE skewJoin on",
    fact.join(dim, "key").groupBy("segment").agg(F.sum("amount")),
    "look for AQEShuffleRead (skewed) in the SQL tab")
```